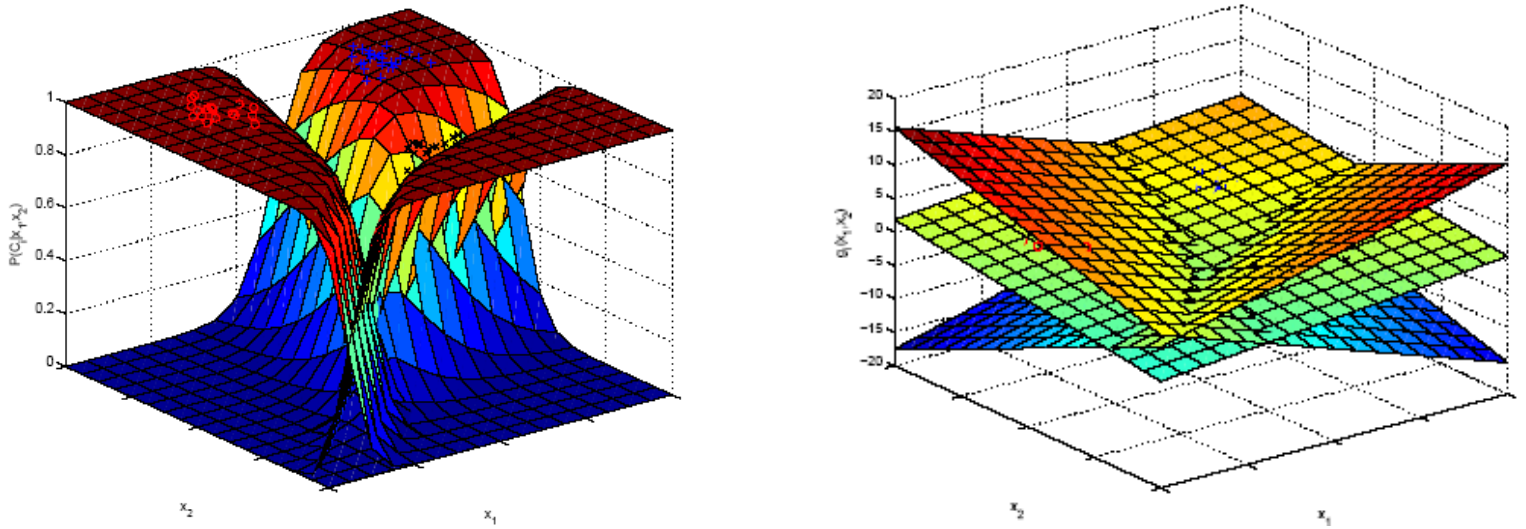
Machine Learning



Lecture 2: Classification & optimisation

Iasonas Kokkinos

Imperial College London

i.kokkinos@ic.ac.uk

Building on Week 1

Last week

A predictor, an objective and fitted parameters.
Likelihood, regularisation and independent evaluation.

This lecture:

Class probabilities and decision boundaries.
Numerical optimisation

Week 2: Classification & optimisation

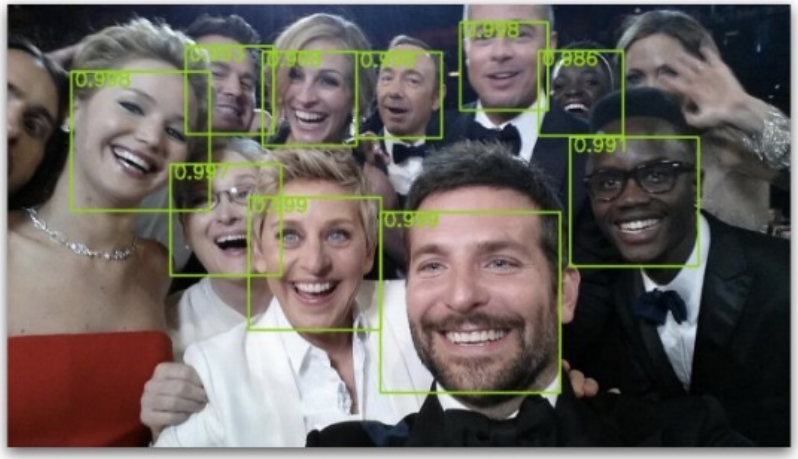
Classification

Problem and classifiers

Scores and logistic loss

Multiclass probabilities

Nonlinear features



Optimisation

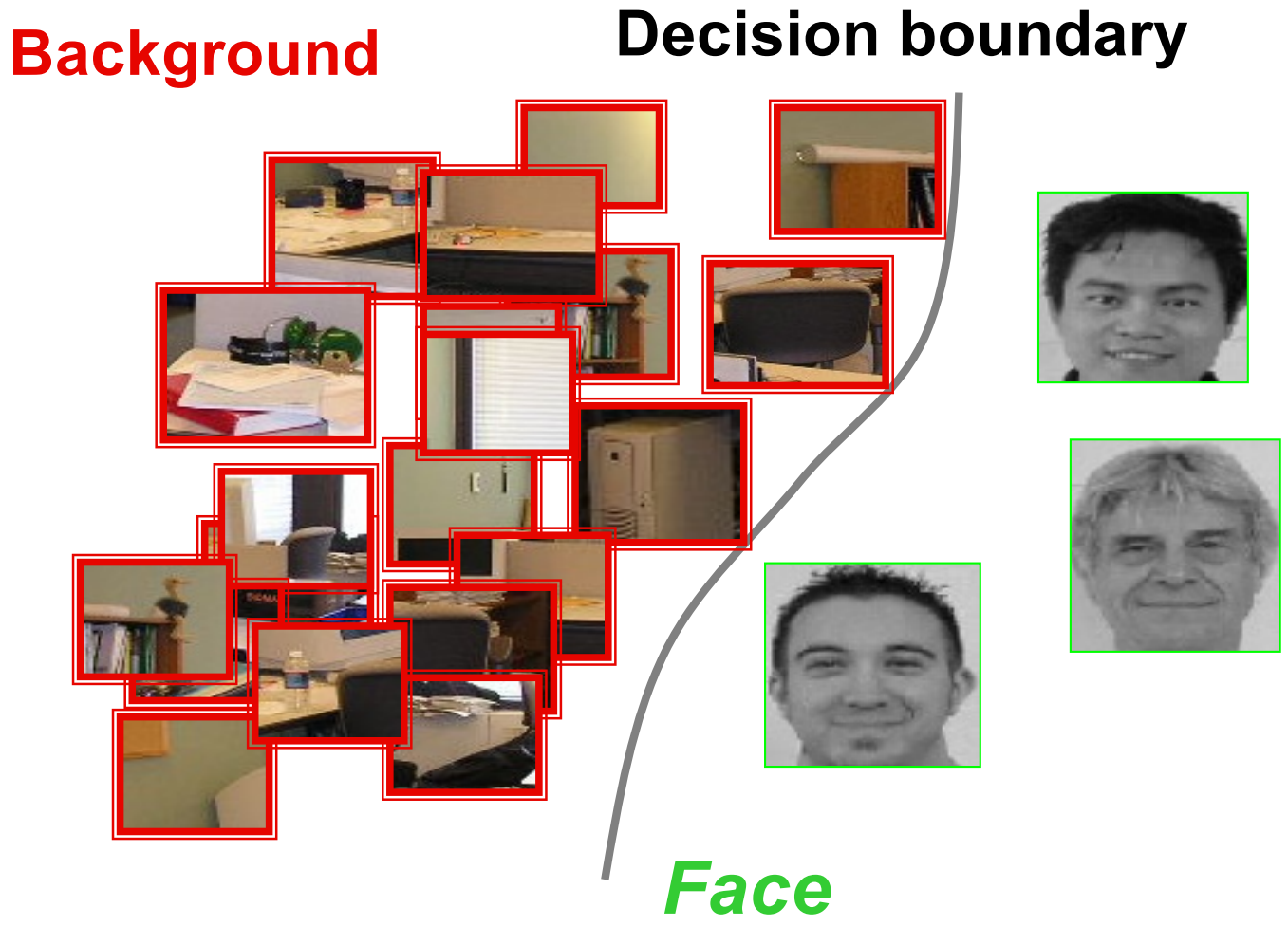
Trees and ensembles

Decisions and evaluation

Murphy, Probabilistic Machine Learning: An Introduction
§1.2.1: classification

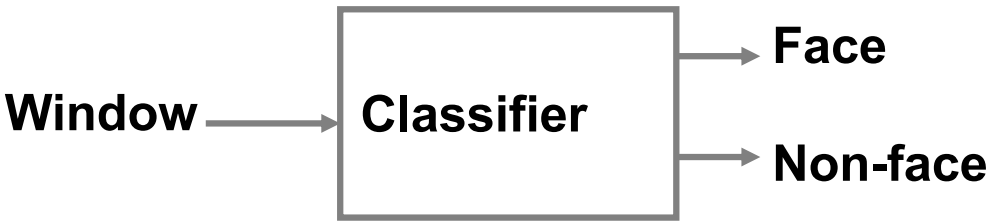
Classification predicts a category

Input: an observed example, such as an image or a message.
Target: a category, such as face / background or spam / legitimate.



Test time: deploy the learned function

- Scan window over image
  - Multiple scales
  - Multiple orientations
- Classify window as either:
  - Face
  - Non-face





Binary training data and label convention

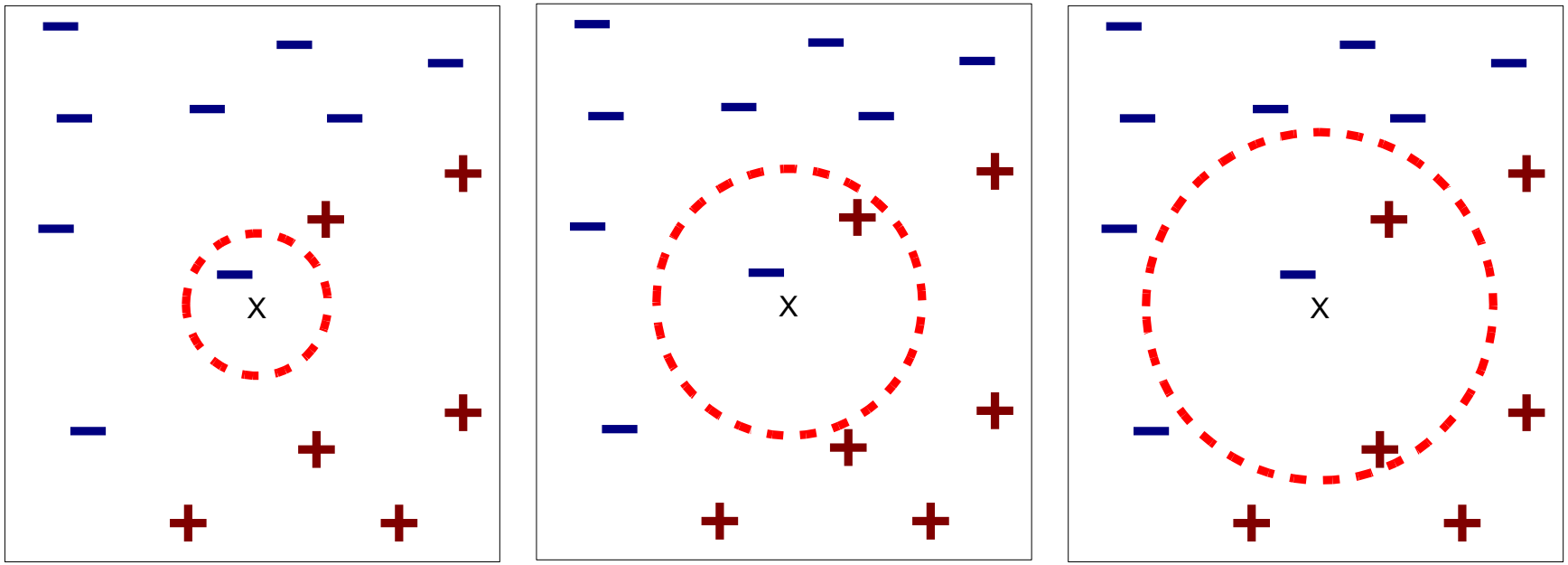
$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}, \qquad y_i \in \{0, 1\}$$

$$\mathbf{x}_i = (1, x_{i1}, \ldots, x_{iD})^T \in \mathbb{R}^{D+1}$$

N is the number of training examples. i indexes an example.
The input has D measured coordinates. y_i is its observed label.

We include the intercept coordinate $x_{i0} = 1$ in the input vector.
Coordinate j runs from 0 to D. Bold symbols denote vectors.

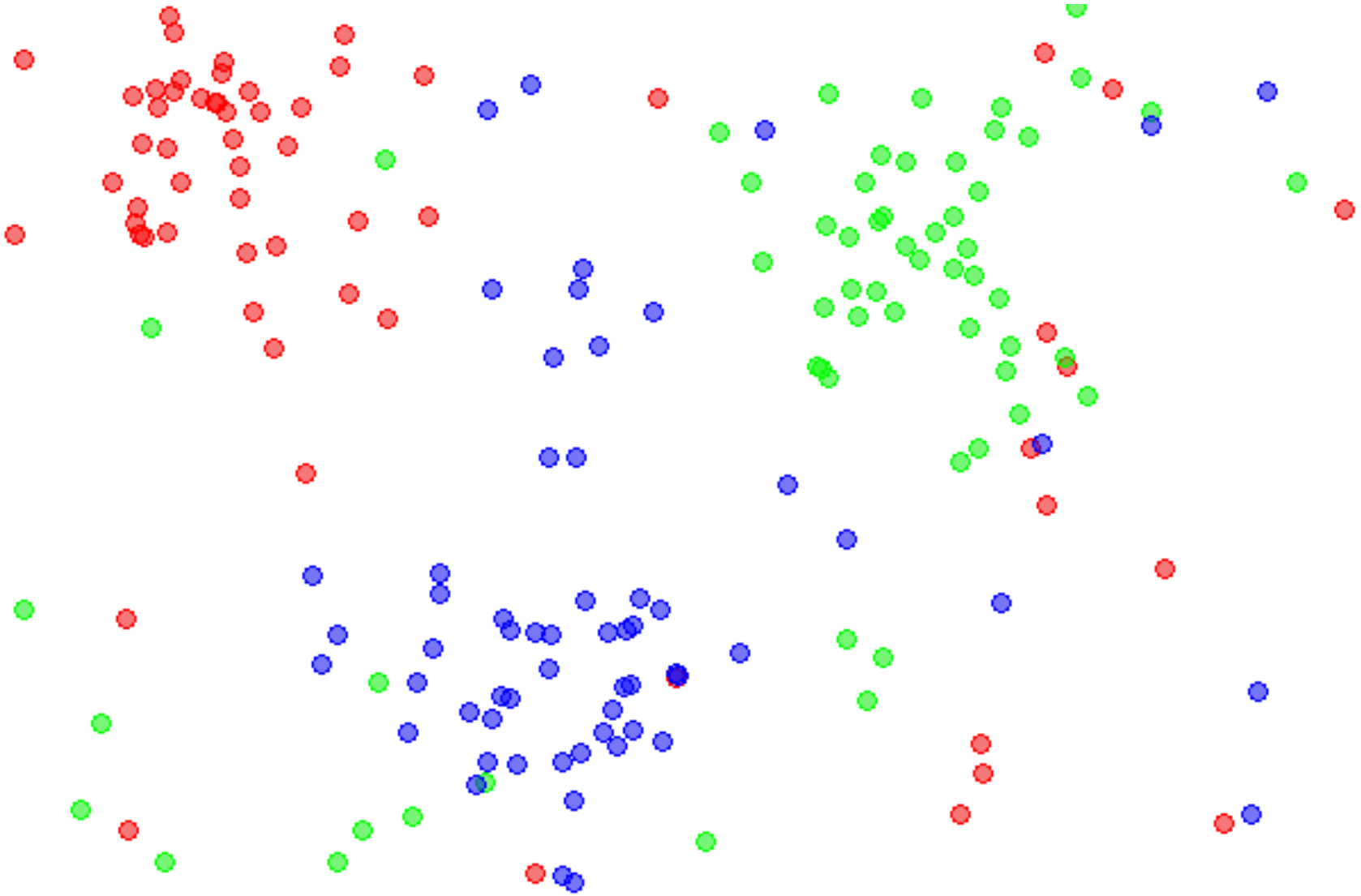
k-nearest-neighbour classification



(a) 1-nearest neighbor (b) 2-nearest neighbor (c) 3-nearest neighbor

k is the number of neighbours; use Euclidean distance here.
Take a majority vote among the k nearest labelled inputs.
× marks a query. Resolve ties using the nearest tied-class neighbour.

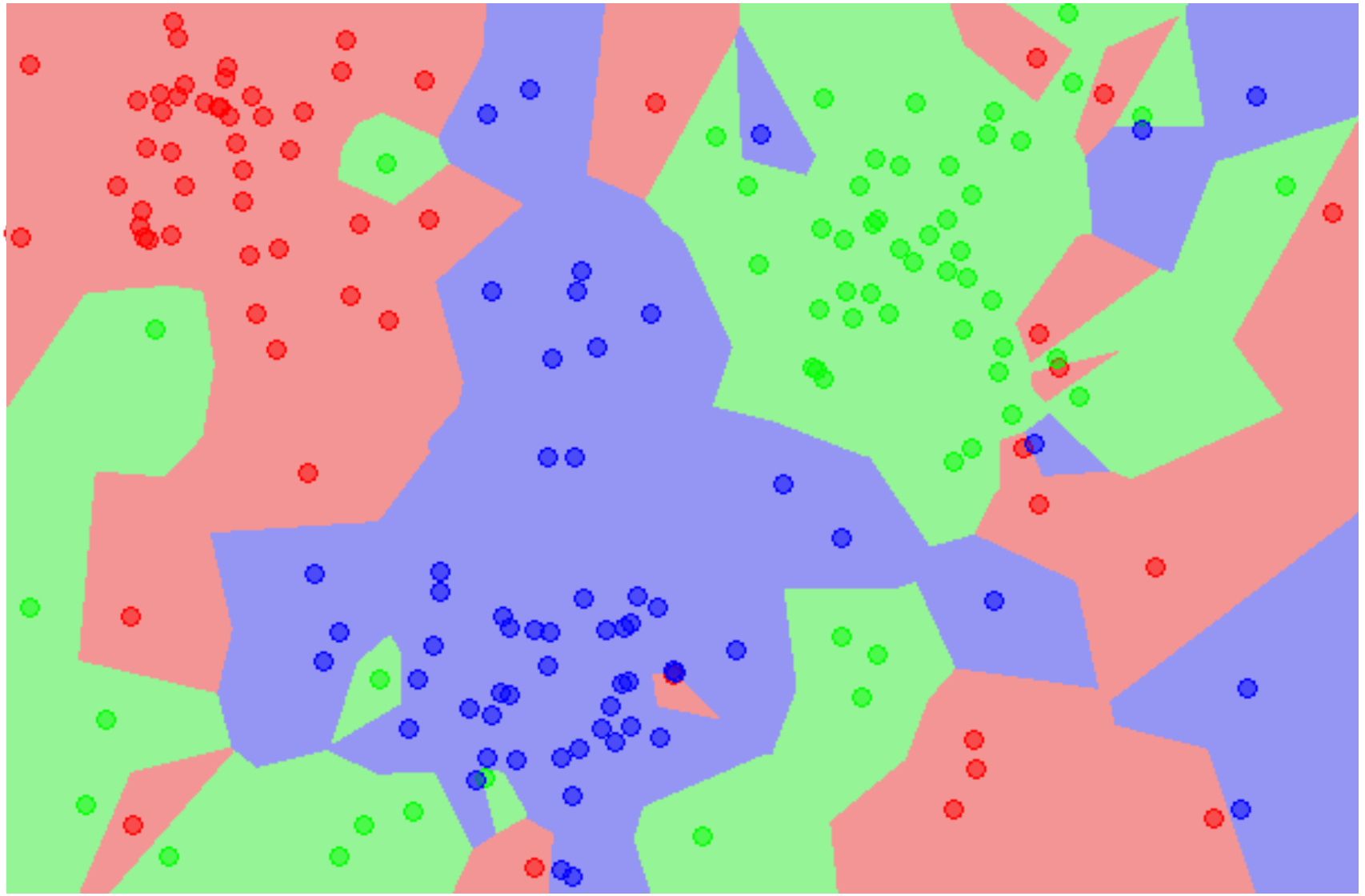
Labelled observations



This nearest-neighbour example has three classes.
Each point is a training input. Colour denotes its class.

Nearest-neighbour decision regions

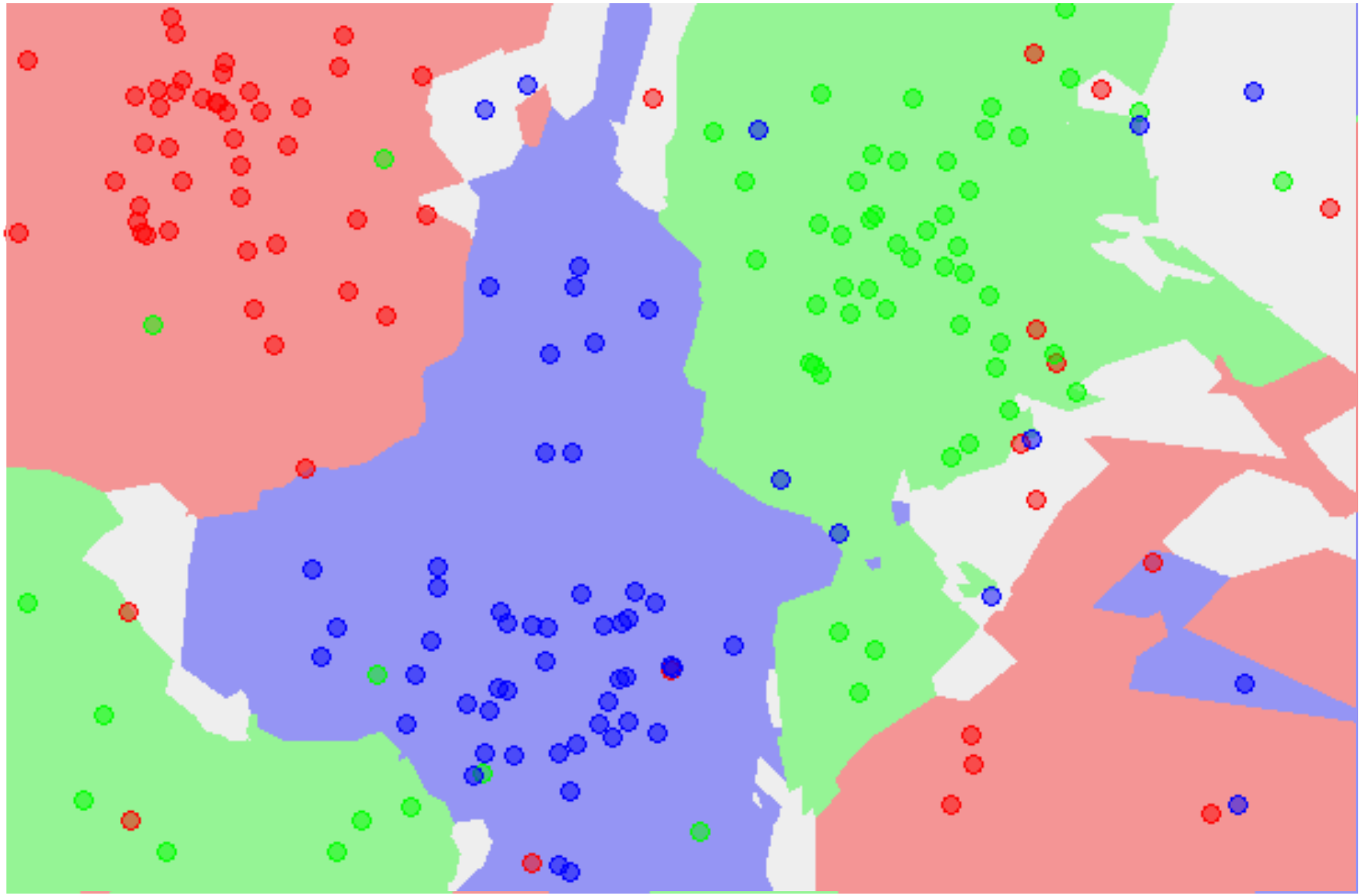
One neighbour



Background colour shows the predicted class. White denotes tied votes

Nearest-neighbour decision regions

Three neighbours



Background colour shows the predicted class. White denotes tied votes.
More neighbours average over a larger local region.

Challenges of k-nearest neighbor classification

Memory Storing the training pairs grows linearly with sample count for a fixed input dimension.

Prediction Access the stored examples. A straightforward full scan grows linearly with sample count. An index may be faster.

Locality The curse of dimensionality: neighbourhoods become sparse. Below: 30 observations, 10 bins per coordinate.

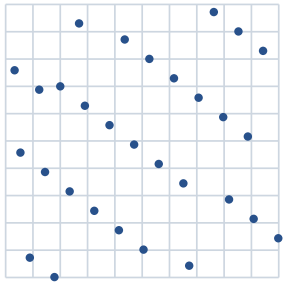
1 dimension



10 cells

3 per cell on average

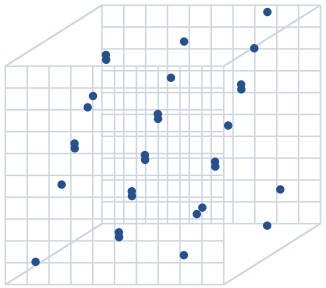
2 dimensions



100 cells

0.3 per cell on average

3 dimensions



1,000 cells

0.03 per cell on average

Lecture 2: Classification & optimisation

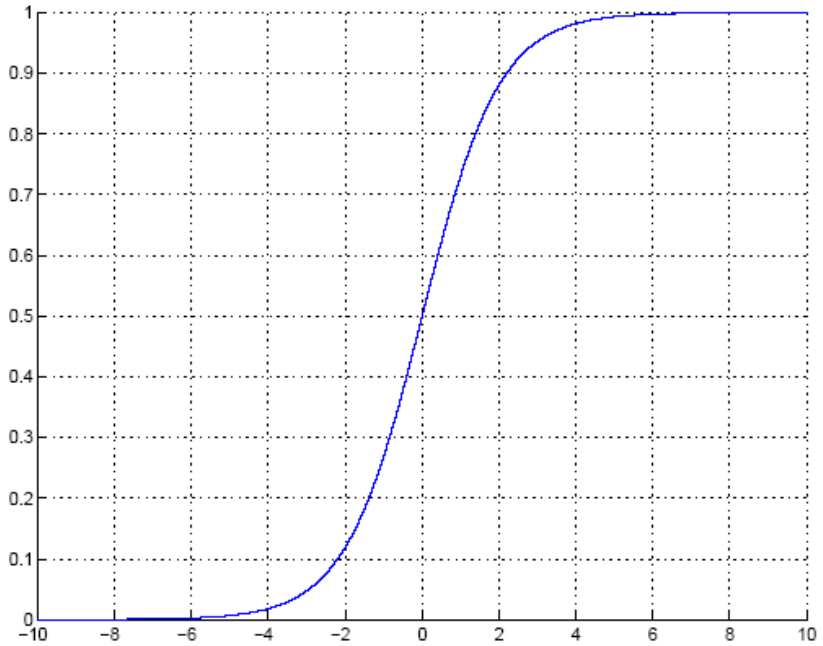
Classification

- Problem and classifiers
- **Scores and logistic loss**
- Multiclass probabilities
- Nonlinear features

Optimisation

Trees and ensembles

Decisions and evaluation



Logistic sigmoid

Murphy, Probabilistic Machine Learning: An Introduction
§§10.2.1–10.2.3: classifiers and maximum likelihood

A linear score and a decision boundary

$$z = f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^T \mathbf{x} = w_0 + \sum_{j=1}^{D} w_j x_j$$

w is the parameter vector. w_0 is the intercept.
z is the scalar score, also called a logit.

$$\hat{y} = \mathbf{1}[z \geq 0], \qquad \mathbf{w}^T \mathbf{x} = 0 \quad \text{on the boundary}$$

ŷ is the predicted label. The indicator 1[condition] is 1 when true.
In two measured dimensions, the boundary is a line.

The margin and classification error

$$y_{\pm} = 2y - 1, \qquad y \in \{0, 1\}, \quad y_{\pm} \in \{-1, +1\}$$

$$m = y_{\pm} z, \qquad z = f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$$

Positive margin: the score has the correct sign.
Negative margin: the classifier makes an error.
At zero, the input is on the decision boundary. Choose a tie rule.

$$\ell_{01}(m) = \begin{cases} 0, & m > 0, \\ 1, & m < 0. \end{cases}$$

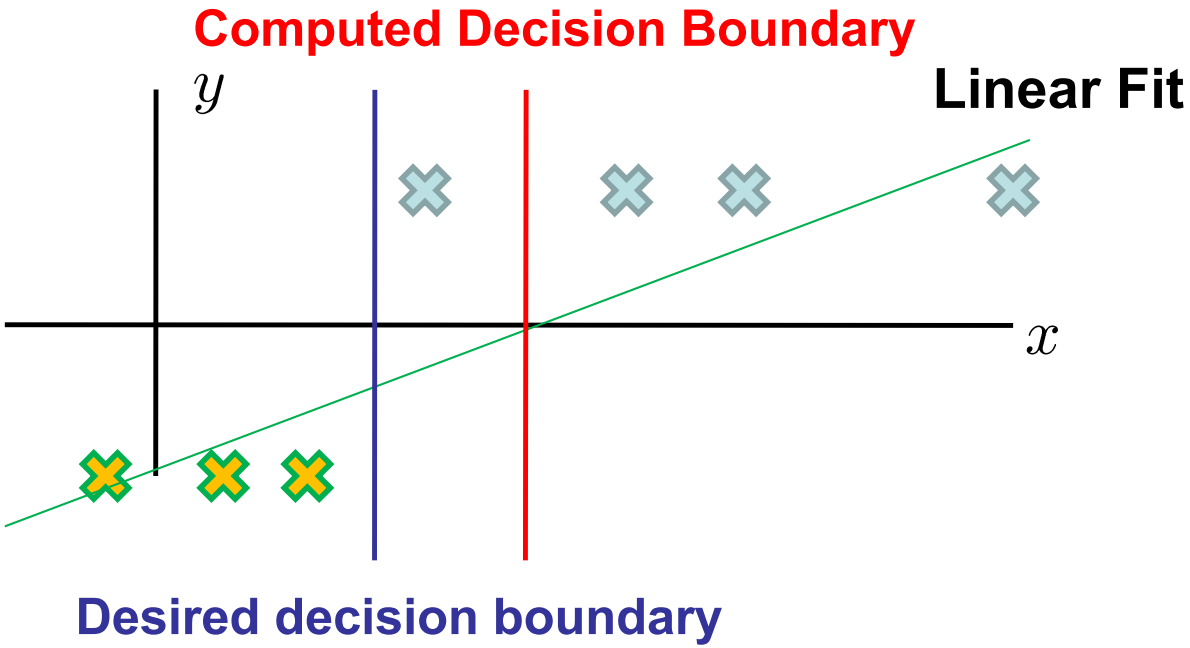
The 0–1 loss is flat away from its jump at zero.
It gives no useful gradient direction, so we use smooth surrogate losses.

Inappropriateness of quadratic loss

We chose the quadratic cost function for convenience

Global minimum & closed-form solution

But does it indicate classification performance?



Rewriting the quadratic loss

Use the signed labels and margin defined earlier.

$$\ell_{\mathrm{sq}} = (y_{\pm} - z)^2$$

$$= y_{\pm}^2 (1 - y_{\pm} z)^2$$

The signed label has square 1.
Its reciprocal equals itself.

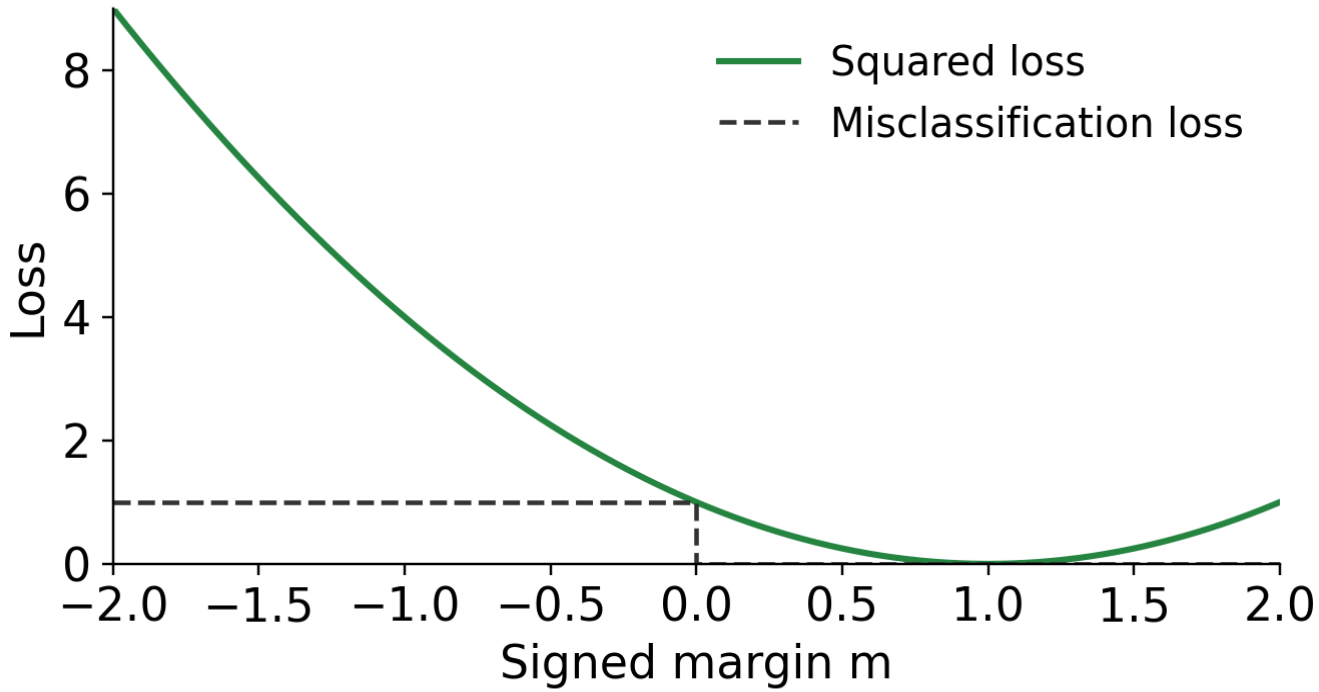
$$= (1 - m)^2$$

The loss depends only on the margin $m = y_{\pm} z$.

Squared loss is smallest at margin 1.
It penalises very confident correct predictions as well as errors.

Quadratic loss and classification margins

$$\ell_{\text{sq}} = (1 - y_{\pm} f_{\mathbf{w}}(\mathbf{x}))^2$$



Squared loss penalises even correct scores when the margin is far above 1.
Large errors can dominate the objective.

Reminder: likelihood for regression

Training set: $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N, \quad y_i \in \mathbb{R}$

$$p(y_1, \ldots, y_N \mid \mathbf{x}_1, \ldots, \mathbf{x}_N; \mathbf{w}) =$$

Conditional independence

$$= \prod_{i=1}^{N} p(y_i \mid \mathbf{x}_i; \mathbf{w})$$

$$= \prod_{i=1}^{N} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(y_i - \mathbf{w}^T\mathbf{x}_i)^2}{2\sigma^2}\right]$$

$$= (2\pi\sigma^2)^{-N/2} \exp\left[-\frac{1}{2\sigma^2}\sum_{i=1}^{N}(y_i - \mathbf{w}^T\mathbf{x}_i)^2\right]$$

Likelihood for classification?

Training set: $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}, \quad y_i \in \{0, 1\}$

$$p(y_1, \ldots, y_N \mid \mathbf{x}_1, \ldots, \mathbf{x}_N; \mathbf{w}) =$$

Conditional independence

$$= \prod_{i=1}^{N} p(y_i \mid \mathbf{x}_i; \mathbf{w})$$

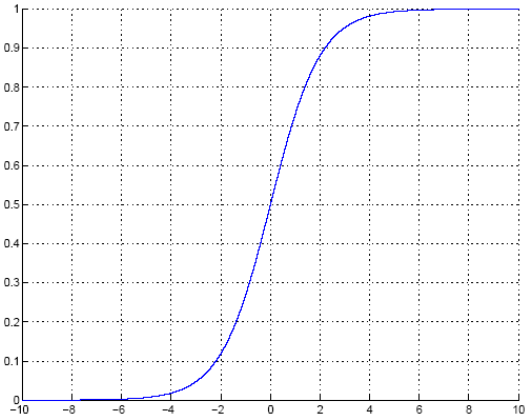
?

The logistic model

Sigmoid function

A real-valued input a is mapped to a value between 0 and 1.

$$g(a) = \frac{1}{1+\exp(-a)} \qquad \log\frac{g(a)}{1-g(a)} = a$$



Sigmoid function

Logistic regression: a linear model for log-odds

$$f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^T\mathbf{x}, \qquad p_{\mathbf{w}}(\mathbf{x}) = g(f_{\mathbf{w}}(\mathbf{x}))$$

Likelihood expression:

$$P(Y = 1 \mid \mathbf{x}; \mathbf{w}) = p_{\mathbf{w}}(\mathbf{x})$$

$$P(Y = 0 \mid \mathbf{x}; \mathbf{w}) = 1 - p_{\mathbf{w}}(\mathbf{x})$$

$$P(Y = y \mid \mathbf{x}; \mathbf{w}) = p_{\mathbf{w}}(\mathbf{x})^y[1 - p_{\mathbf{w}}(\mathbf{x})]^{1-y}$$

Likelihood for binary classification

Training set: $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}, \quad y_i \in \{0, 1\}$

$$p(y_1, \ldots, y_N \mid \mathbf{x}_1, \ldots, \mathbf{x}_N; \mathbf{w}) =$$

$$= \prod_{i=1}^{N} p(y_i \mid \mathbf{x}_i; \mathbf{w})$$

Labels are independent given inputs and w.

$$= \prod_{i=1}^{N} p_i^{y_i} (1 - p_i)^{1 - y_i}$$

$p_i = g(w^T x_i)$

$$\log p(\mathbf{y} \mid \mathbf{X}; \mathbf{w}) = \sum_{i=1}^{N} \log[p_i^{y_i} (1 - p_i)^{1 - y_i}]$$

X stacks input rows x_i^T. y stacks the labels y_i.

$$= \sum_{i=1}^{N} \{y_i \log p_i + (1 - y_i) \log(1 - p_i)\}$$

Binary cross-entropy: the training objective

The Bernoulli negative log-likelihood gives the per-example loss:

$$\ell(y_i, p_i) = -y_i \log p_i - (1 - y_i) \log(1 - p_i)$$

Also known as the log-loss

Q: what does this expression mean?

Add the losses for N independent training examples and fit w:

$$L(\mathbf{w}) = \sum_{i=1}^{N} \ell(y_i, p_i), \qquad \hat{\mathbf{w}} = \arg\min_{\mathbf{w}} L(\mathbf{w})$$

L is the total training loss. ℓ is the loss for one example.
The model supplies $p_i = g(z_i)$.

Log loss as a function of the margin

Use the margin defined earlier, with signed labels $y_{\pm} \in \{-1, +1\}$.

$$P(Y_{\pm} = y_{\pm} \mid \mathbf{x}; \mathbf{w}) = g(m) = \frac{1}{1 + e^{-m}}$$

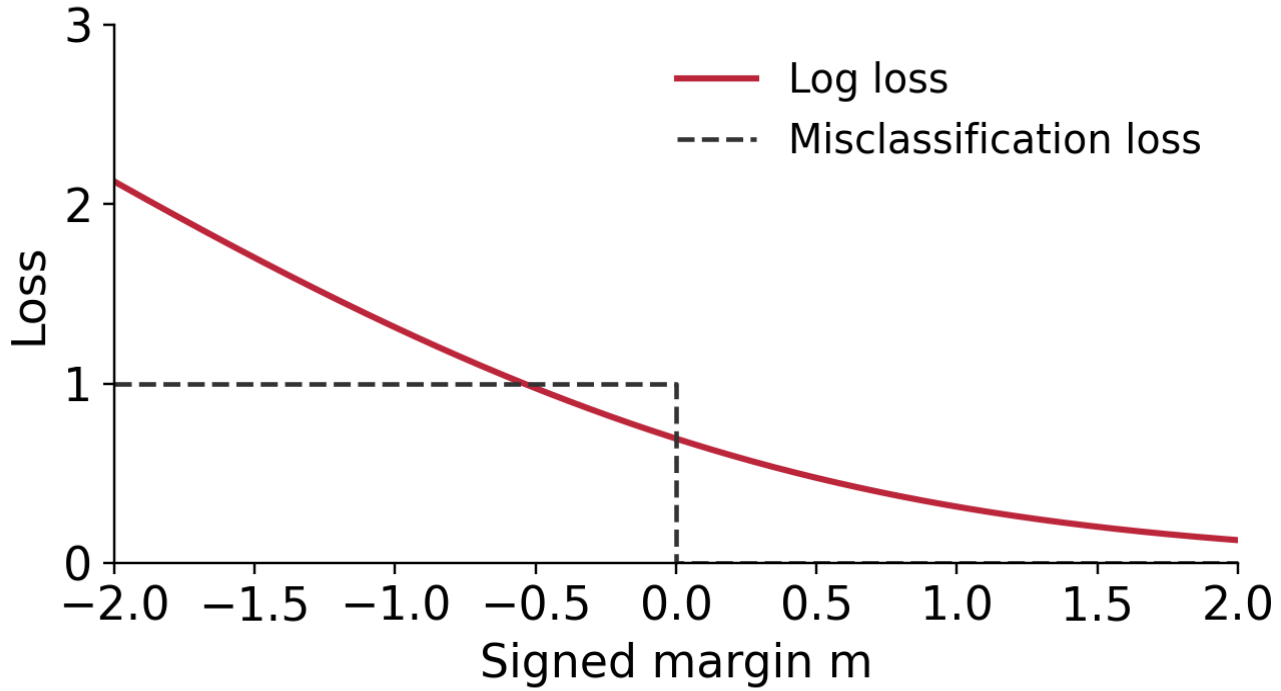
$$\ell_{\log}(m) = -\log g(m) = \log(1 + e^{-m})$$

$$L(\mathbf{w}) = \sum_{i=1}^{N} \log(1 + e^{-m_i}), \quad m_i = y_{i\pm} \mathbf{w}^T \mathbf{x}_i$$

Increasing the margin always reduces logistic loss.
The signed-label probability derivation is in the appendix.

Log loss

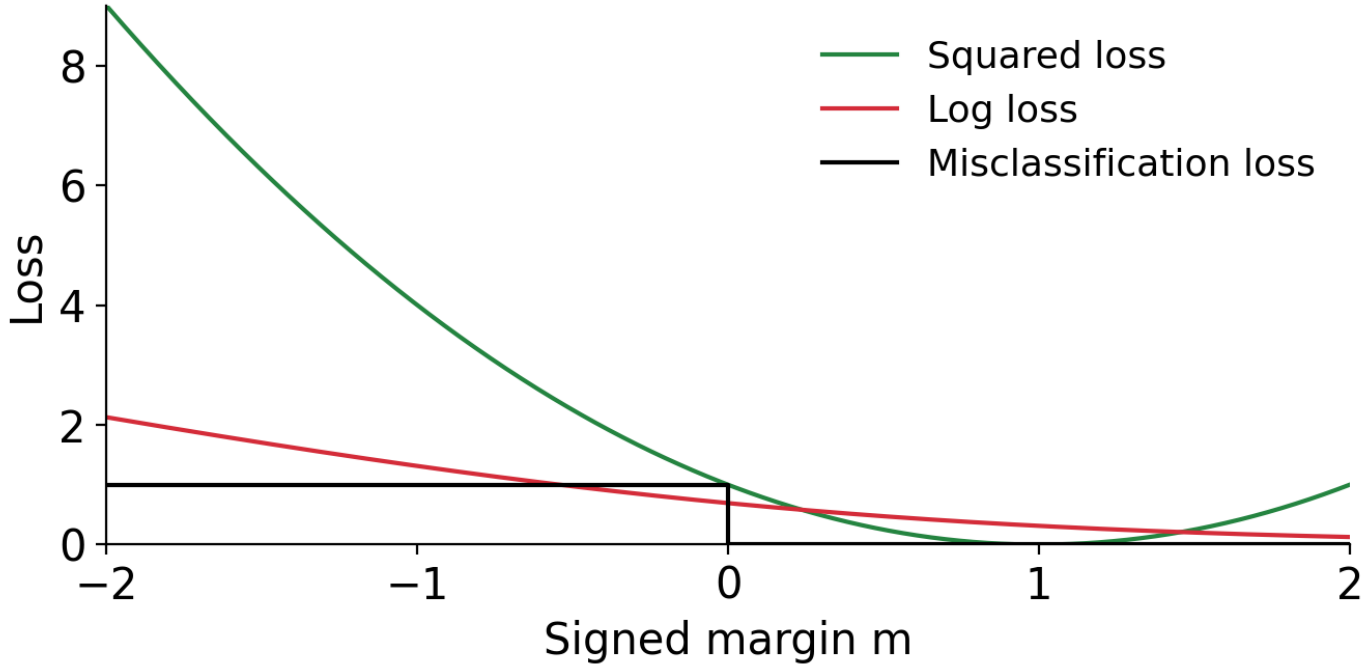
$$\ell_{\log}(m) = -\log g(m) = \log(1 + e^{-m})$$



$$m = y_{\pm} f_{\mathbf{w}}(\mathbf{x}), \qquad y_{\pm} = 2y - 1 \in \{-1, +1\}$$

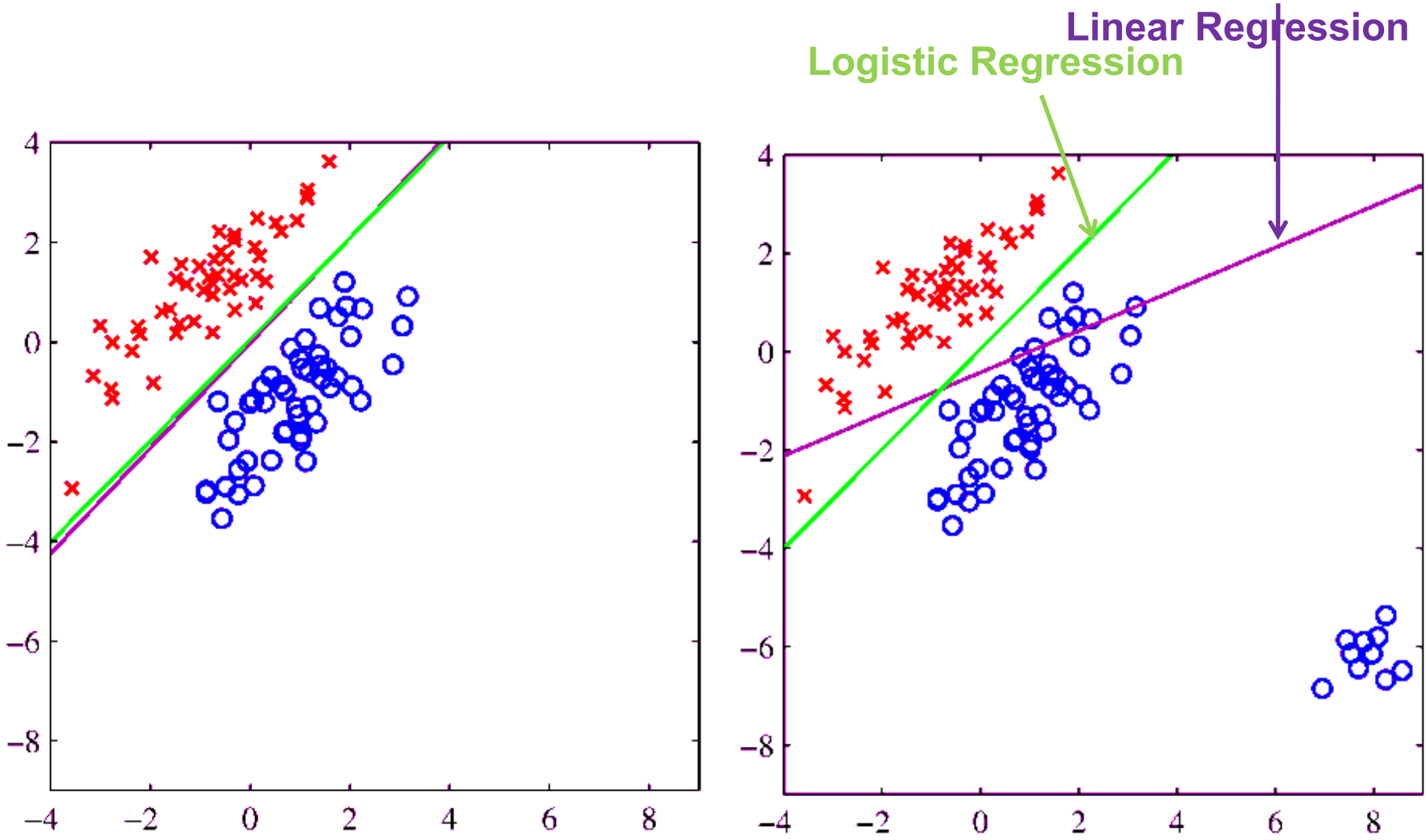
Log loss and squared loss

$$\ell_{\text{sq}} = (1 - m)^2, \qquad \ell_{\log} = \log(1 + \exp(-m))$$



Log loss keeps decreasing; squared loss penalises confident correct scores beyond margin 1.

A distant group changes the fit



Left: original data. Right: a distant group is added.
Squared loss shifts more; the axes are two input features.

Lecture 2: Classification & optimisation

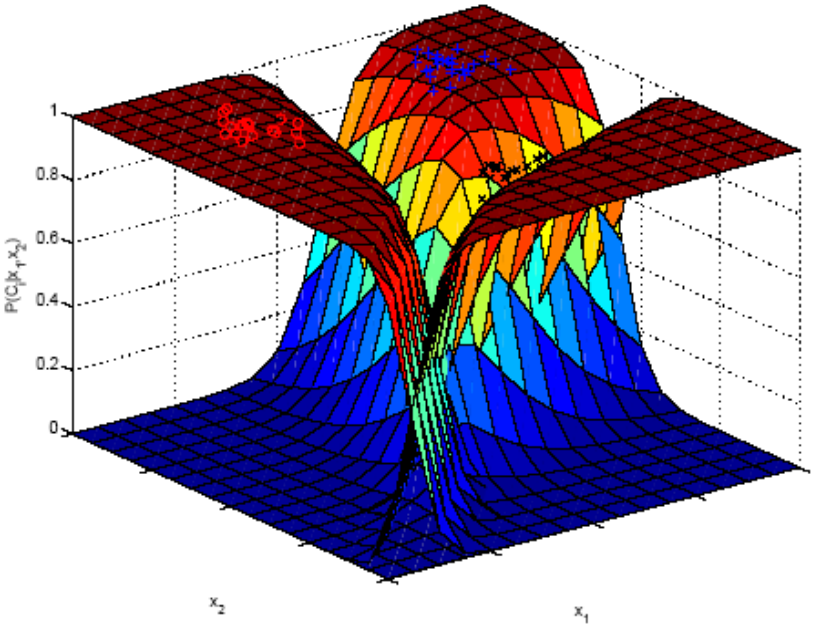
Classification

- Problem and classifiers
- Scores and logistic loss
- **Multiclass probabilities**
- Nonlinear features

Optimisation

Trees and ensembles

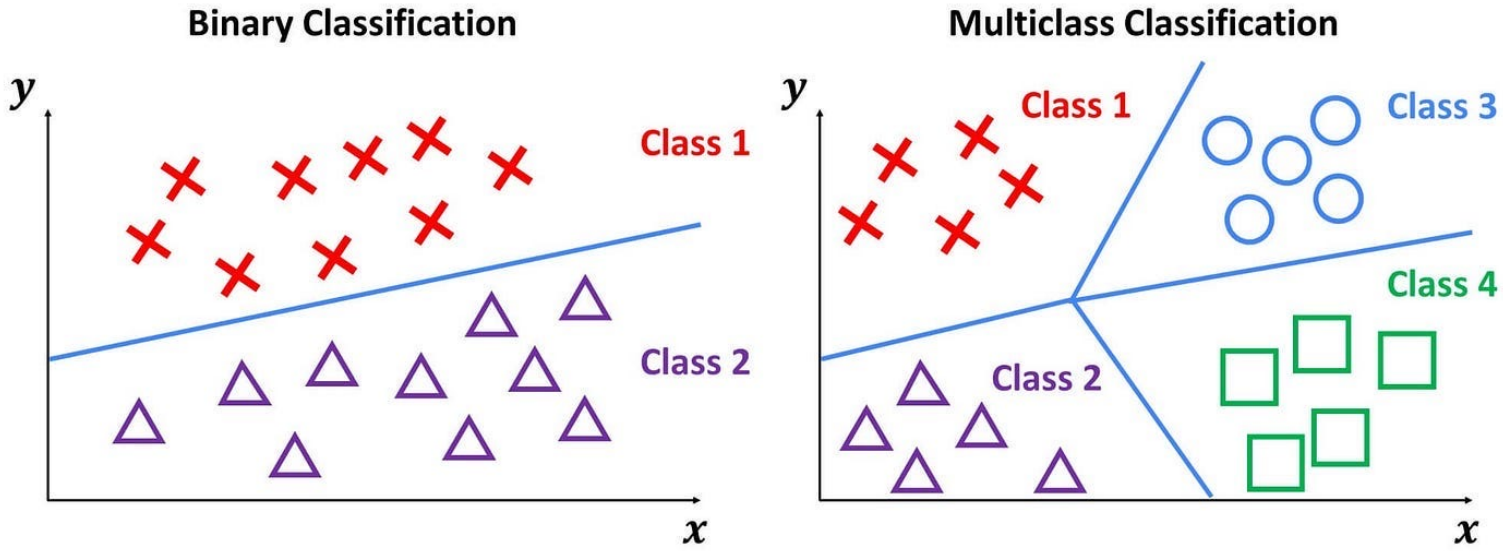
Decisions and evaluation



Softmax class probabilities

Murphy, Probabilistic Machine Learning: An Introduction
§§10.3.1–10.3.2: multiclass classifiers and likelihood

Multiclass classification



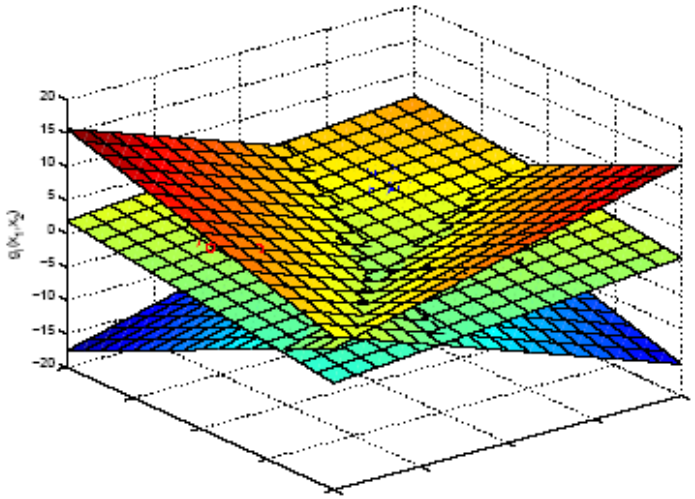
- Binary labels: two possible classes.

$$g(a) = \frac{1}{1 + \exp(-a)} \qquad p_i = g(\mathbf{w}^T \mathbf{x}^i)$$

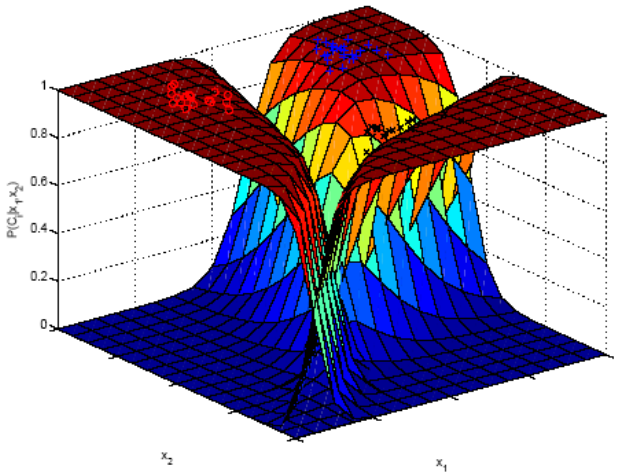
$$\ell(y_i, p_i) = -y_i \log p_i - (1 - y_i) \log(1 - p_i)$$

- Multiclass labels: one of C categories.

Softmax class probabilities



Discriminants (inputs)



Softmax (outputs)

Soft maximum (softmax) of competing classes:

$$p_c(\mathbf{x}) = \frac{\exp(\mathbf{w}_c^T \mathbf{x})}{\sum_{k=1}^{C} \exp(\mathbf{w}_k^T \mathbf{x})}, \quad \mathbf{W} = [\mathbf{w}_1, \ldots, \mathbf{w}_C]$$

k indexes classes in the denominator. Each output is one class probability.
c indexes the C classes (labels 1,…,C); w_c is its parameter vector.
W has D+1 rows and C columns, with w_c as column c.

Multiclass cross-entropy

Likelihood of training sample: $(\mathbf{x}_i, y_i), \quad y_i = 3$

One-hot target vector format: $\mathbf{t}_i = (0, 0, 1, 0)^T$

$$P(Y = y_i \mid \mathbf{x}_i; \mathbf{W}) = \prod_{c=1}^{C} p_c(\mathbf{x}_i)^{t_{ic}}$$

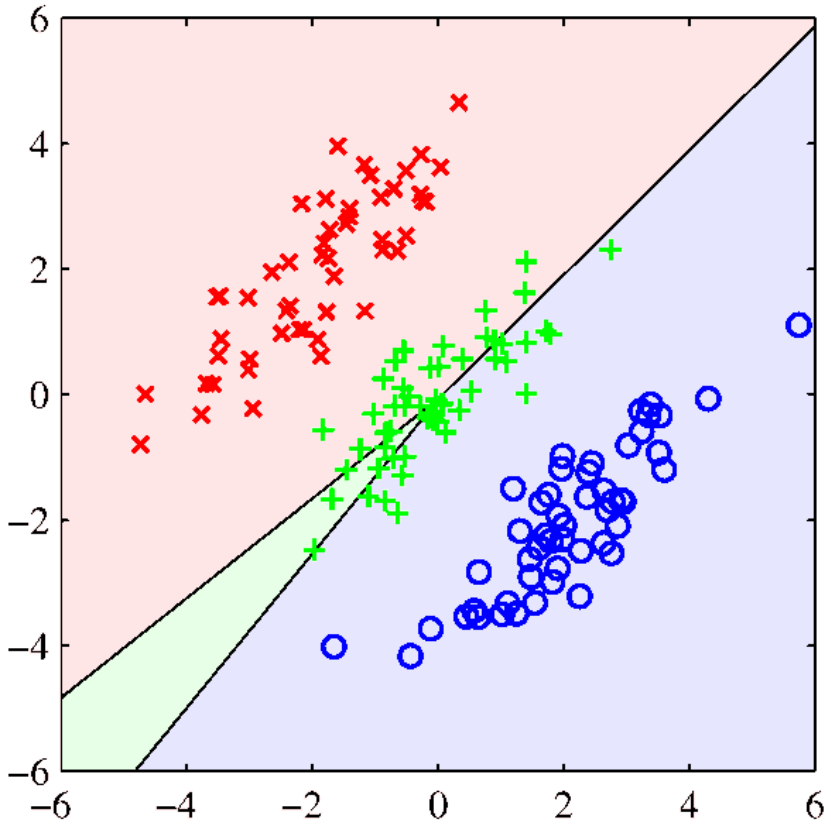
Optimization criterion:

$$L(\mathbf{W}) = -\sum_{i=1}^{N} \sum_{c=1}^{C} t_{ic} \log p_c(\mathbf{x}_i)$$

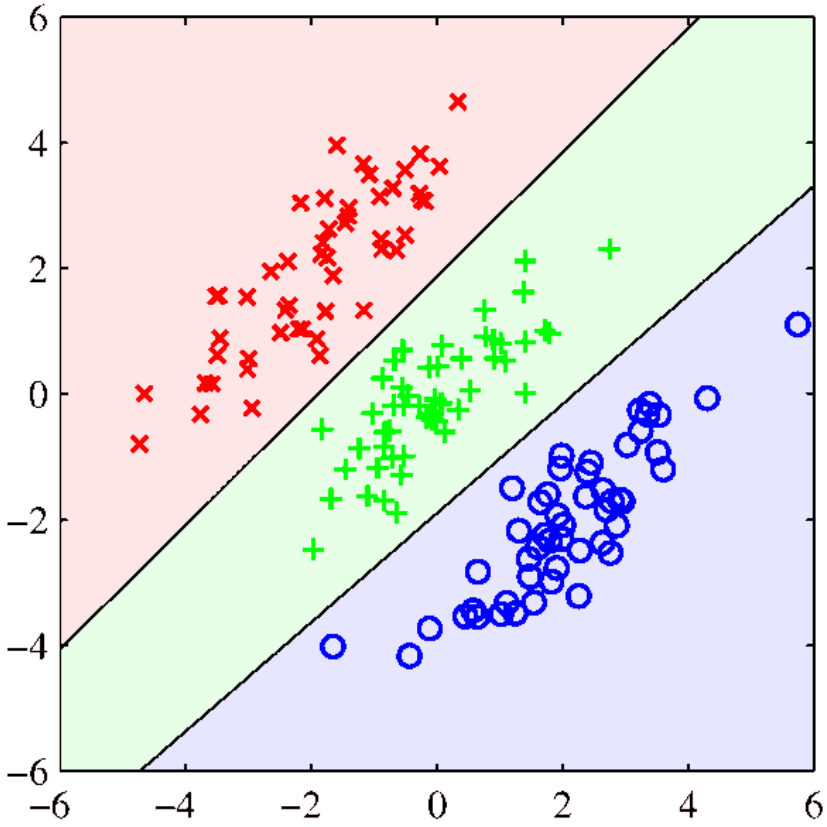
C = 4, y_i = 3; $t_i c$ = 1 for c = y_i, and 0 otherwise.
$p_c(x_i)$ is the predicted probability of class c for example i.
i indexes examples. The logarithms are natural.

Multiclass logistic and linear regression

Linear regression



Logistic regression



Here, squared loss gives the middle class too narrow a region;
logistic regression separates it more effectively.

Lecture 2: Classification & optimisation

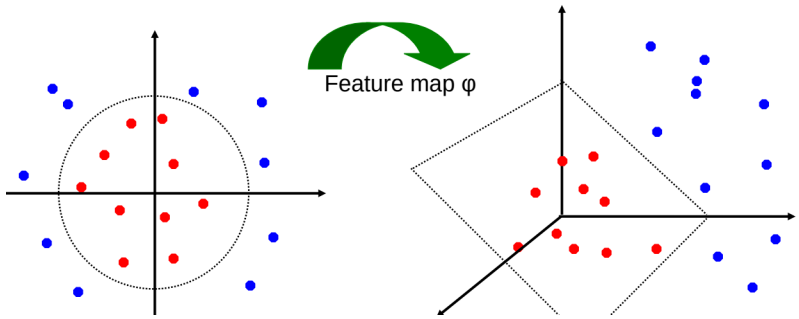
Classification

- Problem and classifiers
- Scores and logistic loss
- Multiclass probabilities
- **Nonlinear features**

Optimisation

Trees and ensembles

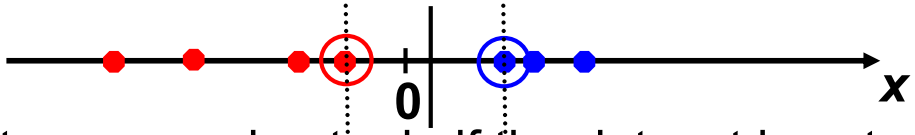
Decisions and evaluation



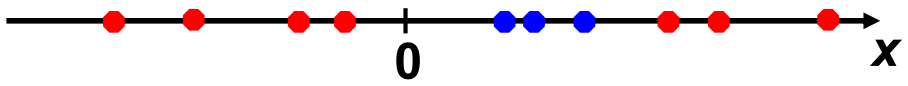
Murphy, Probabilistic Machine Learning: An Introduction
§10.2.2: nonlinear classifiers

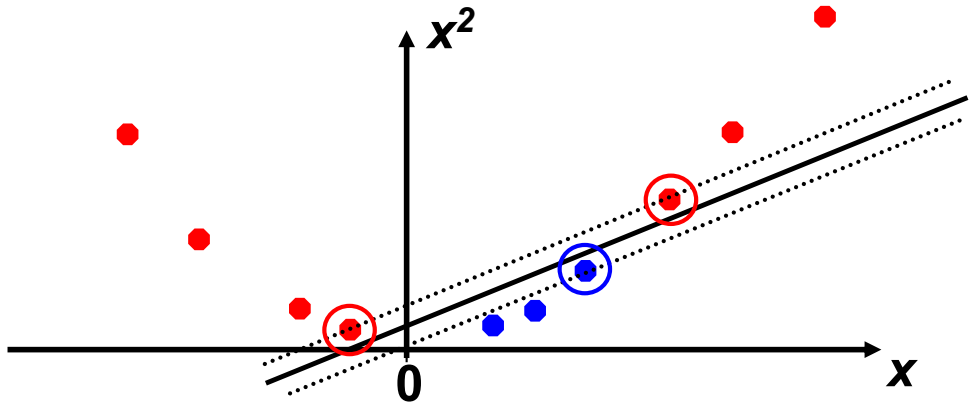
Nonlinear decision boundaries

- Datasets that are linearly separable (with some noise) work out great:

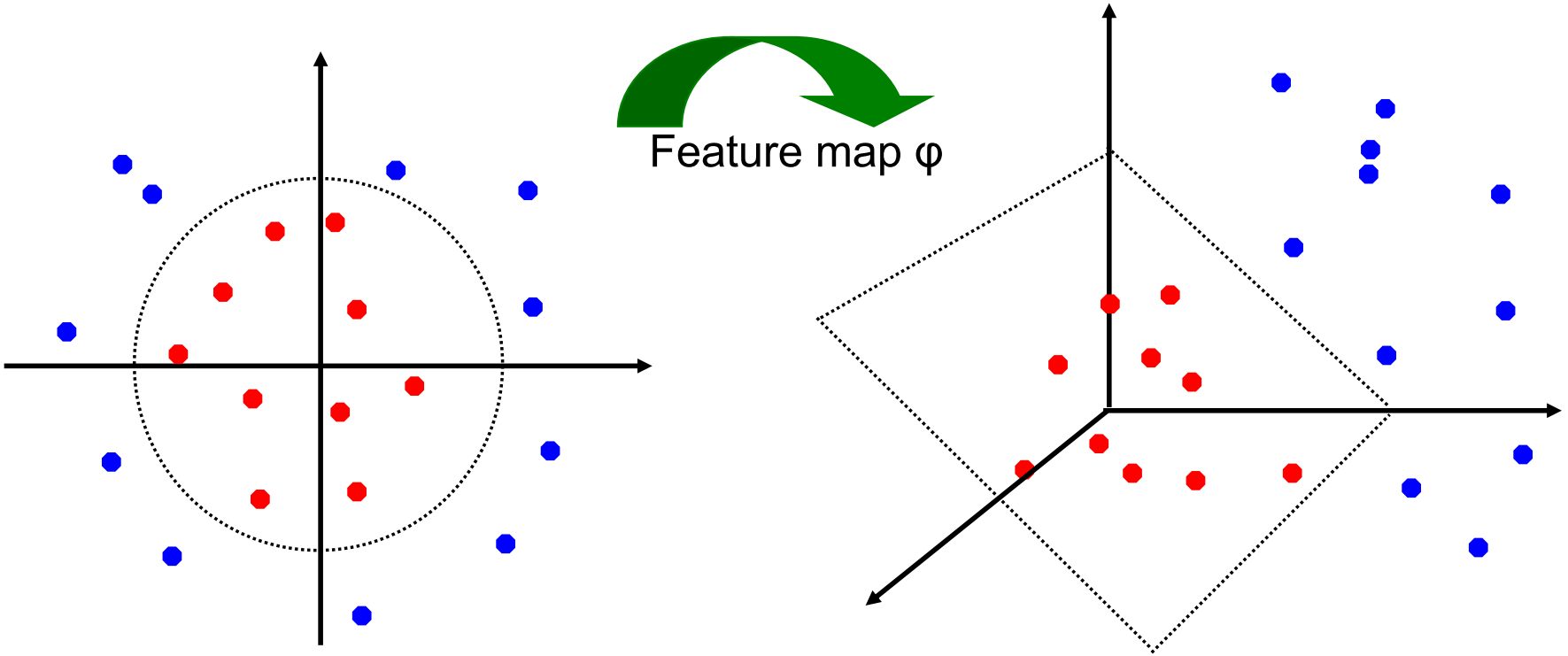


- But what are we going to do if the dataset is not separable?
- Solution: map data to a higher-dimensional space:





A feature map changes the geometry



Map (x_1, x_2) to $(x_1, x_2, x_1^2 + x_2^2)$.
A separating plane in feature space gives a curved boundary in input space.

Classification with nonlinear features

Linear case:

$$L(\mathbf{W}) = -\sum_{i=1}^{N}\sum_{c=1}^{C} t_{ic} \log p_c(\mathbf{x}_i; \mathbf{W})$$

Nonlinear case:

$$L(\mathbf{W}) = -\sum_{i=1}^{N}\sum_{c=1}^{C} t_{ic} \log p_c(\boldsymbol{\phi}(\mathbf{x}_i); \mathbf{W})$$

Each class score is linear in its coefficients. $\varphi_0(x) = 1$ has coefficient w_0c.
$\varphi = (\varphi_0, \ldots, \varphi_{M-1})^T$. Each class vector has M coefficients; W is M × C.

Lecture 2: Classification & optimisation

Classification

Optimisation

Gradient descent

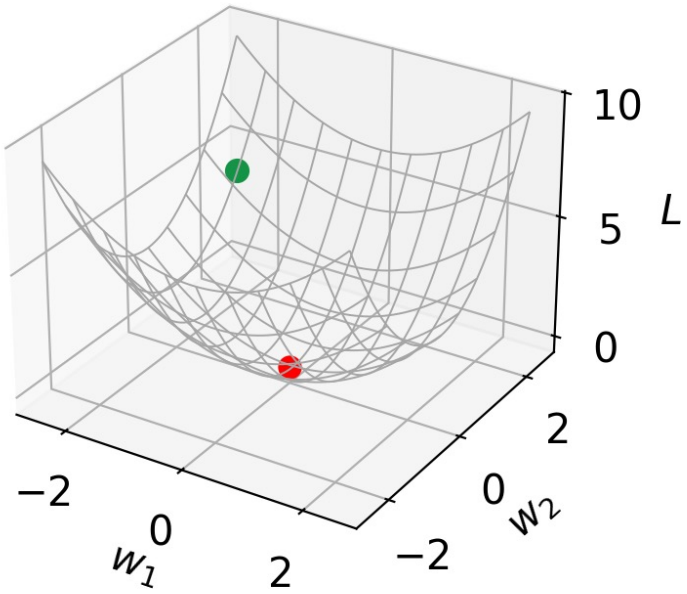
Newton–Raphson

SGD

L1 and L2 regularisation

Trees and ensembles

Decisions and evaluation



Green: current parameters
Red: minimum

Murphy, Probabilistic Machine Learning: An Introduction
§§8.2.1–8.2.3: descent direction, step size and convergence

Differentiating cross-entropy

$$L(\mathbf{w}) = -\sum_{i=1}^{N} \{y_i \log p_i + (1 - y_i) \log(1 - p_i)\}$$

$$\frac{\partial L}{\partial w_j} = -\sum_{i=1}^{N} \left[\frac{y_i}{p_i} - \frac{1 - y_i}{1 - p_i}\right] \frac{\partial p_i}{\partial w_j}$$

j indexes a coefficient and its input coordinate.

$$g'(a) = g(a)[1 - g(a)], \qquad p_i = g(\mathbf{w}^T \mathbf{x}_i)$$

$$= -\sum_{i=1}^{N} \left[\frac{y_i}{p_i} - \frac{1 - y_i}{1 - p_i}\right] p_i(1 - p_i) x_{ij}$$

$$= -\sum_{i=1}^{N} [y_i(1 - p_i) - (1 - y_i)p_i] x_{ij} \qquad = \sum_{i=1}^{N} (p_i - y_i) x_{ij}$$

The logistic gradient has a simple form

$$p_i = g(\mathbf{w}^T \mathbf{x}_i), \qquad \ell_i = -y_i \log p_i - (1 - y_i) \log(1 - p_i)$$

$$z_i = \mathbf{w}^T \mathbf{x}_i$$

$$\frac{\partial \ell_i}{\partial z_i} = p_i - y_i, \qquad \nabla_{\mathbf{w}} \ell_i = (p_i - y_i)\mathbf{x}_i$$

$$L(\mathbf{w}) = \sum_{i=1}^{N} \ell_i, \qquad \nabla L(\mathbf{w}) = \sum_{i=1}^{N} (p_i - y_i)\mathbf{x}_i$$

$\nabla L(\widehat{\mathbf{w}}) = \mathbf{0}$ Nonlinear system of equations in w: Fit iteratively!

p_i is the predicted probability, y_i the observed label.
z_i is the score for example i, and ℓ_i its loss.

Numerical optimisation changes parameters

Classification supplies a loss L(w). An optimiser searches for parameter values that reduce it.

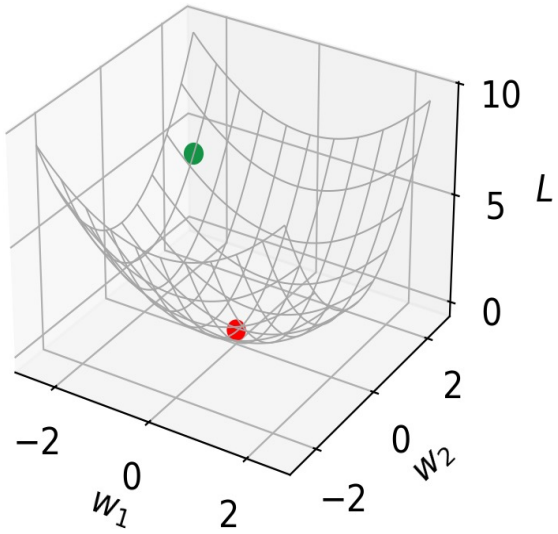
$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \nabla L(\mathbf{w}_t)$$

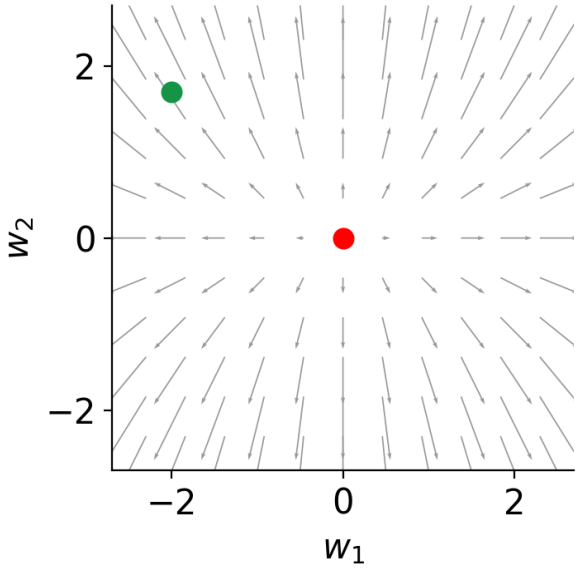
t counts updates. $\eta_t > 0$ is the learning rate.
The gradient contains one partial derivative per parameter.

Gradient descent in parameter space

$$L(\mathbf{w}) = \frac{1}{2}(w_1^2 + 2w_2^2)$$

Green: current parameters.
Red: minimum.





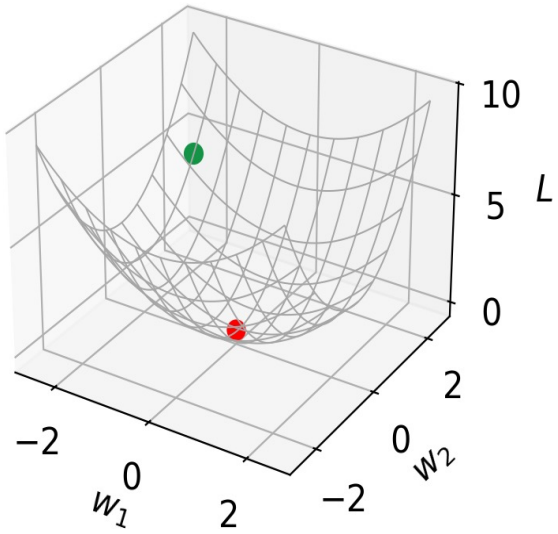
$$\nabla L(\mathbf{w}) = \begin{pmatrix} w_1 \\ 2w_2 \end{pmatrix}$$

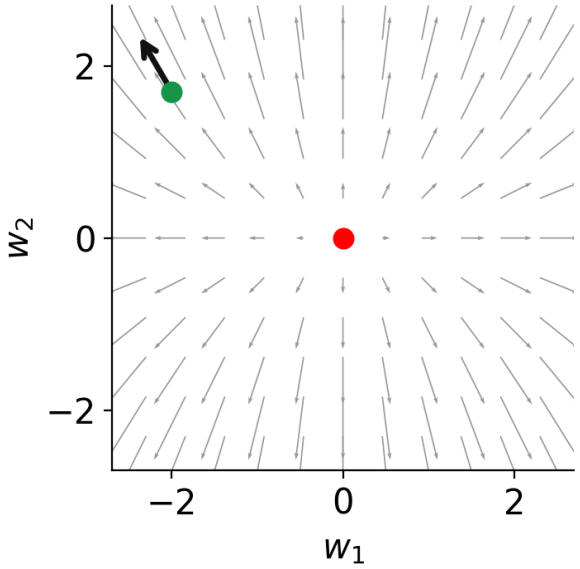
The gradient gives the steepest local increase

Gradient descent in parameter space

$$L(\mathbf{w}) = \frac{1}{2}(w_1^2 + 2w_2^2)$$

Green: current parameters.
Red: minimum.





$$\nabla L(\mathbf{w}) = \begin{pmatrix} w_1 \\ 2w_2 \end{pmatrix}$$

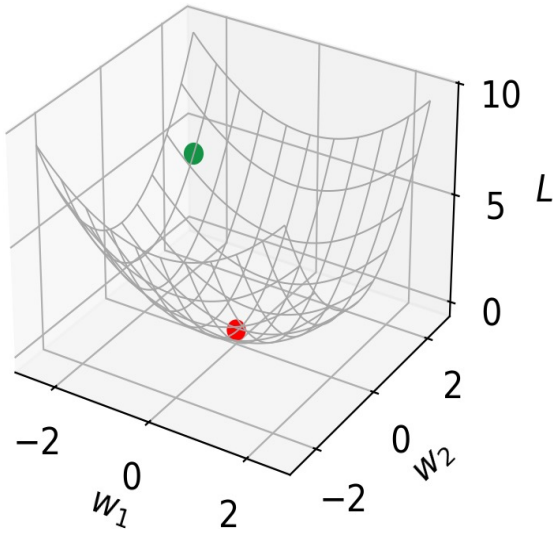
The gradient gives the steepest local increase

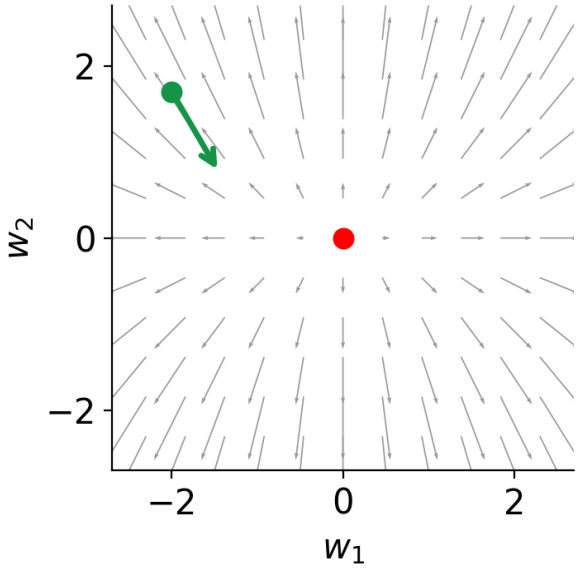
Take a small step in the opposite direction.

Gradient descent in parameter space

$$L(\mathbf{w}) = \frac{1}{2}(w_1^2 + 2w_2^2)$$

Green: current parameters.
Red: minimum.





$$\nabla L(\mathbf{w}) = \begin{pmatrix} w_1 \\ 2w_2 \end{pmatrix}$$

The gradient gives the steepest local increase

Take a small step in the opposite direction.

Initialize: $\mathbf{w}_0$

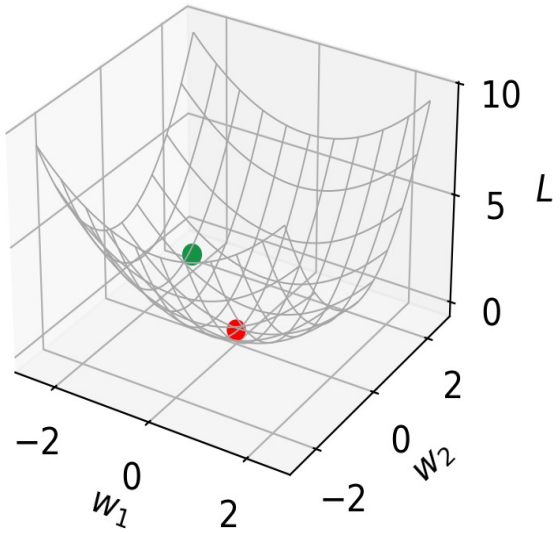
Update: $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \nabla L(\mathbf{w}_t)$

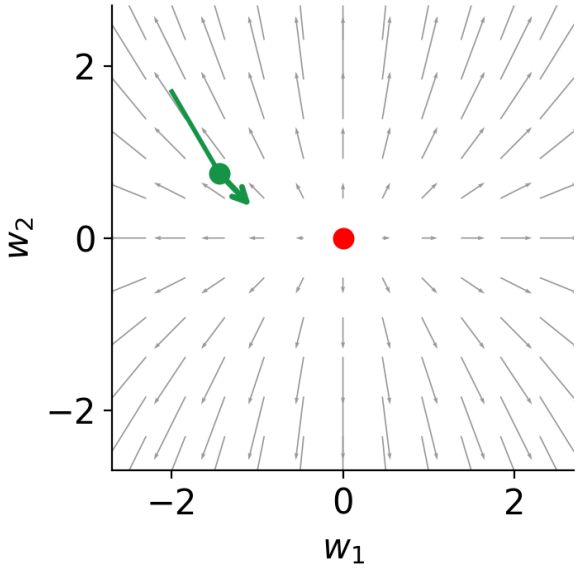
t=0

Gradient descent in parameter space

$$L(\mathbf{w}) = \frac{1}{2}(w_1^2 + 2w_2^2)$$

Green: current parameters.
Red: minimum.





$$\nabla L(\mathbf{w}) = \begin{pmatrix} w_1 \\ 2w_2 \end{pmatrix}$$

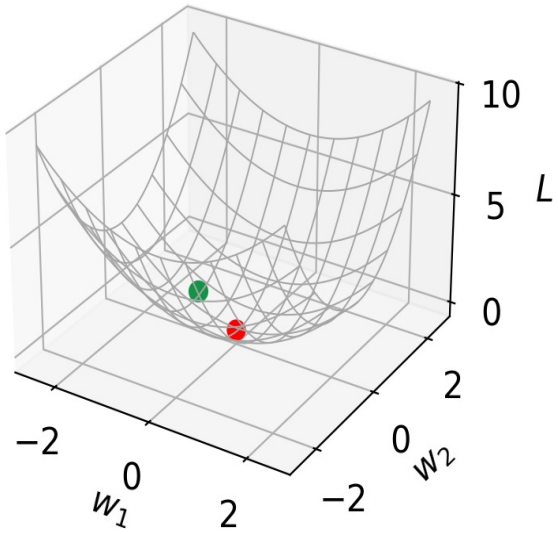
Update: $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \nabla L(\mathbf{w}_t)$

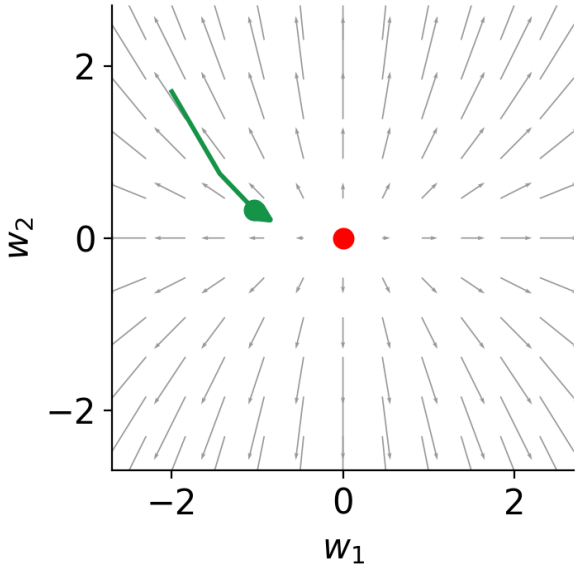
t=1

Gradient descent in parameter space

$$L(\mathbf{w}) = \frac{1}{2}(w_1^2 + 2w_2^2)$$

Green: current parameters.
Red: minimum.





$$\nabla L(\mathbf{w}) = \begin{pmatrix} w_1 \\ 2w_2 \end{pmatrix}$$

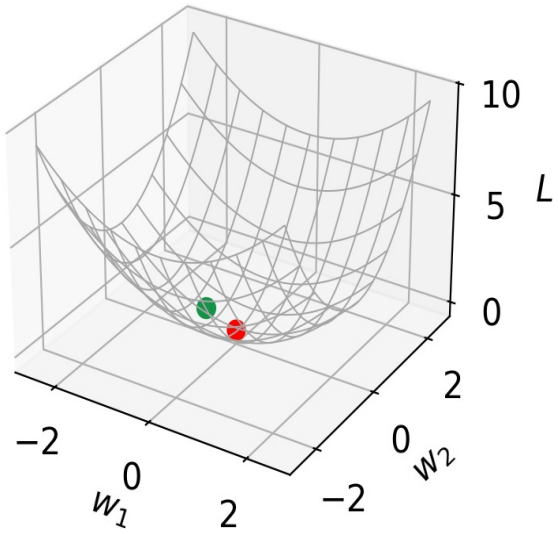
Update: $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \nabla L(\mathbf{w}_t)$

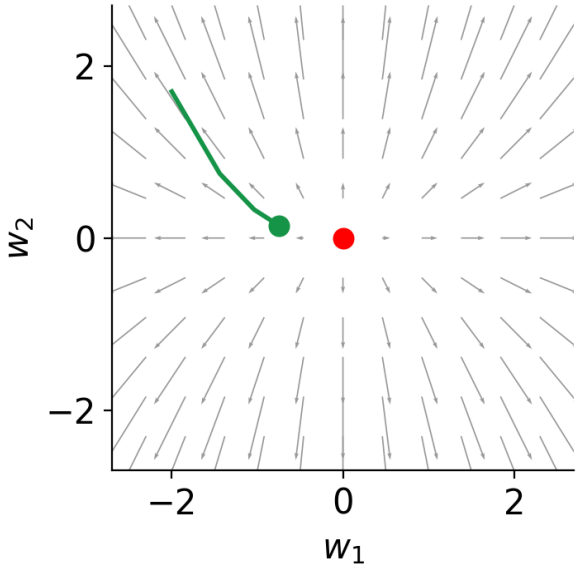
t=2

Gradient descent in parameter space

$$L(\mathbf{w}) = \frac{1}{2}(w_1^2 + 2w_2^2)$$

Green: current parameters.
Red: minimum.



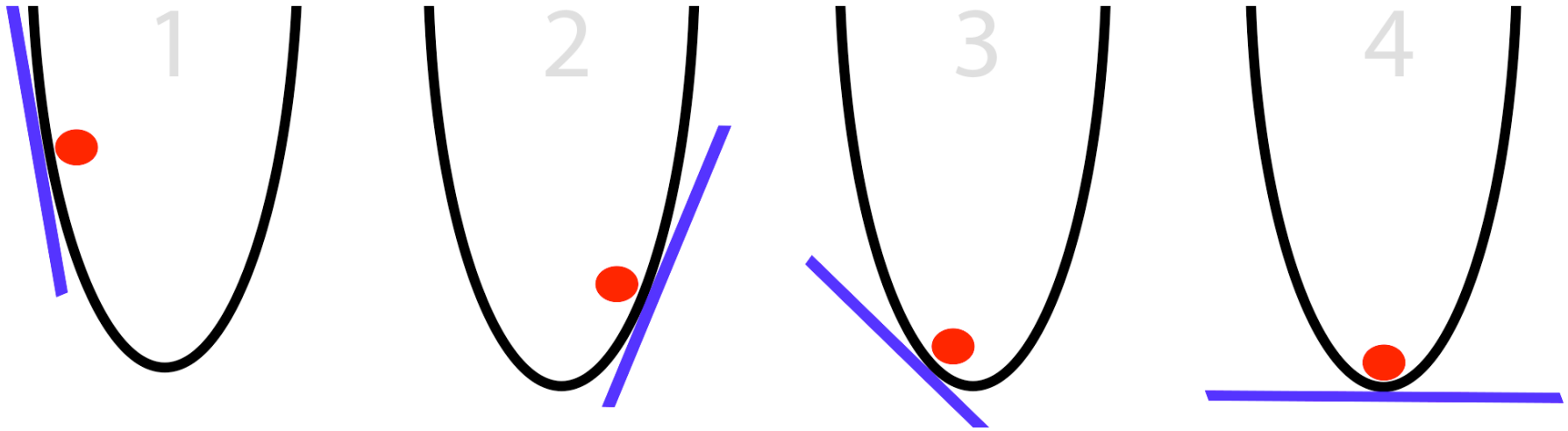


$$\nabla L(\mathbf{w}) = \begin{pmatrix} w_1 \\ 2w_2 \end{pmatrix}$$

Update: $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \nabla L(\mathbf{w}_t)$

t=3

Following local slopes



Learning rate and stopping

$$L(w) = \tfrac{1}{2}w^2, \qquad L'(w) = w, \qquad w_{t+1} = (1-\eta)w_t$$

One scalar parameter w; the minimum is at w = 0.

0 < η < 1: approaches zero without changing sign.
η = 1: reaches the minimum in one step.
1 < η < 2: alternates signs but still approaches zero.
η = 2: oscillates; η > 2: diverges (unless already at zero).

For general objectives

Monitor loss and gradient size, and impose an update budget.
A small update alone can mean that the learning rate is too small.

Lecture 2: Classification & optimisation

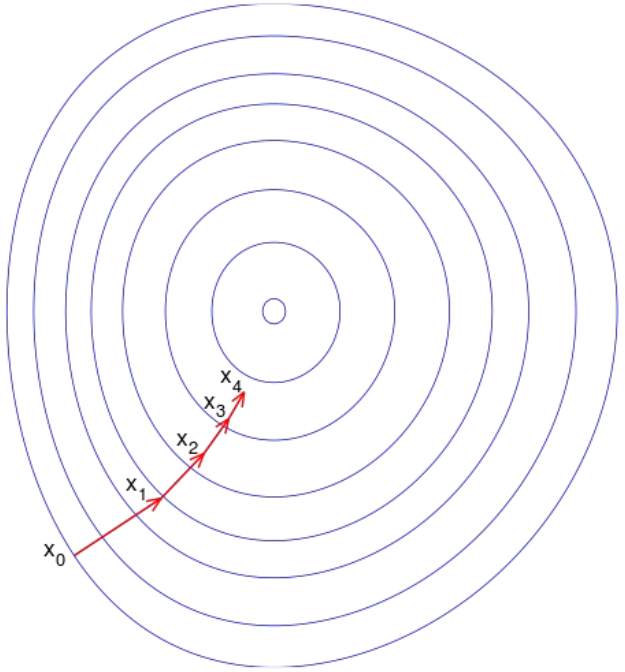
Classification

Optimisation

- Gradient descent
- **Newton–Raphson**
- SGD
- L1 and L2 regularisation

Trees and ensembles

Decisions and evaluation



Successive optimisation steps
Iterates correspond to parameter vectors w

Murphy, Probabilistic Machine Learning: An Introduction
§8.3.1: Newton's method

Learning rate and narrow valleys

Step-size selection:

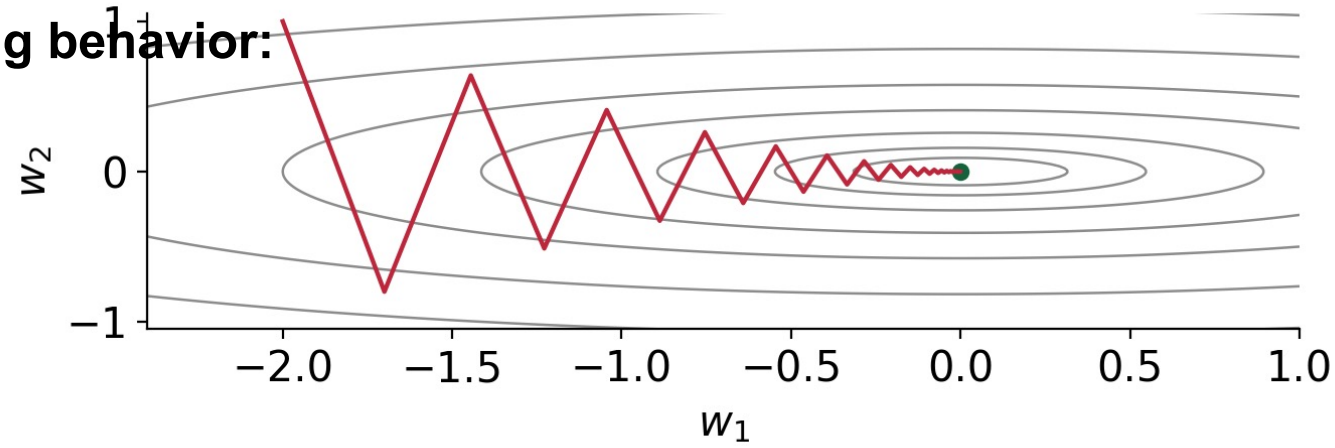
Initialize: $\mathbf{w}_0$

Update: $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \nabla L(\mathbf{w}_t)$

How large a step?

A narrow valley has very different curvature in different parameter directions.
One rate can make progress slow or oscillatory.

Zig-zagging behavior:



Curvature in least squares

Sum of squared errors minimization:

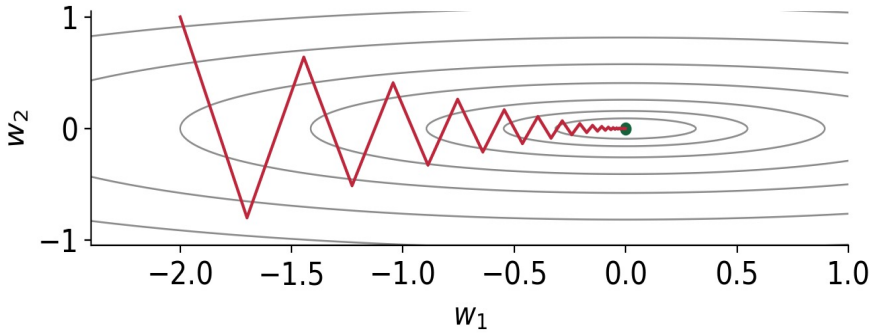
$$L(\mathbf{w}) = (\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w})$$

Gradient descent:

Initialize: $\mathbf{w}_0$

Update: $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \nabla L(\mathbf{w}_t)$

May require many steps!



A full Newton step solves this quadratic when X has full column rank.

Recall: the least-squares solution

$$L(\mathbf{w}) = \boldsymbol{\epsilon}^T \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} = \mathbf{y} - \mathbf{X}\mathbf{w}$$

$$L(\mathbf{w}) = \mathbf{y}^T\mathbf{y} - 2\mathbf{y}^T\mathbf{X}\mathbf{w} + \mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w}$$

$$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

ε = y − Xw is the residual vector.
Dimensions: X is N × (D+1), w is (D+1) × 1, y is N × 1.
The displayed inverse assumes X has full column rank.

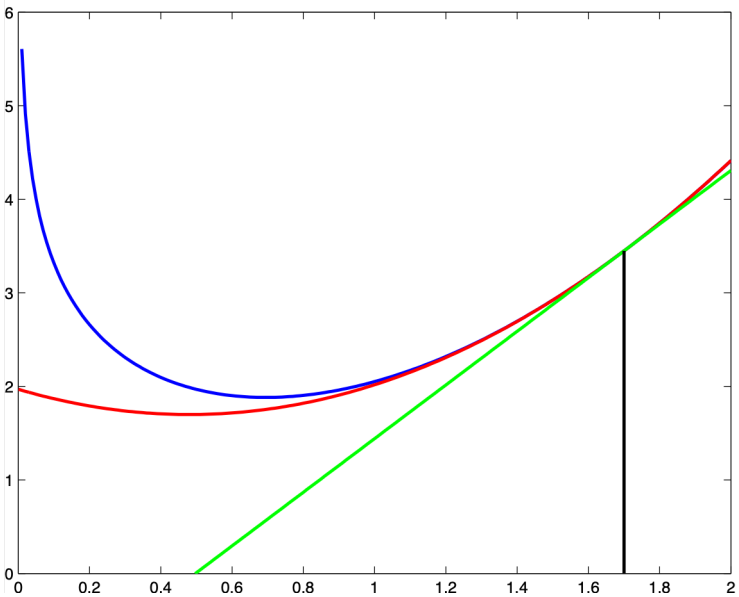
Linear and quadratic approximations

First-order Taylor series approximation:

$$L(w) \simeq L(a) + (w - a)L'(a)$$

Second-order Taylor series approximation:

$$L(w) \simeq L(a) + (w - a)L'(a) + \tfrac{1}{2}(w - a)^2 L''(a)$$



blue:

$$L(w) = w^2 - \log w + \exp(w/20)$$

green: linear approximation

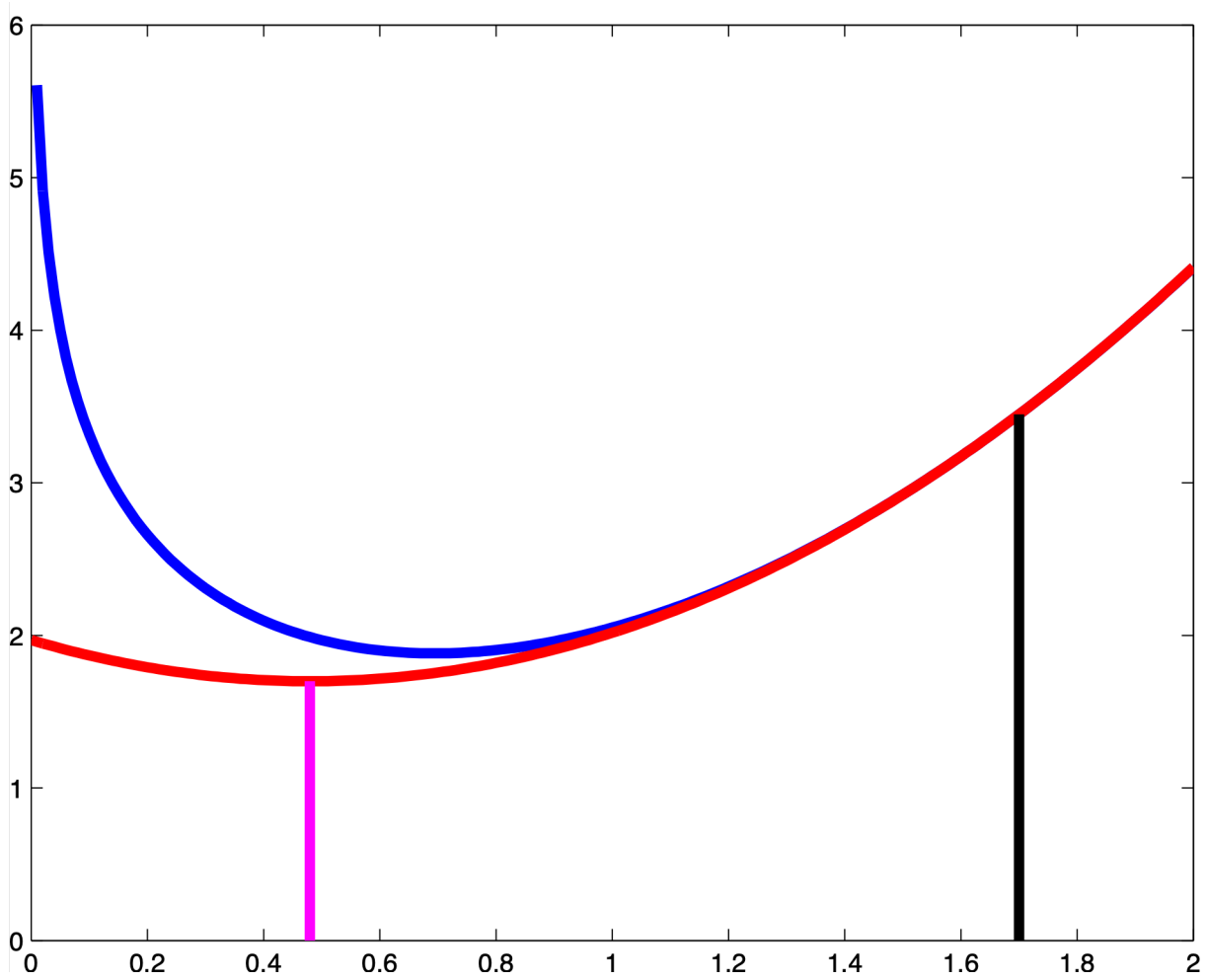
$$l(w) = L(a) + (w - a)L'(a)$$

red: quadratic approximation

$$q(w) = l(w) + \tfrac{1}{2}(w - a)^2 L''(a)$$

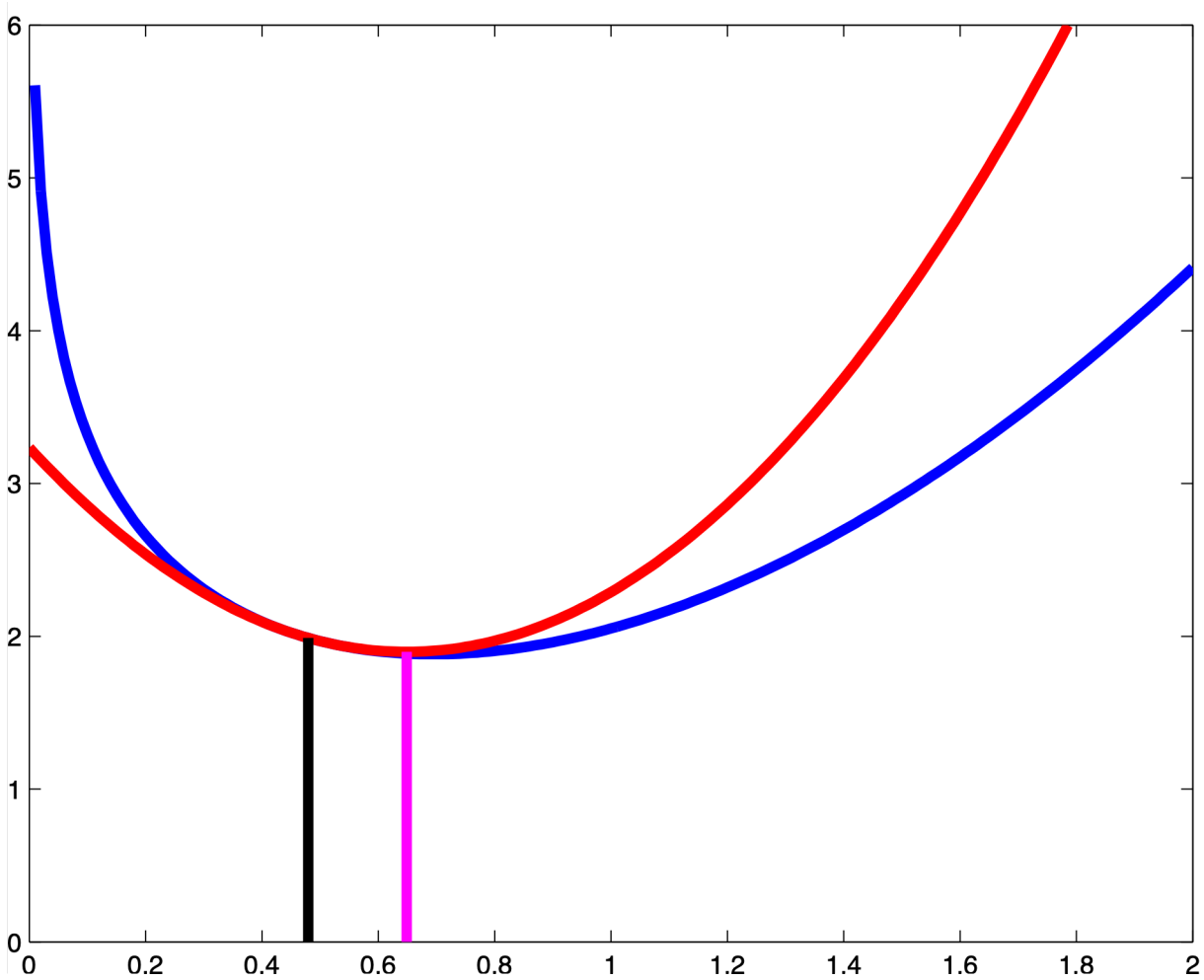
a is the current parameter; l and q are local linear and quadratic models.

Newton's quadratic approximation



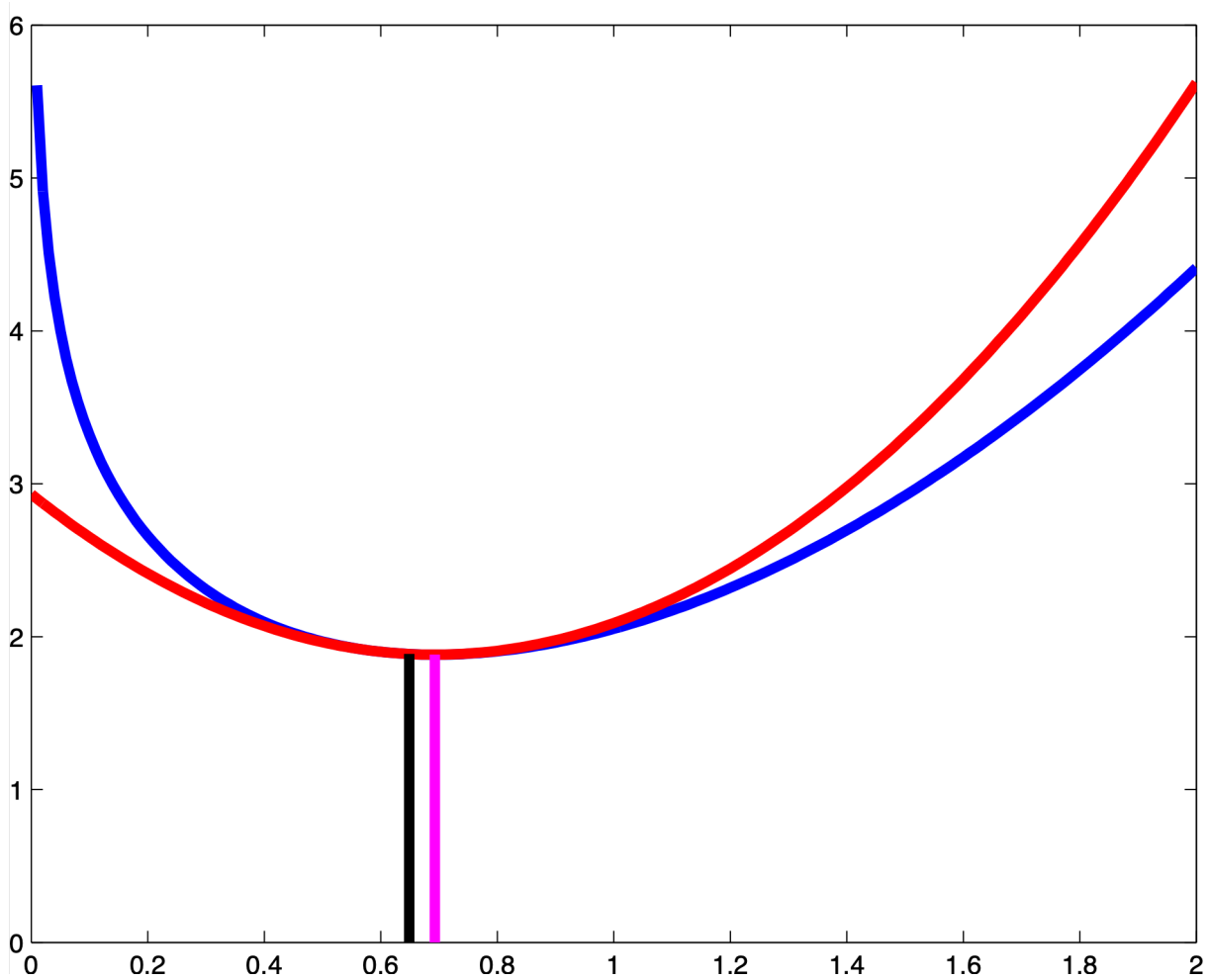
Blue: loss. Red: quadratic model. Horizontal axis: parameter w.
Black marks the current iterate; magenta marks the model's minimum.

Newton's quadratic approximation



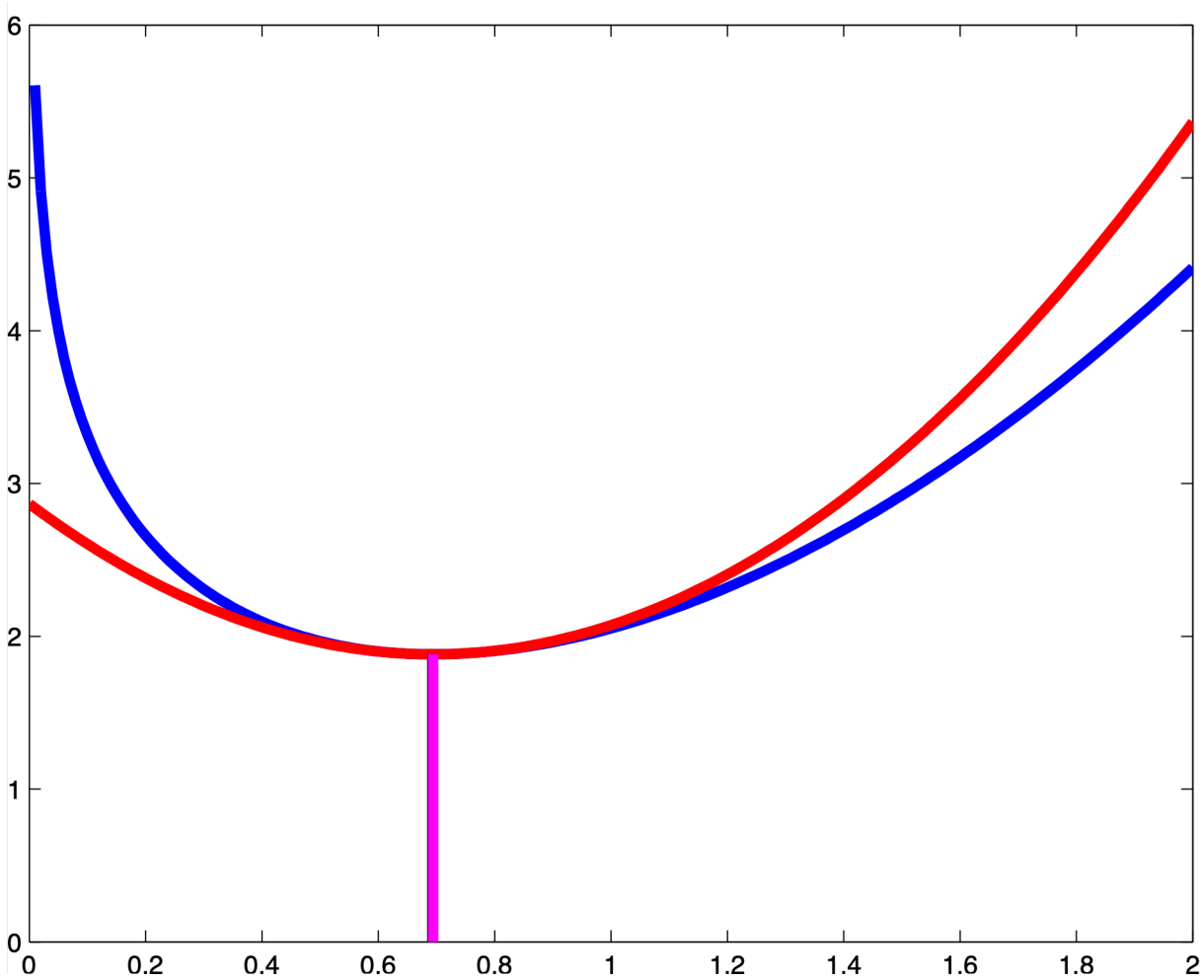
Blue: loss L(w). Red: local quadratic model.
Black: current parameter. Magenta: minimum of the quadratic model.

Newton: a later iteration



Blue: loss. Red: quadratic model. Horizontal axis: parameter w.
Black marks the current iterate; magenta marks the model's minimum.

Newton's quadratic approximation



Blue: loss L(w). Red: local quadratic model.
Black: current parameter. Magenta: minimum of the quadratic model.

Newton's method in one dimension

Initial parameter: w_0

At any point, form quadratic approximation:

$$q_t(w) = L(w_t) + (w - w_t)L'(w_t) + \tfrac{1}{2}(w - w_t)^2 L''(w_t)$$

Stationary condition for the quadratic model:

$$q_t'(w) = L'(w_t) + (w - w_t)L''(w_t) = 0$$

Take the stationary point of this quadratic model.

$$w_{t+1} = w_t - \frac{L'(w_t)}{L''(w_t)}$$

Check convergence and whether the loss decreases.

A quadratic model in several dimensions

First-order Taylor series approximation:

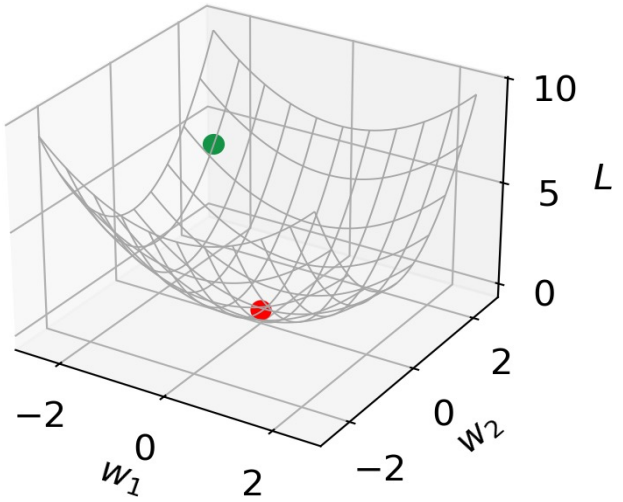
$$L(\mathbf{w}) \simeq L(\mathbf{w}_t) + (\mathbf{w} - \mathbf{w}_t)^T \nabla L(\mathbf{w}_t)$$

Second-order Taylor series approximation:

$$q_t(\mathbf{w}) = L(\mathbf{w}_t) + \mathbf{d}^T \nabla L(\mathbf{w}_t) + \frac{1}{2}\mathbf{d}^T \mathbf{H}_t \mathbf{d}$$

$$\mathbf{d} = \mathbf{w} - \mathbf{w}_t$$

$$(H_t)_{jk} = \left. \frac{\partial^2 L}{\partial w_j \partial w_k} \right|_{\mathbf{w}_t}$$



H_t contains second derivatives: it describes local curvature.
With D+1 coefficients, it has D+1 rows and columns; j and k index these.

Newton's method in several dimensions

Initial parameter: $\mathbf{w}_0$

At any point, form quadratic approximation:

$$q_t(\mathbf{w}) = L(\mathbf{w}_t) + \mathbf{d}^T \nabla L(\mathbf{w}_t) + \tfrac{1}{2}\mathbf{d}^T \mathbf{H}_t \mathbf{d}$$

Stationary condition for the quadratic model:

$$\nabla q_t(\mathbf{w}) = \mathbf{0} \quad \Longrightarrow \quad \mathbf{H}_t \mathbf{d} = -\nabla L(\mathbf{w}_t)$$

Take the stationary point of this quadratic model.

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{H}_t^{-1} \nabla L(\mathbf{w}_t)$$

Check convergence and whether the loss decreases.

For logistic regression, Newton gives iteratively reweighted least squares (IRLS). See the appendix derivation.

When Newton's method needs safeguards

$$\mathbf{H}_t \mathbf{d}_t = -\nabla L(\mathbf{w}_t), \qquad \mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t \mathbf{d}_t$$

H_t is the Hessian at the current parameters; d_t is the step direction.
α_t is a step length, often reduced until the loss decreases.

A positive-definite Hessian gives a descent direction.
An indefinite or singular Hessian requires a modification.

Solving a linear system avoids forming an explicit inverse.
For many parameters, storing and solving with the Hessian is costly.

Lecture 2: Classification & optimisation

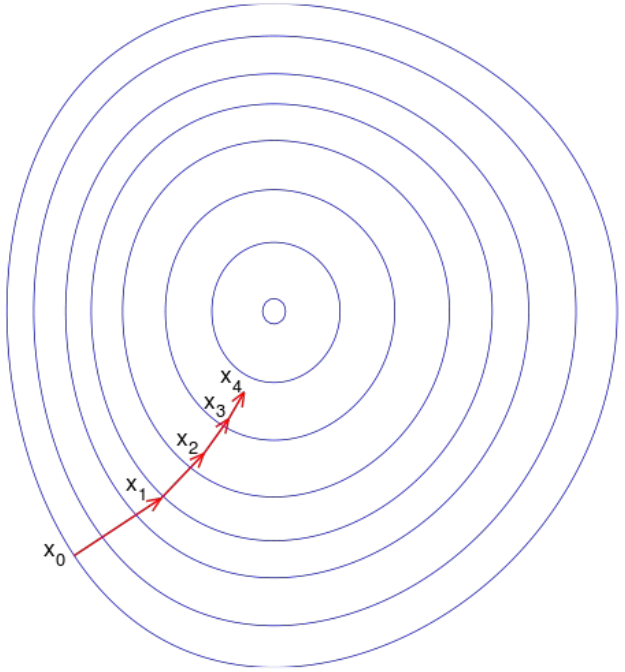
Classification

Optimisation

- Gradient descent
- Newton–Raphson
- **SGD**
- L1 and L2 regularisation

Trees and ensembles

Decisions and evaluation



Successive optimisation steps
Iterates correspond to parameter vectors w

Murphy, Probabilistic Machine Learning: An Introduction
§8.2.4; §§8.4.1–8.4.3: momentum and SGD

Summed, mean and penalised losses

Probability of the observed label: $p(y_i \mid \mathbf{x}_i; \mathbf{w})$

Cost function:

$$L(\mathbf{w}) = \sum_{i=1}^{N} \ell_i(\mathbf{w})$$

Normalize:

$$\overline{L}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} \ell_i(\mathbf{w})$$

$$\ell_i(\mathbf{w}) = -\log p(y_i \mid \mathbf{x}_i; \mathbf{w})$$

Add regularization:

$$J(\mathbf{w}) = \overline{L}(\mathbf{w}) + \frac{\lambda}{2} \sum_{j=1}^{D} w_j^2$$

λ ≥ 0 sets penalty strength. The intercept coefficient is unpenalised.

A full gradient uses every example

$$J(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\ell_i(\mathbf{w}) + \frac{\lambda}{2}\sum_{j=1}^{D}w_j^2$$

Gradient descent: $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \nabla J(\mathbf{w}_t)$

Average the example gradients, then add the penalty gradient:

$$\nabla J(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\nabla\ell_i(\mathbf{w}) + \lambda\mathbf{r}(\mathbf{w})$$

$\nabla\ell_i$ is an individual example's gradient; r(w) excludes the intercept.

$$\mathbf{r}(\mathbf{w}) = (0, w_1, \ldots, w_D)^T$$

If N = 1,000,000, every full update uses a million examples.

Stochastic gradient descent

Gradient: **Full batch: N examples**

$$\nabla J(\mathbf{w}_t) = \frac{1}{N}\sum_{i=1}^{N} \nabla \ell_i(\mathbf{w}_t) + \lambda \mathbf{r}(\mathbf{w}_t)$$

Estimated gradient: **Mini-batch: B examples**

$\mathcal{B}_t$ lists B uniform draws with replacement; repeated indices are allowed.
B = 1 is single-example SGD.

$$\mathbf{g}_t = \frac{1}{B}\sum_{i \in \mathcal{B}_t} \nabla \ell_i(\mathbf{w}_t) + \lambda \mathbf{r}(\mathbf{w}_t)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \mathbf{g}_t$$

Uniform sampling gives the full gradient on average; each update is noisy.

SGD and coefficient shrinkage

Gradient: **Full batch: N examples**

$$\nabla J(\mathbf{w}_t) = \frac{1}{N}\sum_{i=1}^{N} \nabla \ell_i(\mathbf{w}_t) + \lambda \mathbf{r}(\mathbf{w}_t)$$

Estimated gradient: **Mini-batch: B examples**

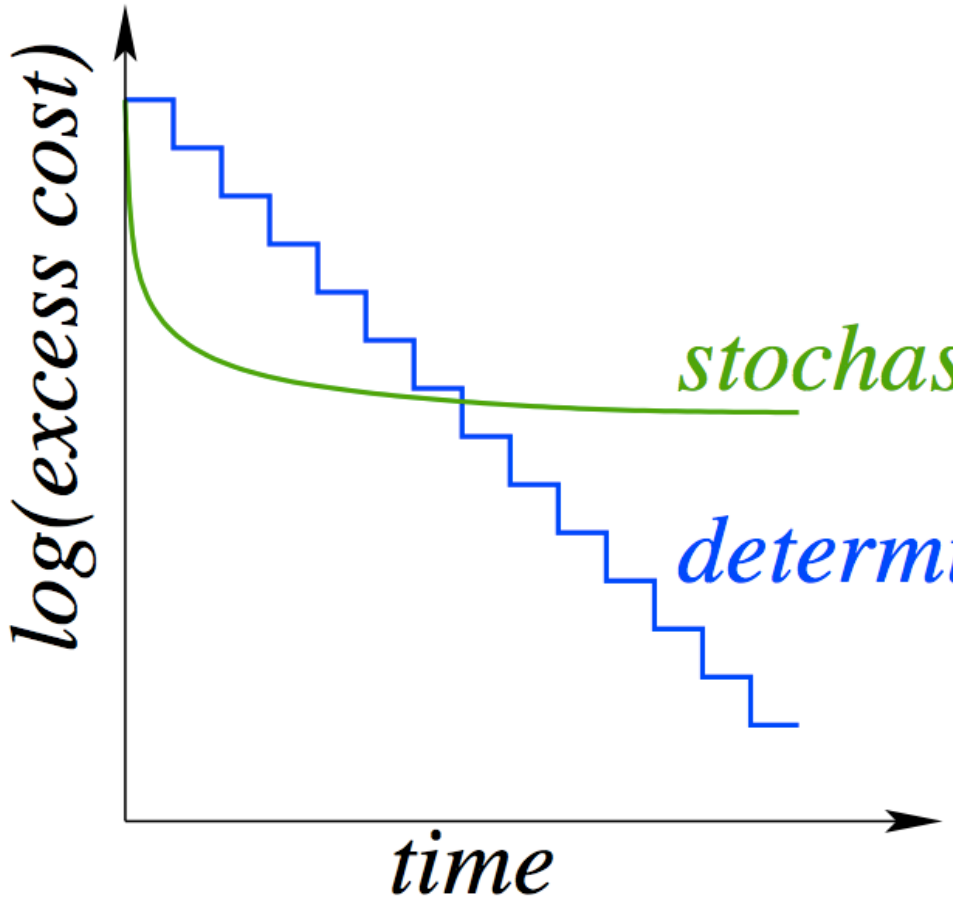
$\mathcal{B}_t$ lists B uniform draws with replacement; repeated indices are allowed.
B = 1 is single-example SGD.

$$\mathbf{g}_t = \underbrace{\frac{1}{B}\sum_{i\in\mathcal{B}_t} \nabla \ell_i(\mathbf{w}_t)}_{\text{mean data gradient}} + \underbrace{\lambda \mathbf{r}(\mathbf{w}_t)}_{\text{penalty gradient}}$$

$$w_{t+1,j} = (1-\eta_t\lambda) w_{t,j} - \eta_t \frac{1}{B}\sum_{i\in\mathcal{B}_t} \frac{\partial \ell_i}{\partial w_j}(\mathbf{w}_t), \quad j \geq 1$$

$\eta_t\lambda < 1$: shrink non-intercept coefficients, then take the data step.

Mini-batches change the cost per update

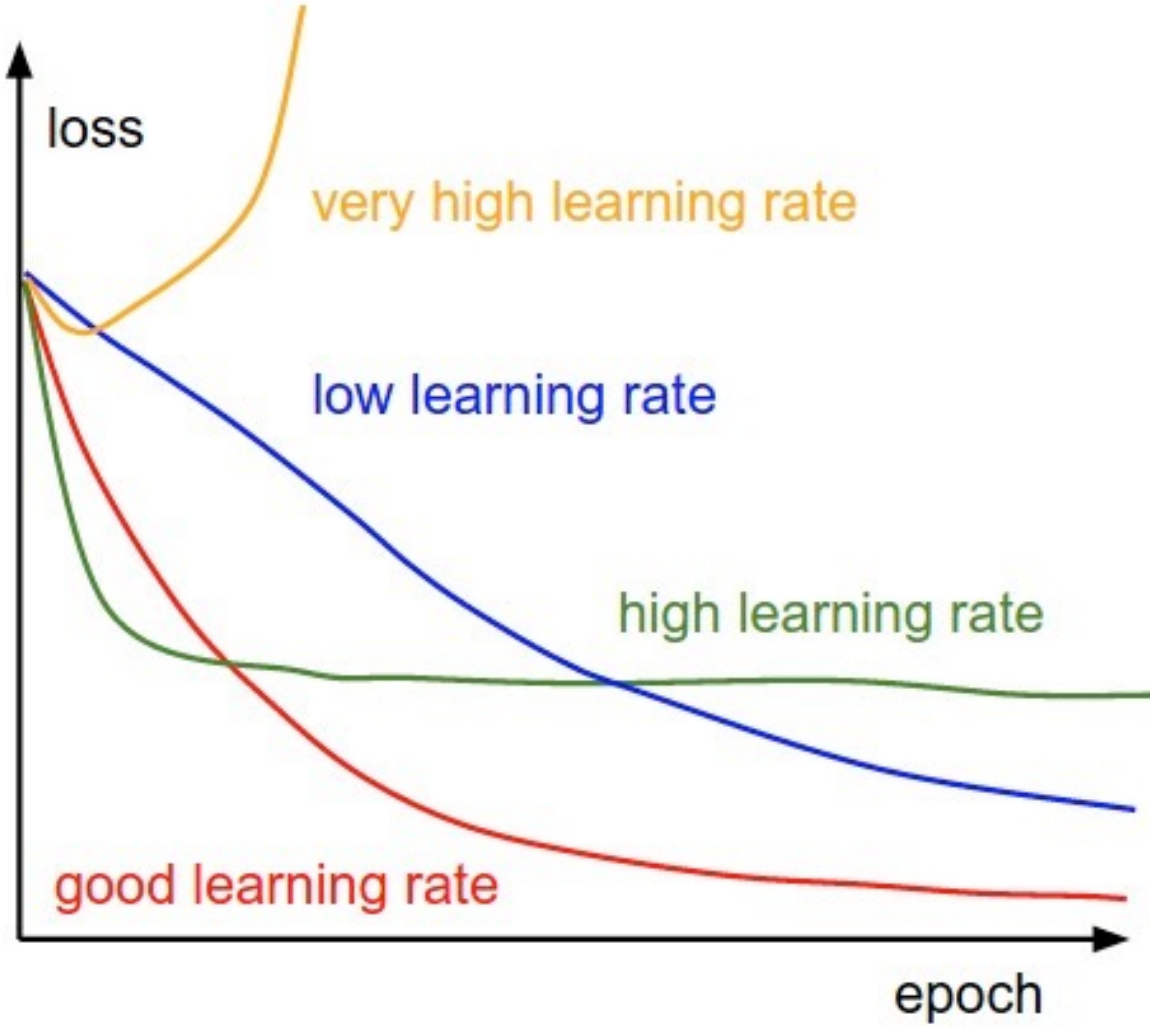


With replacement, N/B updates process N sampled examples: one epoch-equivalent.

More updates do not imply the same factor of speedup.

Schematic: excess cost is J minus its minimum value.
Compare time to a useful objective value, not just update counts.

Choosing the learning rate

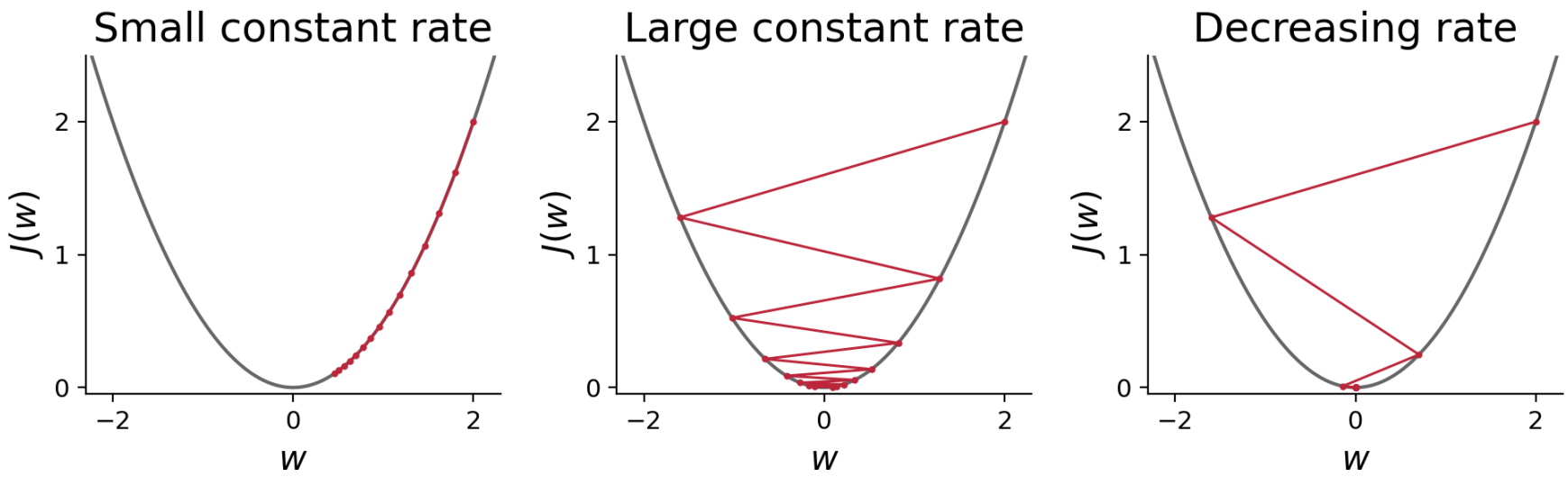


Schematic SGD training curves.

A fixed rate can leave a noise floor.

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \mathbf{g}_t$$

Decreasing the learning rate



Same J(w) = w²/2, start w = 2; 14 updates.
Rates: 0.1, 1.8, 1.8/(1+t/4), left to right.
In SGD, decreasing the rate reduces the effect of gradient noise.

Example schedule

$$\eta_t = \frac{\eta_0}{1 + t/t_0}$$

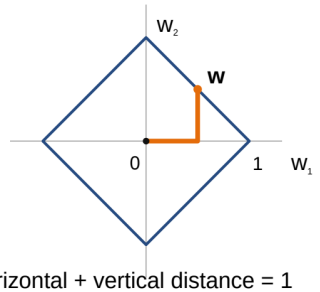
- $\eta_0 > 0$ is the initial rate; $t_0 > 0$ controls the decay timescale.

Lecture 2: Classification & optimisation

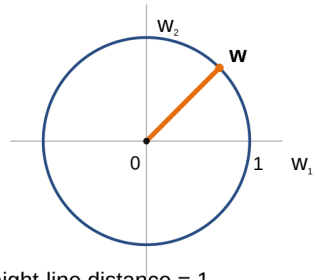
Classification

Optimisation

- Gradient descent
- Newton–Raphson
- SGD
- **L1 and L2 regularisation**



Horizontal + vertical distance = 1



Straight-line distance = 1

Trees and ensembles

Decisions and evaluation

Murphy, Probabilistic Machine Learning: An Introduction
§§11.4.1–11.4.2: L1 regularisation and sparsity

L2 shrinkage weakens near zero

Squared L2 penalty, excluding the intercept:

$$\Omega_2(\mathbf{w}) = \frac{\lambda}{2}\sum_{j=1}^{D} w_j^2, \qquad \frac{\partial \Omega_2}{\partial w_j} = \lambda w_j$$

Full-batch and SGD updates include this penalty gradient.
Its magnitude decreases as the coefficient becomes smaller.

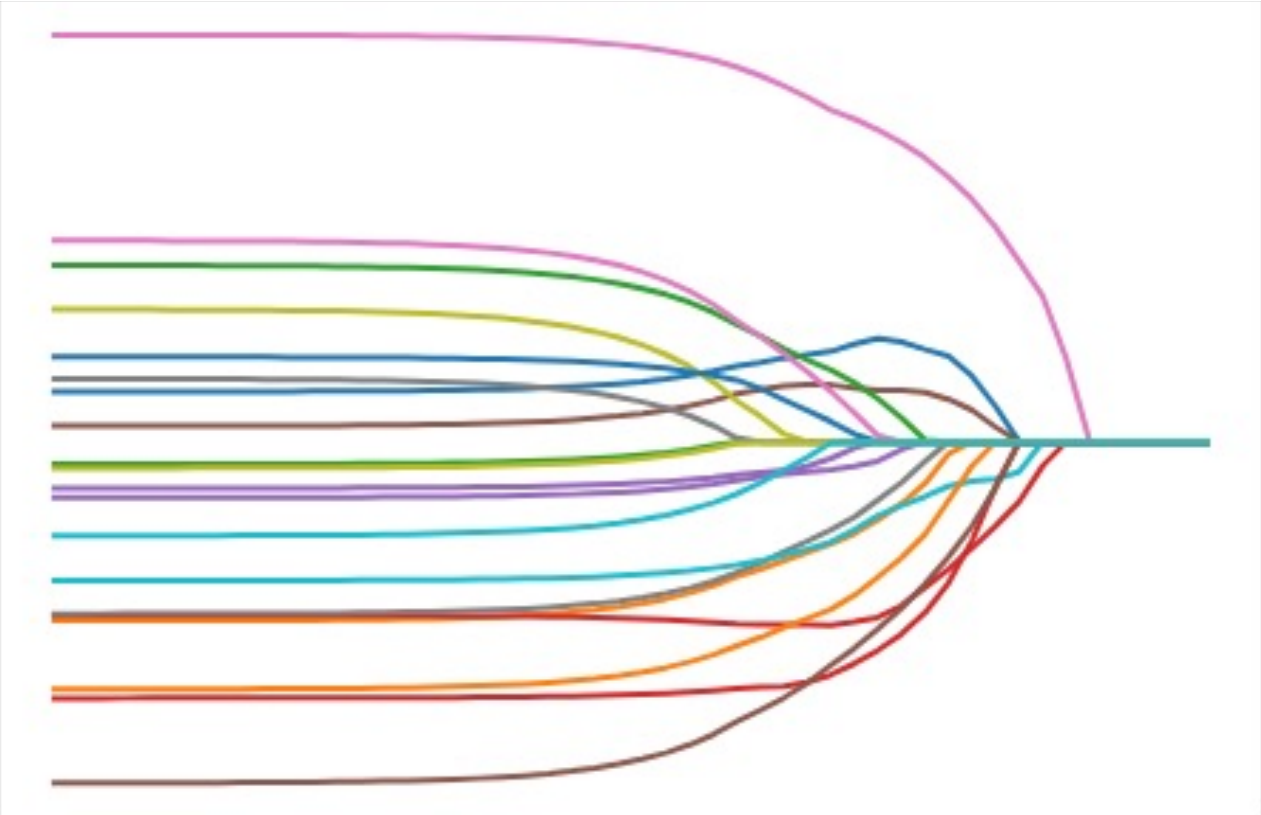
Considering the penalty alone

$$w_{t+1,j} = (1 - \eta\lambda) w_{t,j}, \qquad 0 < \eta\lambda < 1$$

L2 typically shrinks coefficients without setting them to zero.

Can we fit well using only a subset of the features?

L1 regularisation and LASSO



LASSO: least absolute shrinkage and selection operator

Tibshirani (1996), JRSS B

LASSO: an L1 coefficient penalty

Fit the data while penalising the sum of absolute coefficient values.

$$J_{\text{lasso}}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda \|\mathbf{w}_{1:D}\|_1$$

w_1:D excludes the intercept w_0. The intercept is not penalised.

$$\|\mathbf{w}_{1:D}\|_1 = \sum_{j=1}^{D} |w_j|$$

L1 norm

Some coefficients can become exactly zero.
LASSO selects a subset of the features.

Tibshirani (1996), JRSS B

L1 shrinkage and feature selection

The L1 penalty has a gradient of constant magnitude away from zero:

$$\frac{\mathrm{d}}{\mathrm{d}w_j}\left[\lambda|w_j|\right] = \lambda\ \mathrm{sign}(w_j), \qquad w_j \neq 0$$

The pull does not weaken as a coefficient approaches zero.
The corner at zero allows an optimum with exact zeros.

Selecting features for prediction

Zero coefficients remove features from the fitted predictor.
Selection concerns predictive value given the other features.

Choose λ using validation. Put features on comparable scales.

The same L1 penalty can also be used with logistic loss.

Regularisation penalties and constraints

LASSO penalises L1. Ridge penalises L2 squared.

Penalty formulation

$$\min_{\mathbf{w}} \ \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda \|\mathbf{w}_{1:D}\|_1$$

Constraint formulation

$$\min_{\mathbf{w}} \ \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 \quad \text{s.t.} \quad \|\mathbf{w}_{1:D}\|_1 \leq r$$

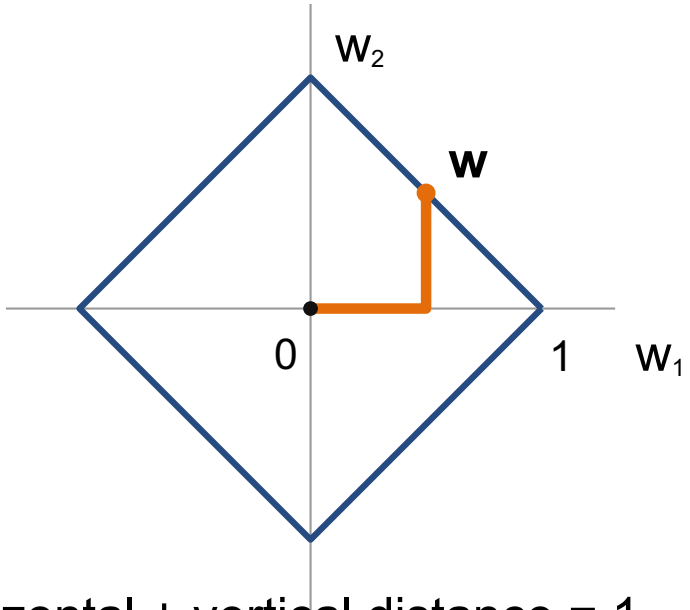
r ≥ 0 is the constraint radius. s.t. means subject to.
For each penalty strength, compare its corresponding constraint radius.

Unit balls: L1 and L2 norms

The geometry concerns the non-intercept coefficients w_1:D.

L1: Manhattan distance

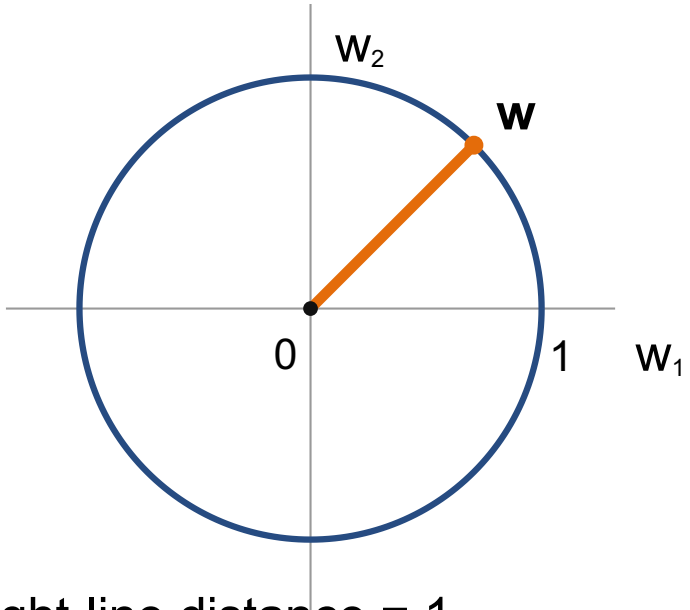
$$\|\mathbf{w}_{1:D}\|_1 = \sum_{j=1}^{D} |w_j|$$



Horizontal + vertical distance = 1

L2: Euclidean distance

$$\|\mathbf{w}_{1:D}\|_2 = \left(\sum_{j=1}^{D} w_j^2 \right)^{1/2}$$

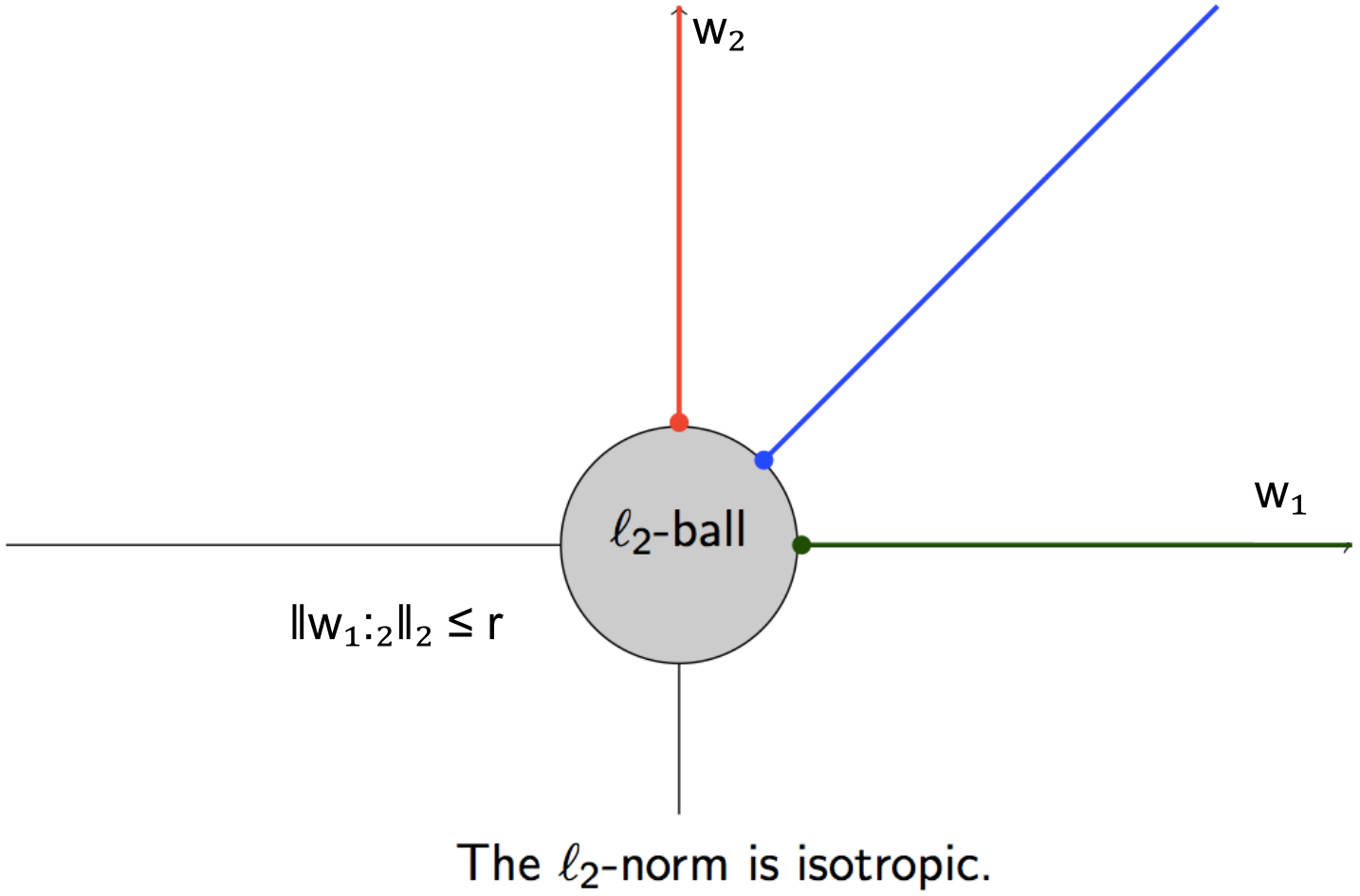


Straight-line distance = 1

Boundary: norm = 1. Unit ball: norm ≤ 1.

L2 geometry: projection onto a ball

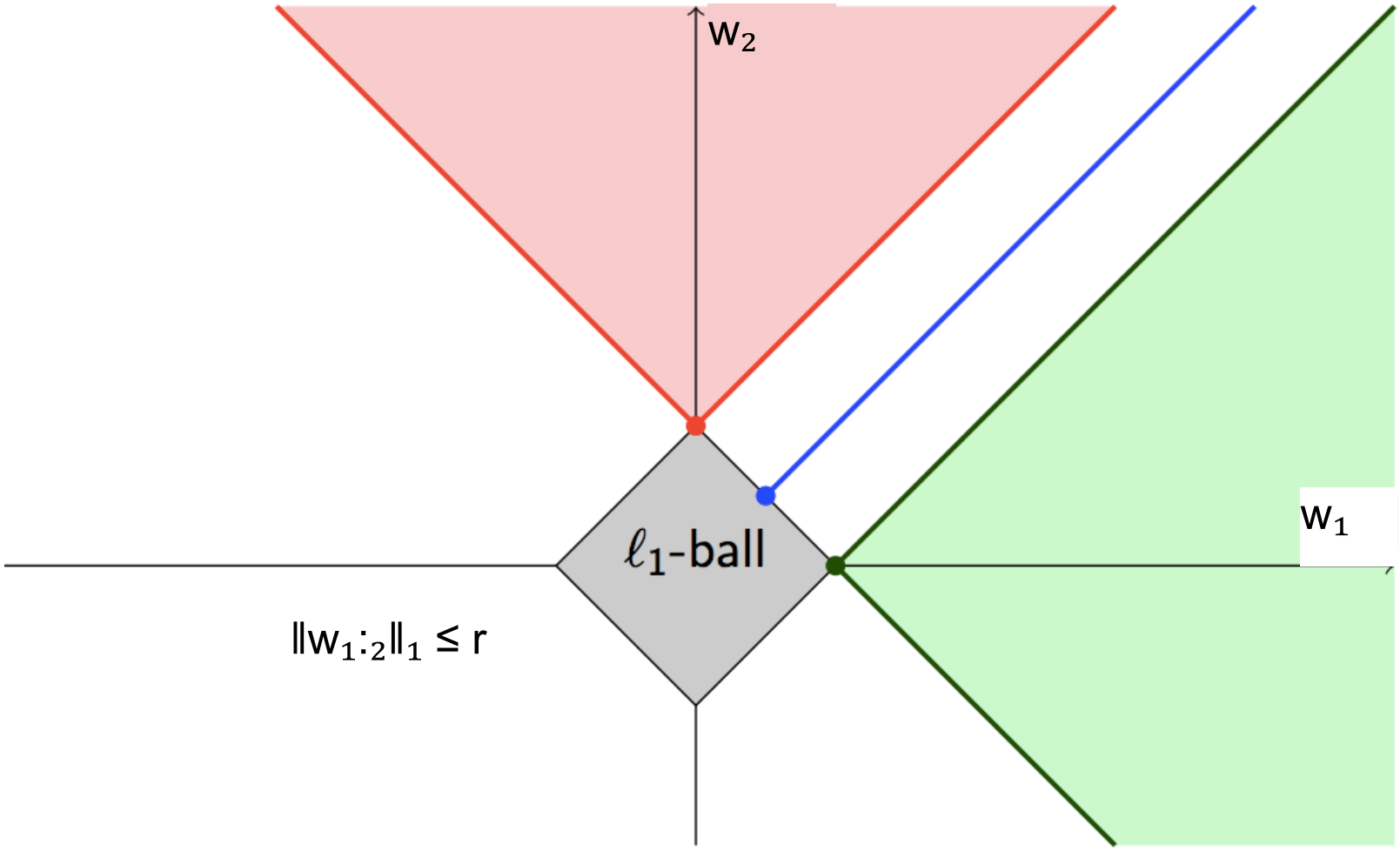
Euclidean projection illustration



The nearest point usually has several nonzero coordinates.

L1 geometry: projection onto corners

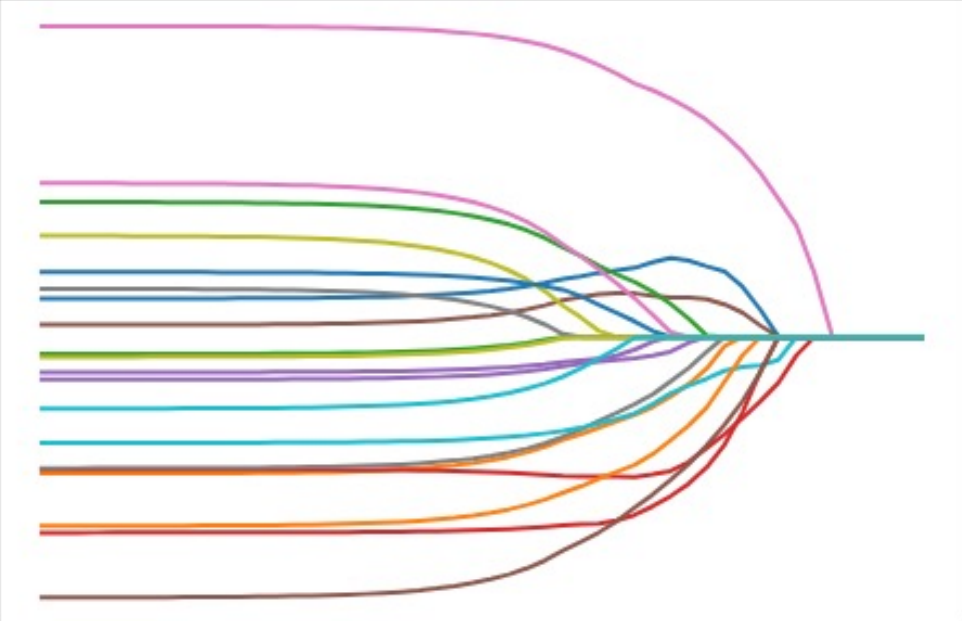
Euclidean projection illustration



Many coefficient vectors project onto a corner, where one coordinate is zero.

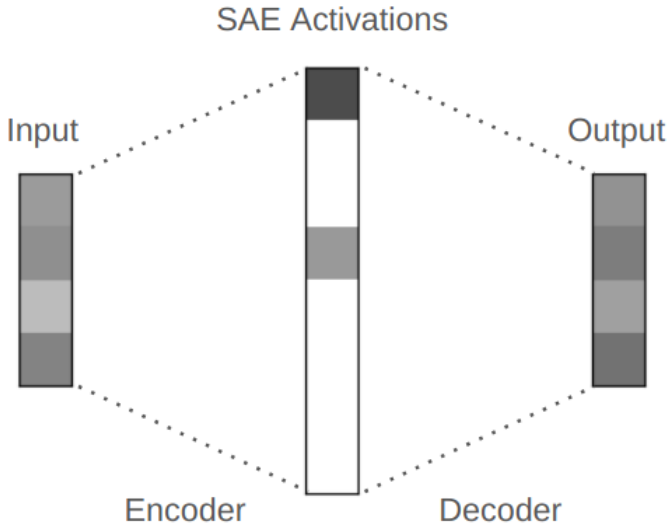
L1 sparsity and interpretability

LASSO selects coefficients.
A sparse autoencoder (SAE) learns a representation with few active units.



LASSO: least absolute shrinkage and selection operator

Tibshirani (1996), JRSS B



Cunningham et al. (2023): sparse autoencoders

https://arxiv.org/abs/2309.08600

Lecture 2: Classification & optimisation

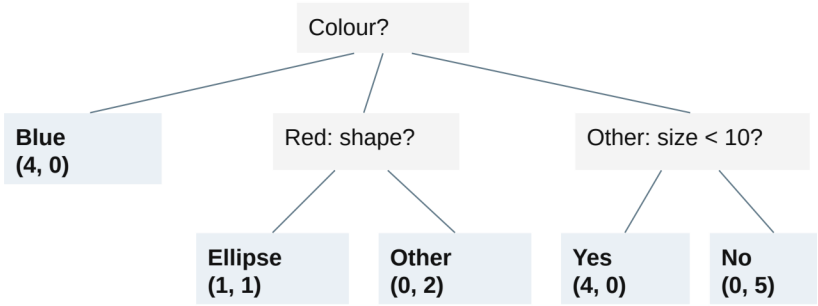
Classification

Optimisation

Trees and ensembles

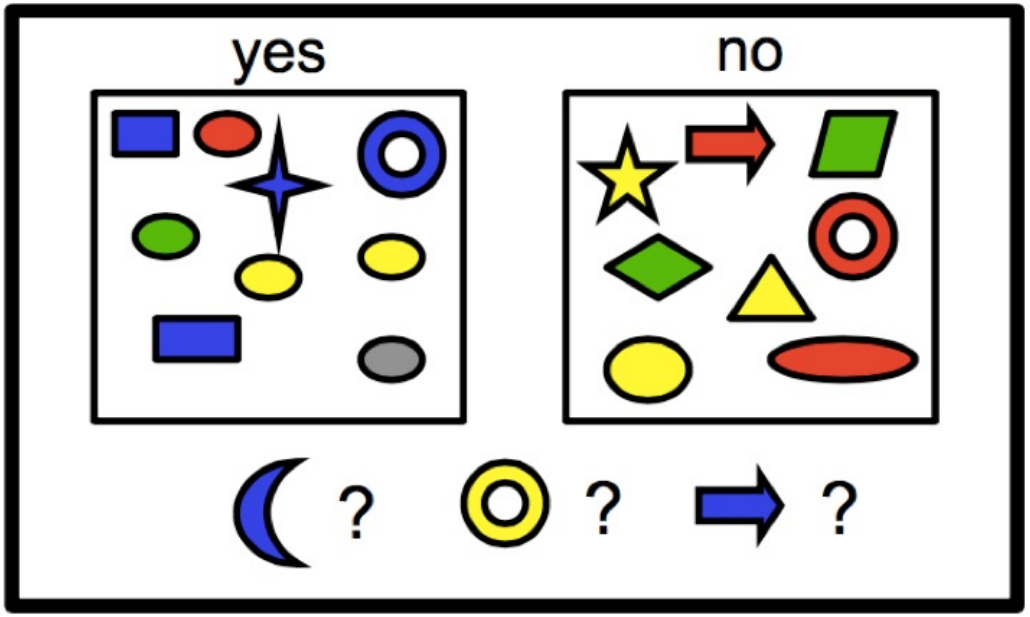
- **Decision trees**
- Ensembles and bagging
- Random forests

Decisions and evaluation



Murphy, Probabilistic Machine Learning: An Introduction
§18.1: classification and regression trees

A tree asks feature questions

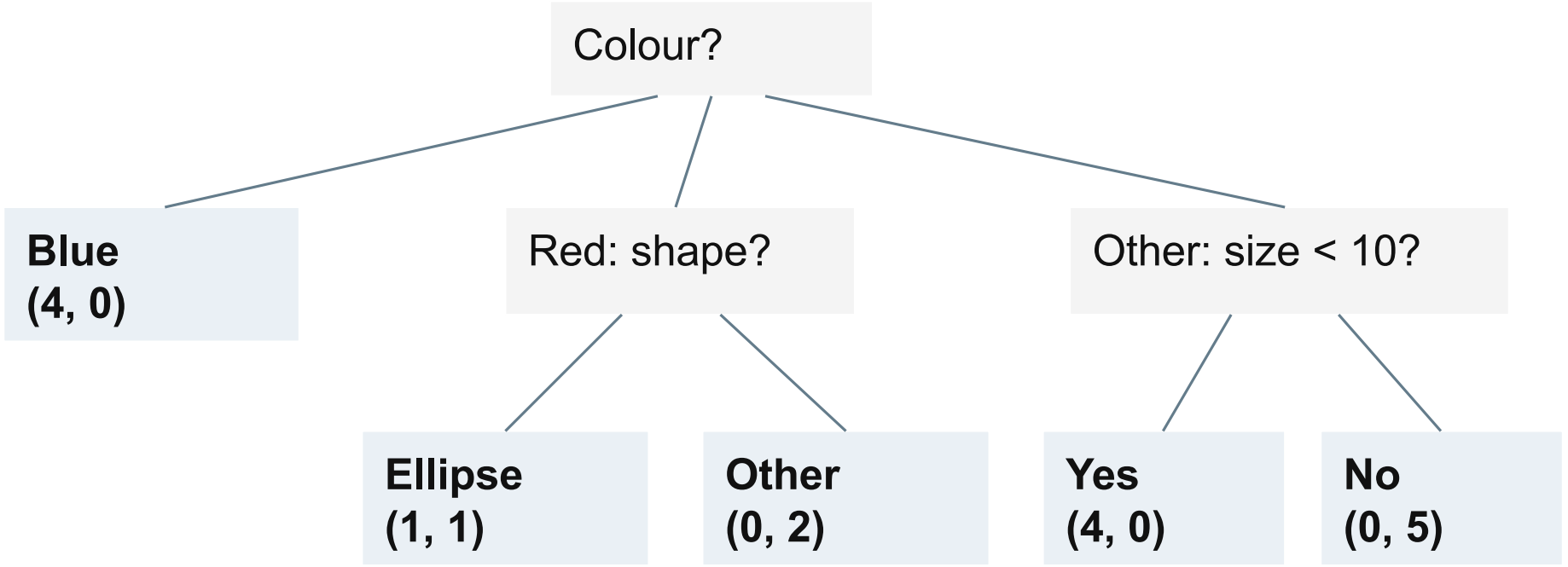


Features: color, shape, size

Machine learning: can work also for discrete inputs, strings, trees, graphs,…

A tree predicts from its leaves

Inputs have colour, shape and size. A test sends each input to a child.

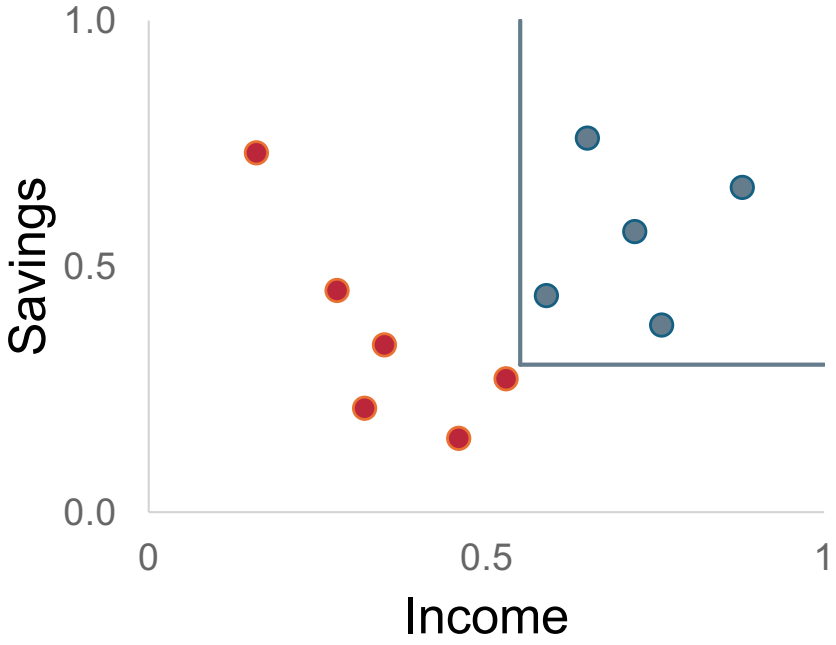


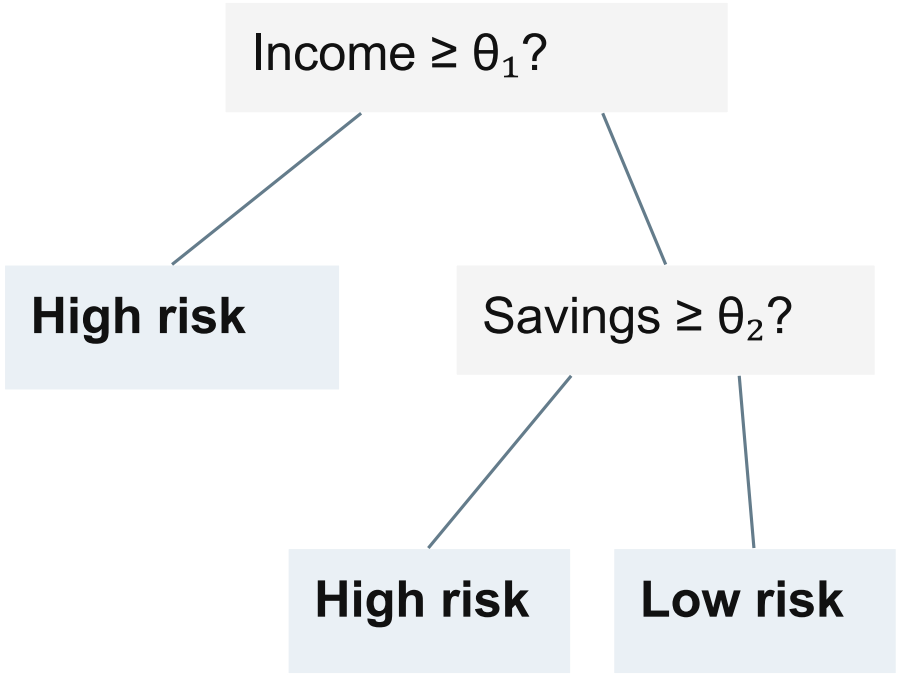
Each pair is (yes-label count, no-label count) at a leaf.
Predict by the leaf's class frequencies, or choose the majority label.

Depth counts tests along a path

Income and savings define a simple risk classification example.

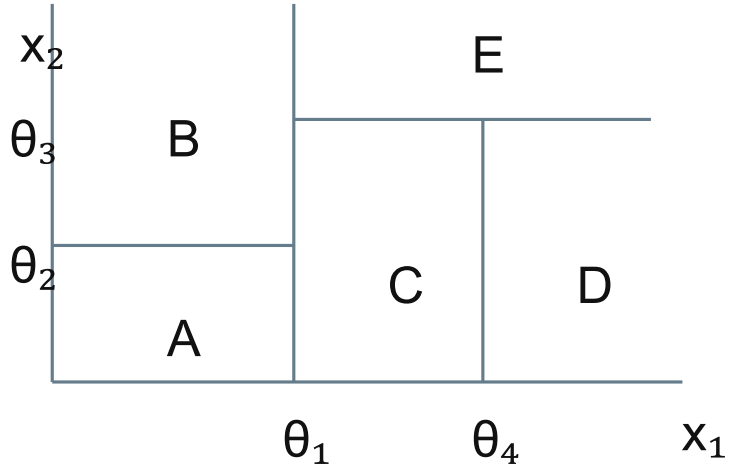
Red: high risk. Blue: low risk.

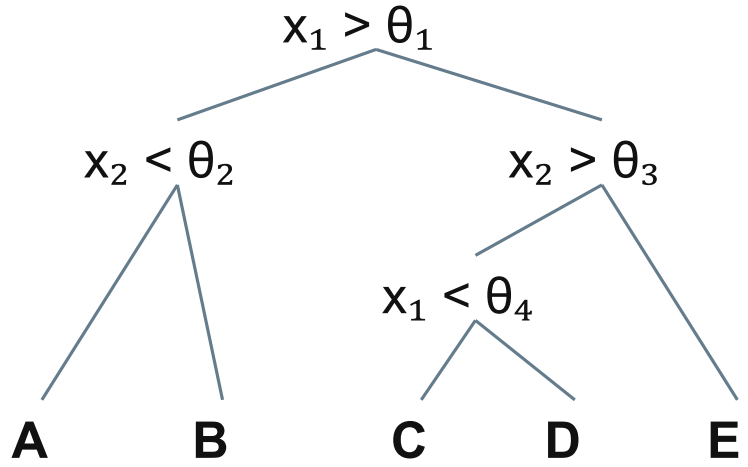




Maximum depth: 2 tests. A leaf can occur after only 1 test.
θ_1 and θ_2 are feature thresholds. The plotted points are schematic.

Decision-tree leaf models and objective





$\mathcal{R}_j$ is a leaf region. In regression, μ_j and σ_j^2 are its mean and variance. For classification, μ_j is the probability of class 1 in that leaf.

Regression

$$p(y_i \mid \mathbf{x}_i \in \mathcal{R}_j) = \mathcal{N}(y_i \mid \mu_j, \sigma_j^2)$$

Binary classification

$$p(y_i \mid \mathbf{x}_i \in \mathcal{R}_j) = \text{Bernoulli}(y_i \mid \mu_j)$$

$$J(T) = \lambda C(T) - \sum_{i=1}^{N} \log p(y_i \mid \mathbf{x}_i; T)$$

C(T) measures tree complexity, such as the number of nodes. $\lambda \geq 0$.

Greedy decision-tree learning

Start with a leaf — Choose a split — Grow the children

There are many possible tree structures and split thresholds.
For a fixed structure, estimating leaf distributions is straightforward.
Searching all structures is difficult.

A greedy method chooses the best local split and keeps that choice.
Repeat at each child, with stopping rules that control complexity.
Use validation data to choose depth and other settings.

CART means Classification and Regression Trees. C4.5 and ID3 are related methods.

Which objective should choose the next split?

Information and entropy

Information content of an outcome x

$$I(x) = \log_2 \frac{1}{P(x)}$$

measure of surprise: predictable events do not provide information

The French live to eat and the British eat to

live (low IC, cliché)

die (high IC, funny)

Entropy of a distribution: expected information content of a random variable

$$H(X) = \sum_{x \in A_X} P(x) \log_2 \frac{1}{P(x)}$$

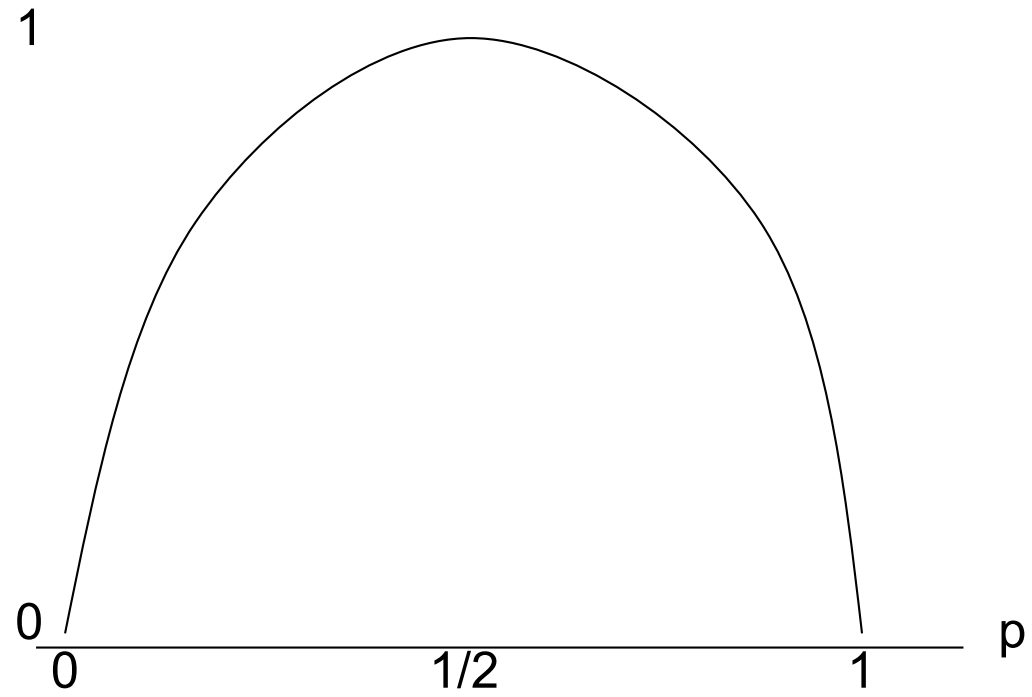
measure of uncertainty: should we expect to be surprised?

A_X: possible outcomes of X. Entropy is in bits here.

Binary entropy

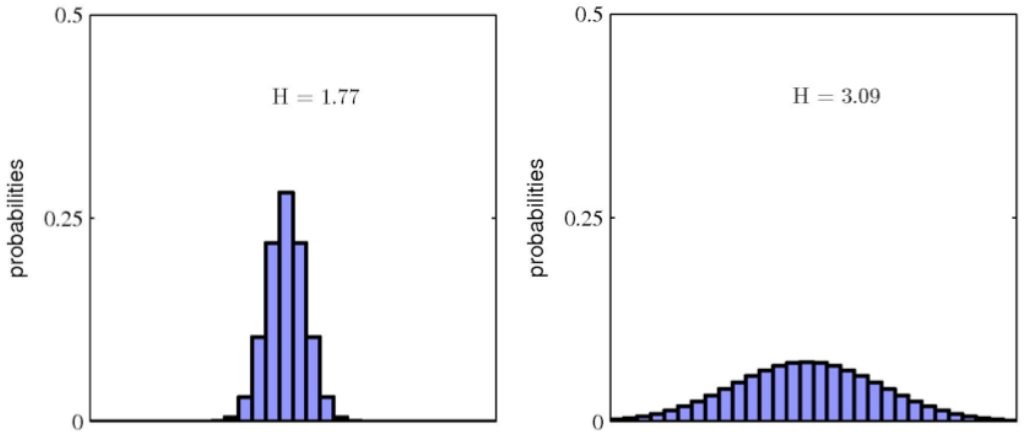
P(X=1) = p
P(X=0) = 1−p

$H = p \log_2(1/p) + (1-p) \log_2(1/(1-p))$



p = 0.5: as unpredictable as it gets
p = 0, or p=1: no information gained by observing

Entropy measures uncertainty



- Histograms of two probability distributions over 30 bins.
- The largest entropy will arise from a uniform distribution
  H = -ln(1/30) = 3.40.

For class labels Y: p_c = P(Y = c). There are C classes. Uniform: H = log C.

Discrete entropy

$$H(Y) = -\sum_{c=1}^{C} p_c \log p_c$$

Here natural logs give entropy in nats.
Use the same log base for every term of an information-gain comparison.

A split score ranks candidate tests

Choose a feature test h from the feature pool and a threshold θ.
I_n indexes examples at the parent node; I_l and I_r are its children.

$$I_{\mathrm{L}} = \{i \in I_n : h_n(\mathbf{x}_i) < \theta_n\}$$
$$I_{\mathrm{R}} = I_n \setminus I_{\mathrm{L}}$$

$$\theta_n \in \left(\min_{i \in I_n} h_n(\mathbf{x}_i), \max_{i \in I_n} h_n(\mathbf{x}_i) \right)$$
$$IG = H(I_n) - \frac{|I_{\mathrm{L}}|}{|I_n|} H(I_{\mathrm{L}}) - \frac{|I_{\mathrm{R}}|}{|I_n|} H(I_{\mathrm{R}})$$

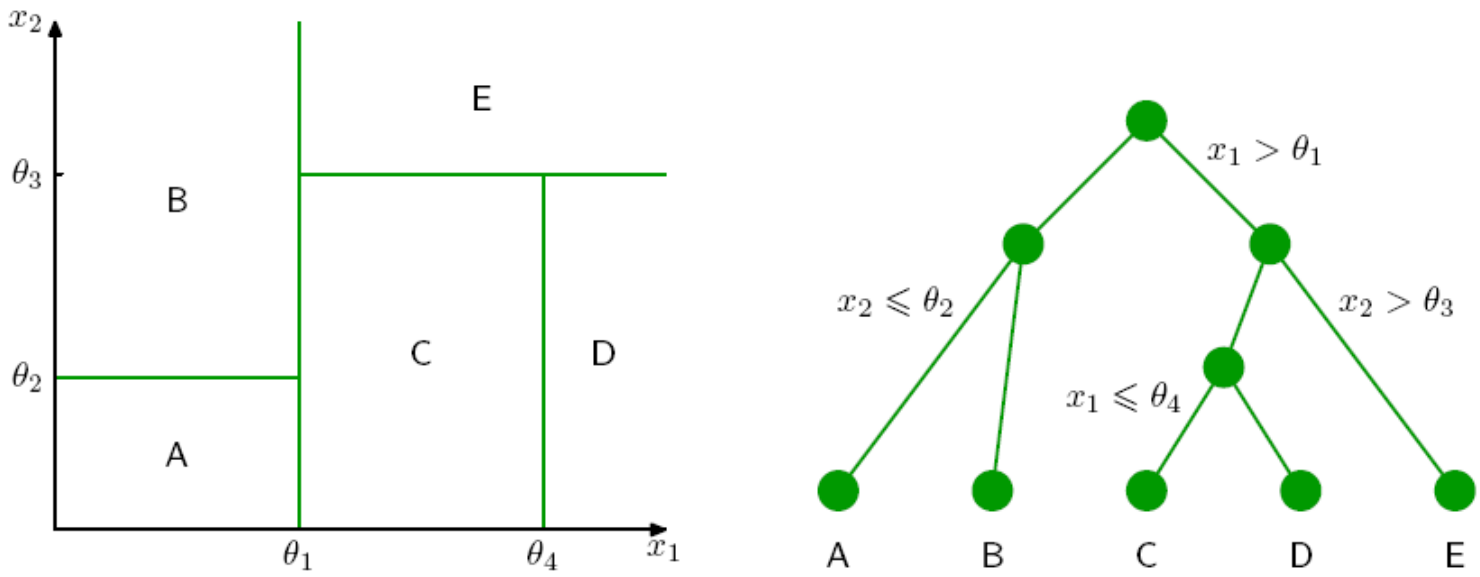
H(I) is the entropy of the class-label histogram for examples in I.
For a fixed parent, maximising IG minimises weighted child entropy.
Choose h and θ to maximise this information gain.

Decision trees: fitting and complexity

- Fast greedy training algorithms
  - Work with heterogeneous pool of features (e.g. shape, size, color,…)
- Fast testing algorithm
- Needs careful choice of hyper-parameters
  - maximum depth
  - number of features and thresholds to try
- Prone to over-fitting

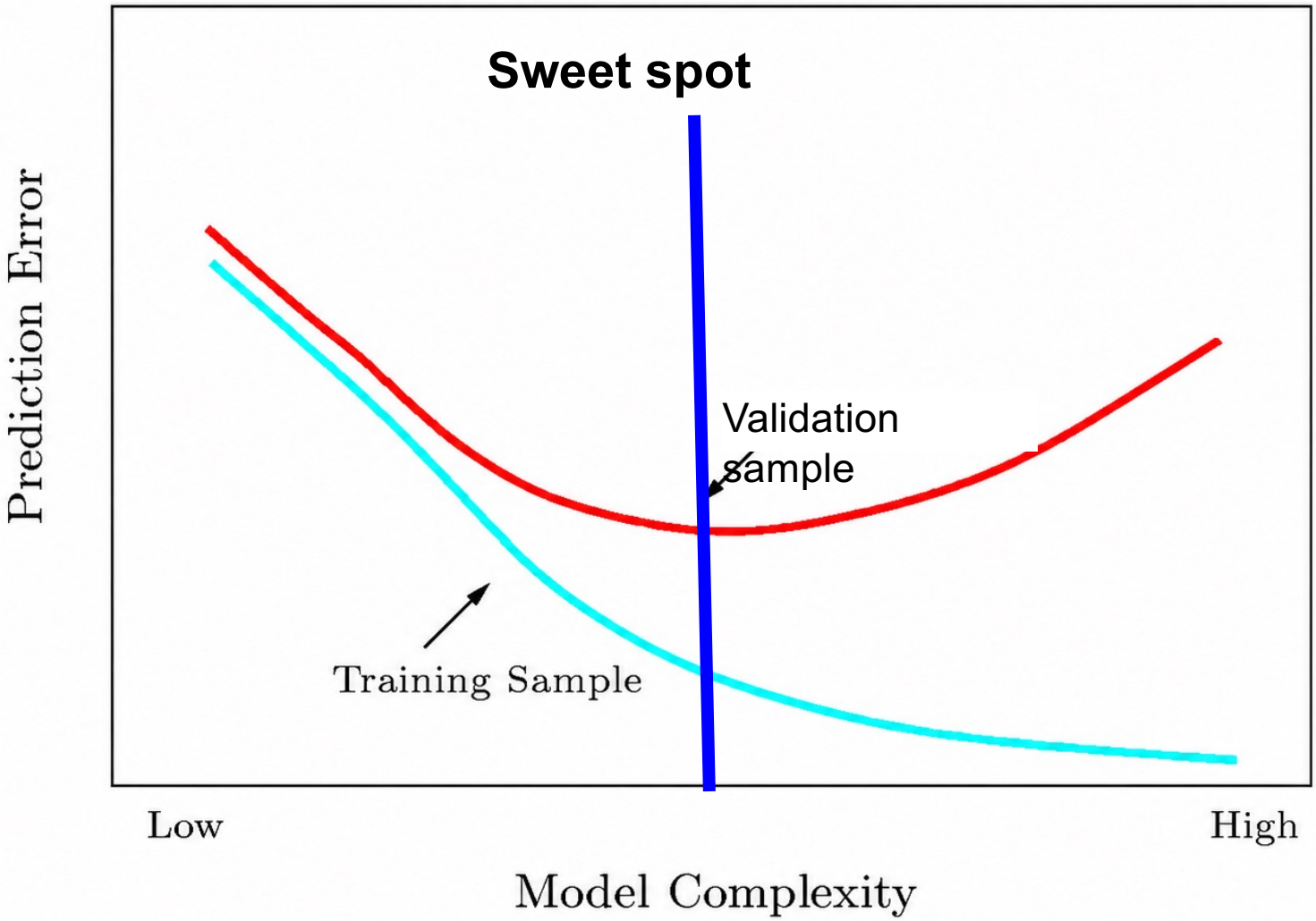
Tree depth and overfitting

Deeper trees make more complex classifiers.



Beyond a useful depth, fitting training detail can hurt unseen performance.

Model complexity and held-out error



Larger ridge λ means lower complexity. The held-out curve is schematic.

Generalisation: a Lecture 1 reminder

Lower training loss does not guarantee better predictions.

Validation chooses model complexity and penalty strength.

A flexible model can change substantially when its training set changes.
Averaging learners can reduce this variation when their errors differ.

Next: why ensembles help, and how random forests create diversity.

The Lecture 1 risk and bias–variance details remain in the appendix.

Lecture 2: Classification & optimisation

Classification

Optimisation

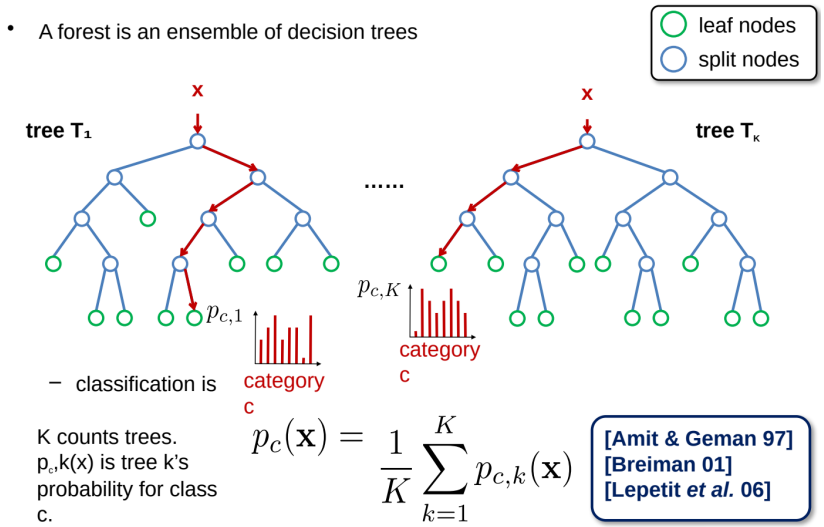
Trees and ensembles

Decision trees

Ensembles and bagging

Random forests

Decisions and evaluation



Murphy, Probabilistic Machine Learning: An Introduction
§18.3: bagging

Ensembles combine predictions

1) Train several base learners whose prediction errors differ.

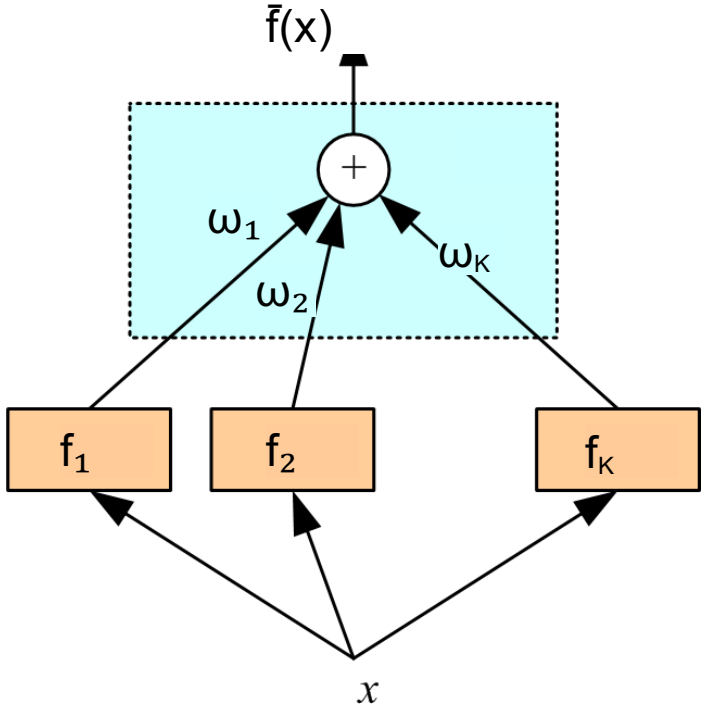
2) Obtain an aggregate decision by combining their results

Two main methods are bagging and boosting.
Here we study bagging and random forests.

Base learners and their combination

- Combine several base learners; improvement depends on error dependence.

In this diagram:
f_k is base learner k.
ω_k is its combination weight.
The top node combines the outputs.
K is the number of learners.



Error of an average predictor

K predictors $f_k(x)$ form a committee.
$f^*(x)$ is the true function and $\varepsilon_k(x)$ the error of predictor k.

$$\overline{f}(\mathbf{x}) = \frac{1}{K}\sum_{k=1}^{K} f_k(\mathbf{x}), \qquad f_k(\mathbf{x}) = f^*(\mathbf{x}) + \epsilon_k(\mathbf{x})$$

E_k is expected squared error. E_{AV} averages it over the K predictors.

$$E_k = \mathbb{E}_{\mathbf{x}}\left[\epsilon_k(\mathbf{x})^2\right], \qquad E_{\mathrm{AV}} = \frac{1}{K}\sum_{k=1}^{K} E_k$$

$$E_{\mathrm{COM}} = \mathbb{E}_{\mathbf{x}}\left[(\overline{f}(\mathbf{x}) - f^*(\mathbf{x}))^2\right] = \mathbb{E}_{\mathbf{x}}\left[\left(\frac{1}{K}\sum_{k=1}^{K}\epsilon_k(\mathbf{x})\right)^2\right]$$

Uncorrelated errors can average out

Assume zero-mean errors, uncorrelated across different predictors.

$$\mathbb{E}_{\mathbf{x}}\left[\epsilon_k(\mathbf{x})\epsilon_{k'}(\mathbf{x})\right] = 0 \quad \text{for } k \neq k'$$

Expand the square. Cross terms vanish for k ≠ k′.

$$E_{\mathrm{COM}} = \frac{1}{K^2}\left(\sum_k \mathbb{E}_{\mathbf{x}}[\epsilon_k^2] + \sum_k \sum_{k' \neq k} \mathbb{E}_{\mathbf{x}}[\epsilon_k \epsilon_{k'}]\right)$$

$$= \frac{1}{K^2}\sum_k E_k = \frac{1}{K} E_{\mathrm{AV}}$$

Averaging reduces independent variation. Shared bias remains.

Lecture 2: Classification & optimisation

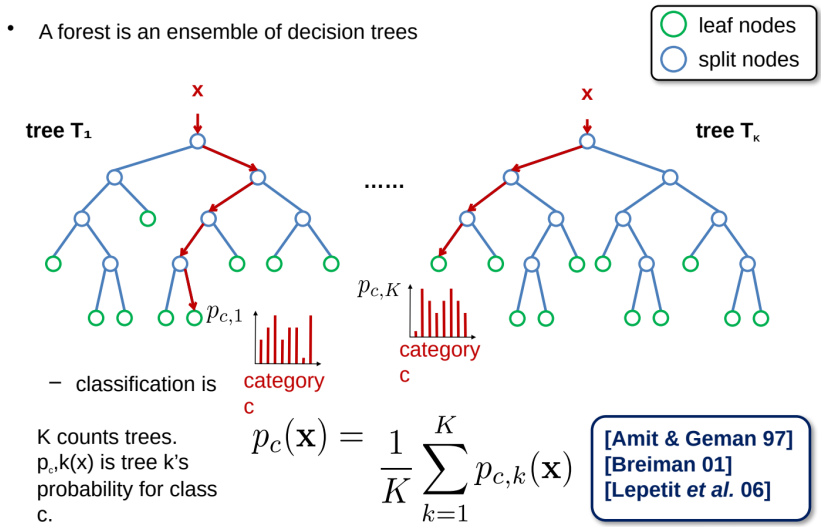
Classification

Optimisation

Trees and ensembles

- Decision trees
- Ensembles and bagging
- **Random forests**

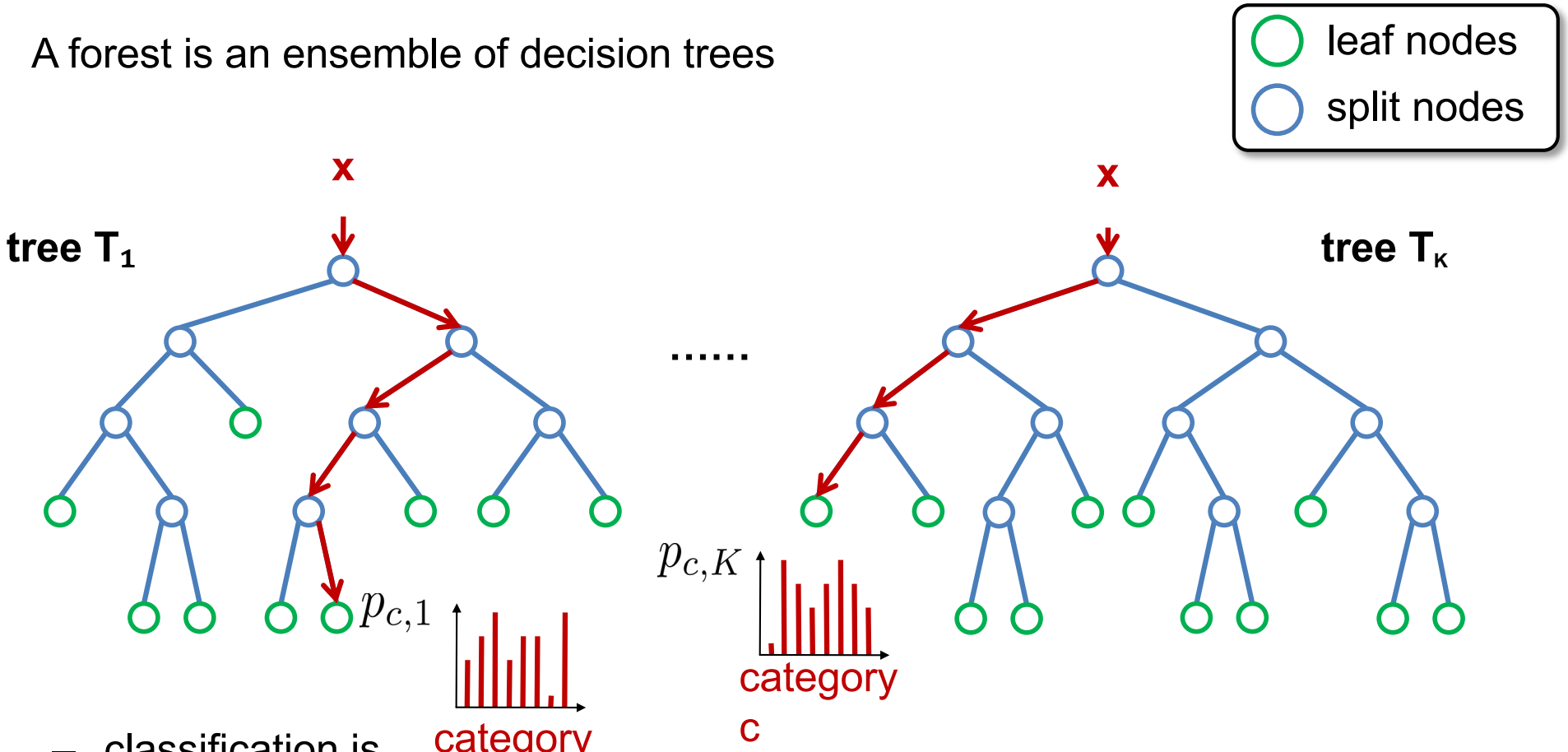
Decisions and evaluation



Murphy, Probabilistic Machine Learning: An Introduction
§18.4: random forests

A forest combines trees

- A forest is an ensemble of decision trees



  - classification is

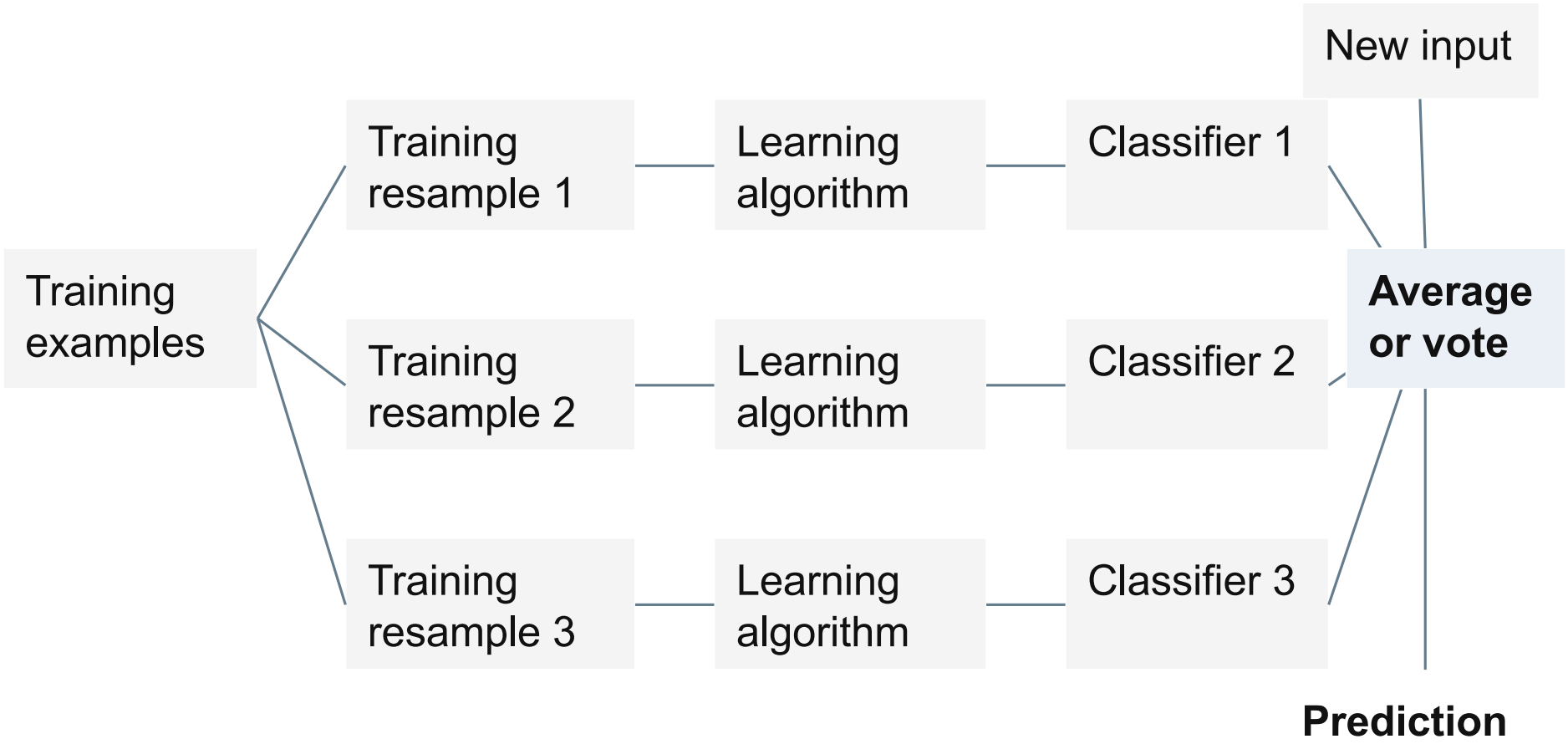
K counts trees.
p_c,k(x) is tree k's probability for class c.

$$p_c(\mathbf{x}) = \frac{1}{K}\sum_{k=1}^{K} p_{c,k}(\mathbf{x})$$

[Amit & Geman 97]
[Breiman 01]
[Lepetit *et al.* 06]

Bagging uses bootstrap samples

Draw N training examples with replacement for each resample.
Fit one base learner per resample, then combine votes or probabilities.



Training a random forest

- Draw N examples with replacement for each tree.
- At each node, search a random subset of measured features.
- Fit each tree independently, then combine predictions.

- Resampling and feature variation make trees less similar.
- Use validation or out-of-bag predictions (from trees that omitted each example) to choose complexity.

A bootstrap sample

Original indices: 1 2 3 4 5 6

One bootstrap sample: 1 1 2 4 4 6.
Six draws with replacement; repeated indices repeat observations.

Out-of-bag examples for this tree: 3 and 5.
Evaluate them using only trees that omitted them during fitting.

At each node, also sample candidate features.
Bootstrap samples overlap; they are not disjoint partitions.

Randomized Learning

- Recursively split examples at node n.
- I_n indexes labelled training examples (x_i, y_i).

left split

$$I_{\mathrm{L}} = \{i \in I_n \mid h_n(\mathbf{x}_i) < \theta_n\}$$

right split

$$I_{\mathrm{R}} = I_n \setminus I_{\mathrm{L}}$$

threshold

function of example i's feature vector

h_n is the feature test at node n. θ_n is its split threshold.

Randomising candidate features

Choose a random subset of candidate tests from feature pool F.
For each candidate h, search thresholds in its observed feature range.

$$\theta \in [\min_{i \in I_n} h(\mathbf{x}_i),\ \max_{i \in I_n} h(\mathbf{x}_i)]$$

$$I_l = \{i \in I_n : h(\mathbf{x}_i) < \theta\}$$

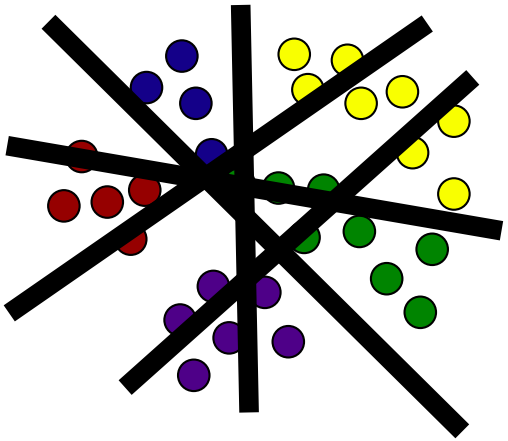
$$I_r = I_n \setminus I_l$$

Choose h and θ to maximise the quality of the child nodes.
For entropy, use the information-gain criterion introduced earlier.
Random feature subsets make the trees less correlated.

Why do random tests work?

For a small number of classes

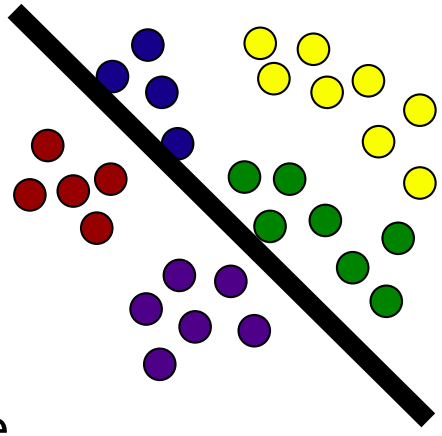
- we can try several tests, and
- retain the best one according to some criterion.



Why do random tests work?

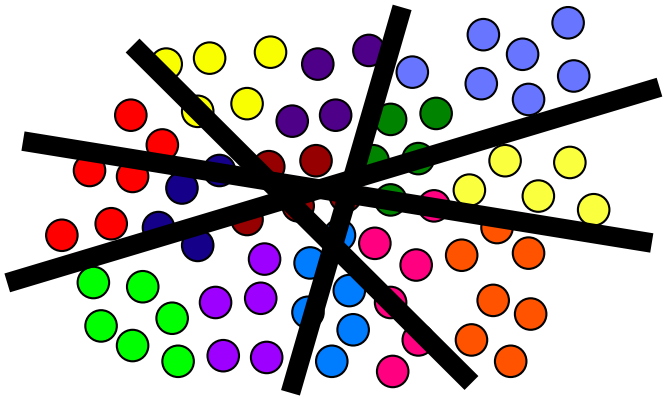
For a small number of classes

- we can try several tests, and
- retain the best one according to some criterion.



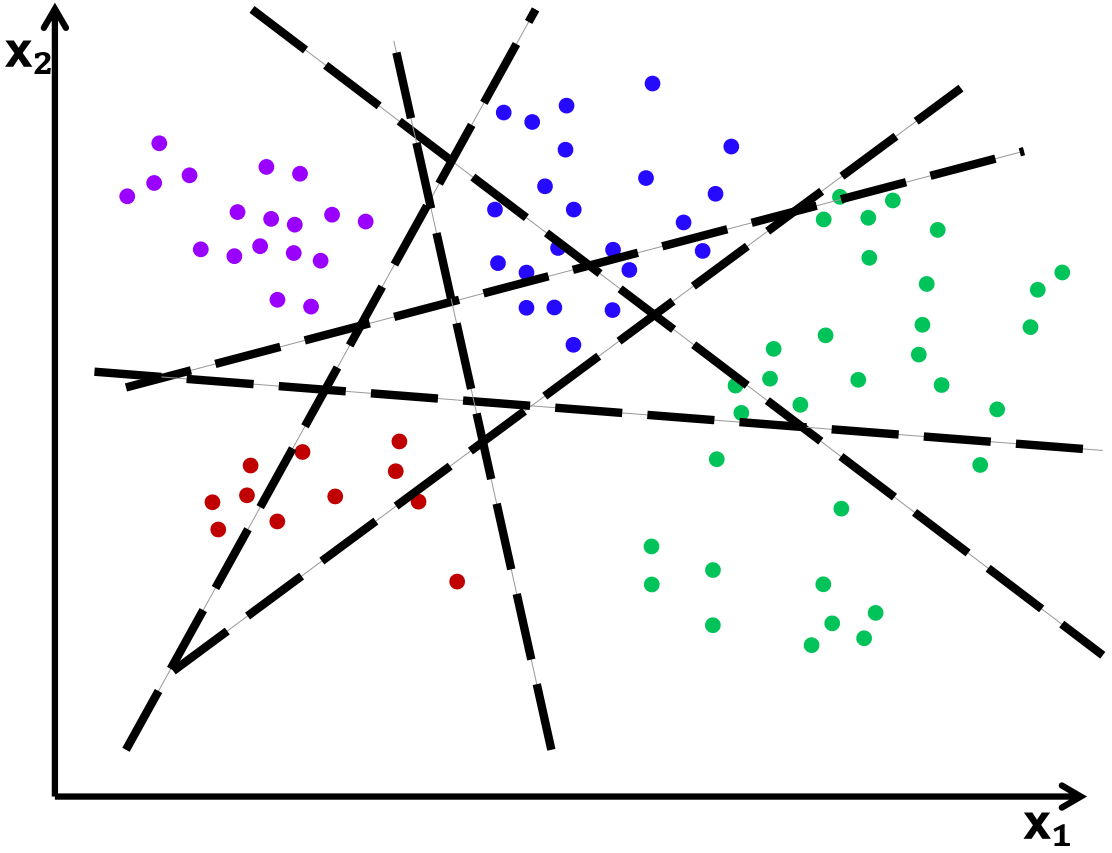
In this many-class example,

- many candidate tests give useful splits:



Toy Learning Example

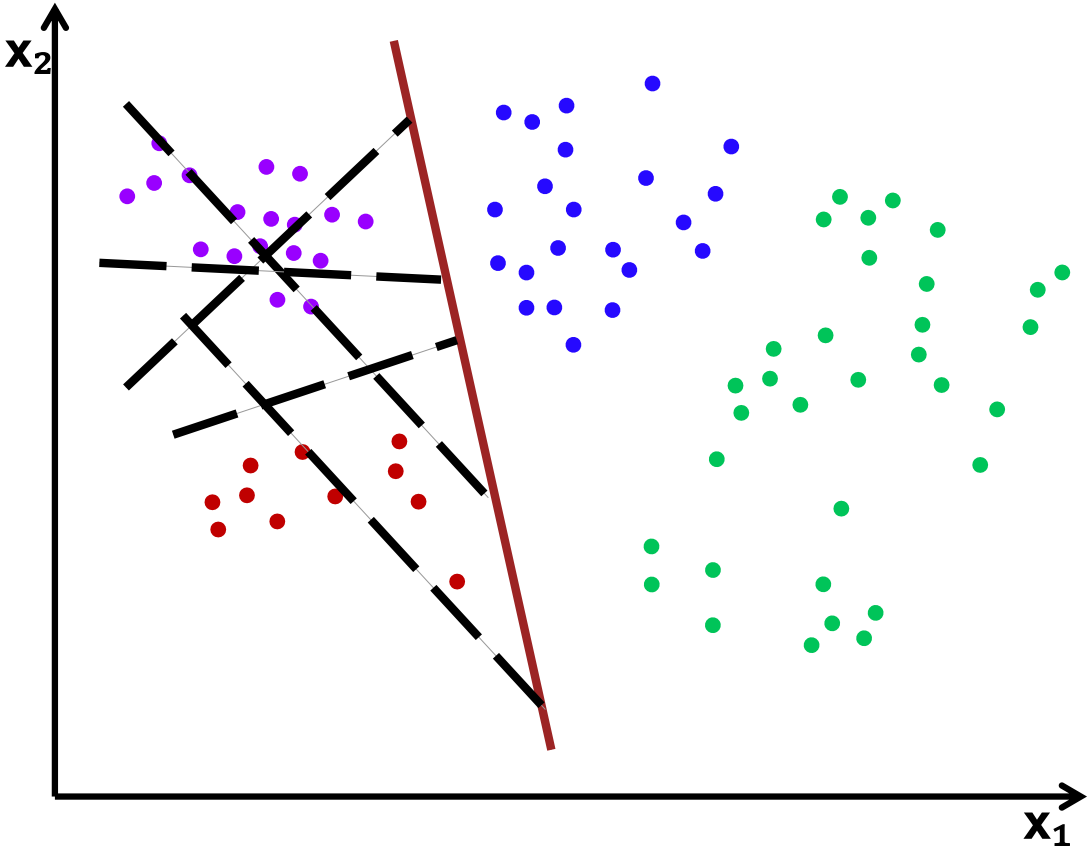
- **Try several lines, chosen at random**
- **Keep line that best separates data**
  - information gain
- **Recurse**



- **Feature vector: $x = (x_1, x_2)^T$**
- **At node n, test $h_n(x) = w_{n1}x_1 + w_{n2}x_2$ against θ_n.**
- **Four classes: purple, blue, red, green.**

Toy Learning Example

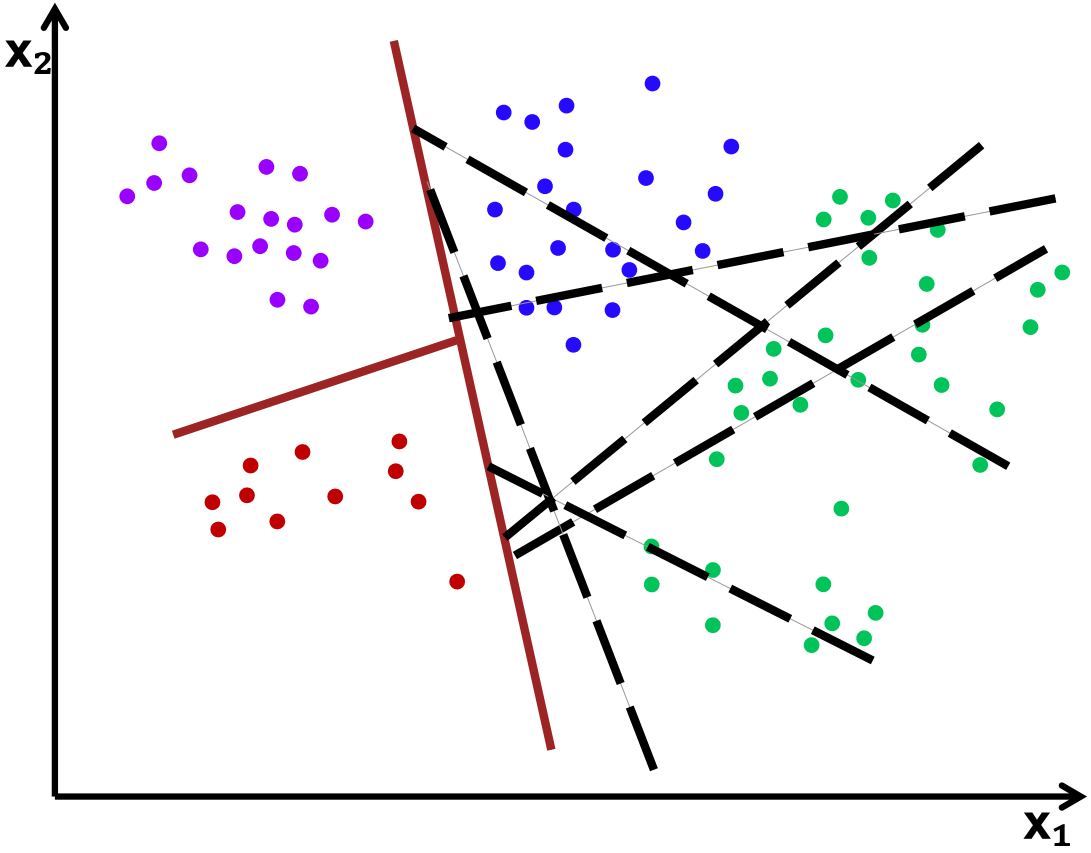
- **Try several lines, chosen at random**
- **Keep line that best separates data**
  - information gain
- **Recurse**



- **Feature vector: $x = (x_1, x_2)^T$**
- **At node n, test $h_n(x) = w_{n1}x_1 + w_{n2}x_2$ against θ_n.**
- **Four classes: purple, blue, red, green.**

Toy Learning Example

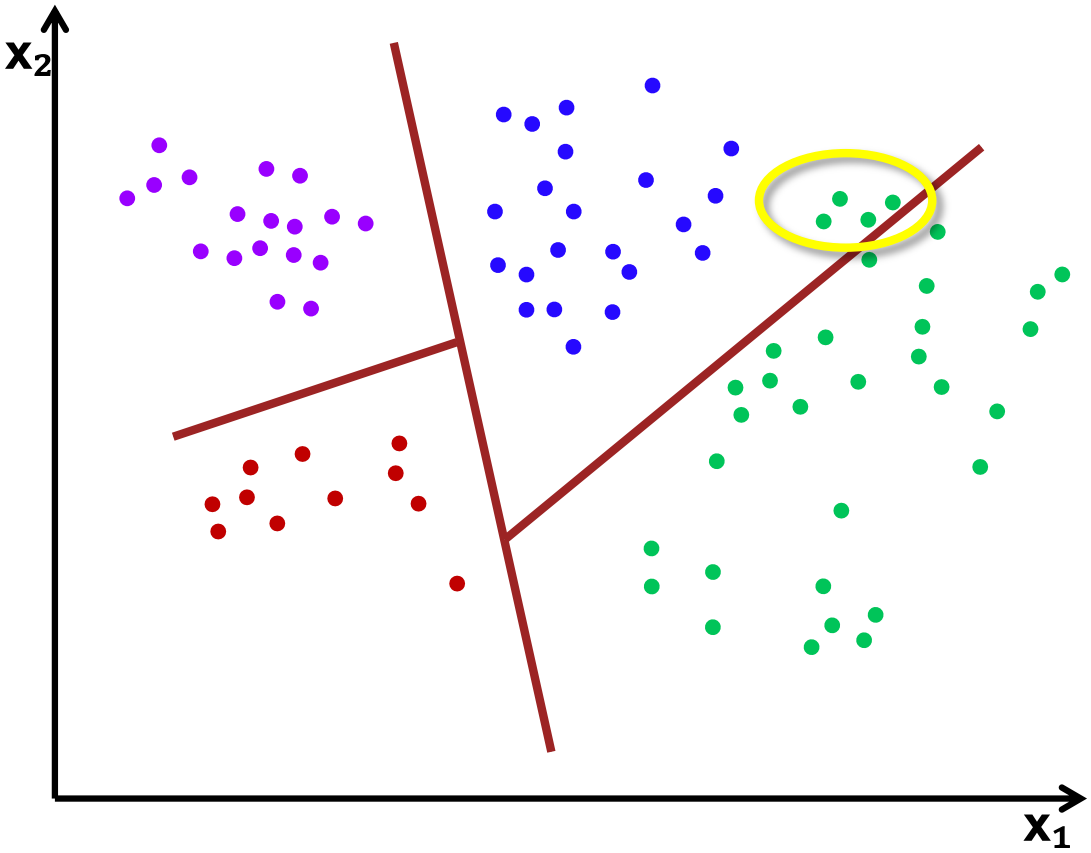
- **Try several lines, chosen at random**
- **Keep line that best separates data**
  - information gain
- **Recurse**



- **Feature vector: $x = (x_1, x_2)^T$**
- **At node n, test $h_n(x) = w_{n1}x_1 + w_{n2}x_2$ against θ_n.**
- **Four classes: purple, blue, red, green.**

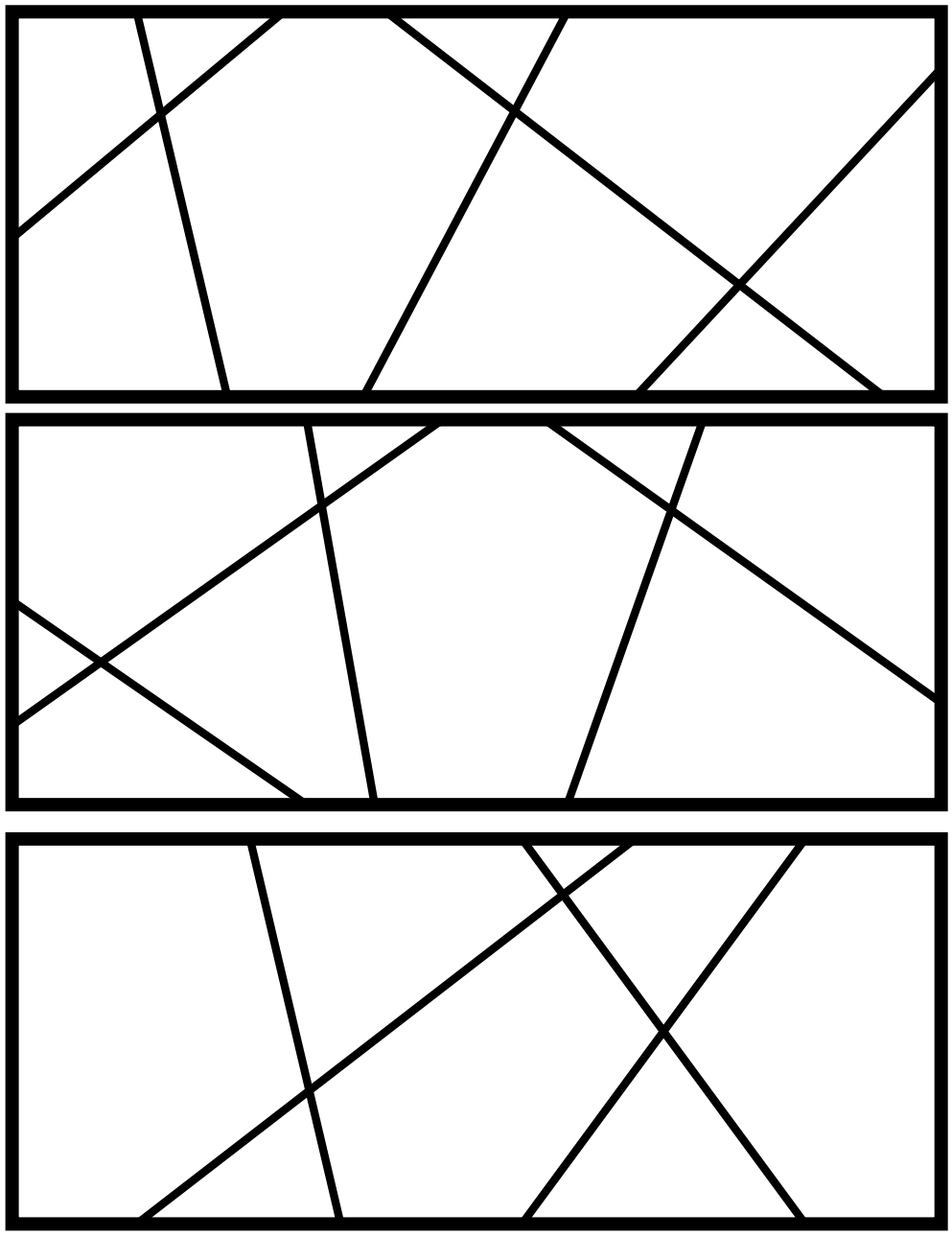
Toy Learning Example

- Try several lines, chosen at random
- Keep line that best separates data
  - information gain
- Recurse



- **Feature vector: $x = (x_1, x_2)^T$**
- **At node n, test $h_n(x) = w_{n1}x_1 + w_{n2}x_2$ against θ_n.**
- **Four classes: purple, blue, red, green.**

Different trees partition the input space



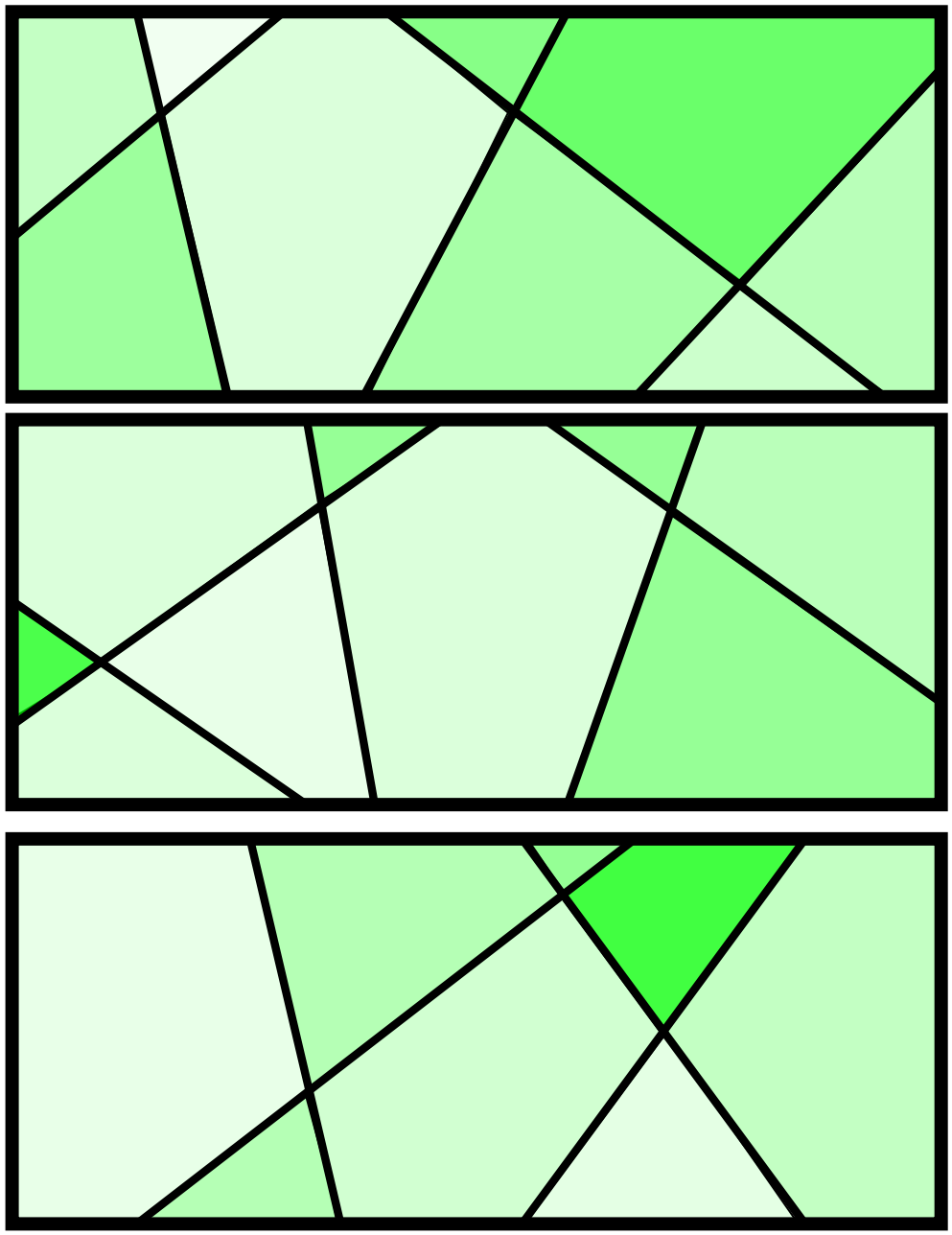
Different tree partitions.

Each leaf carries a class probability

Tree 1

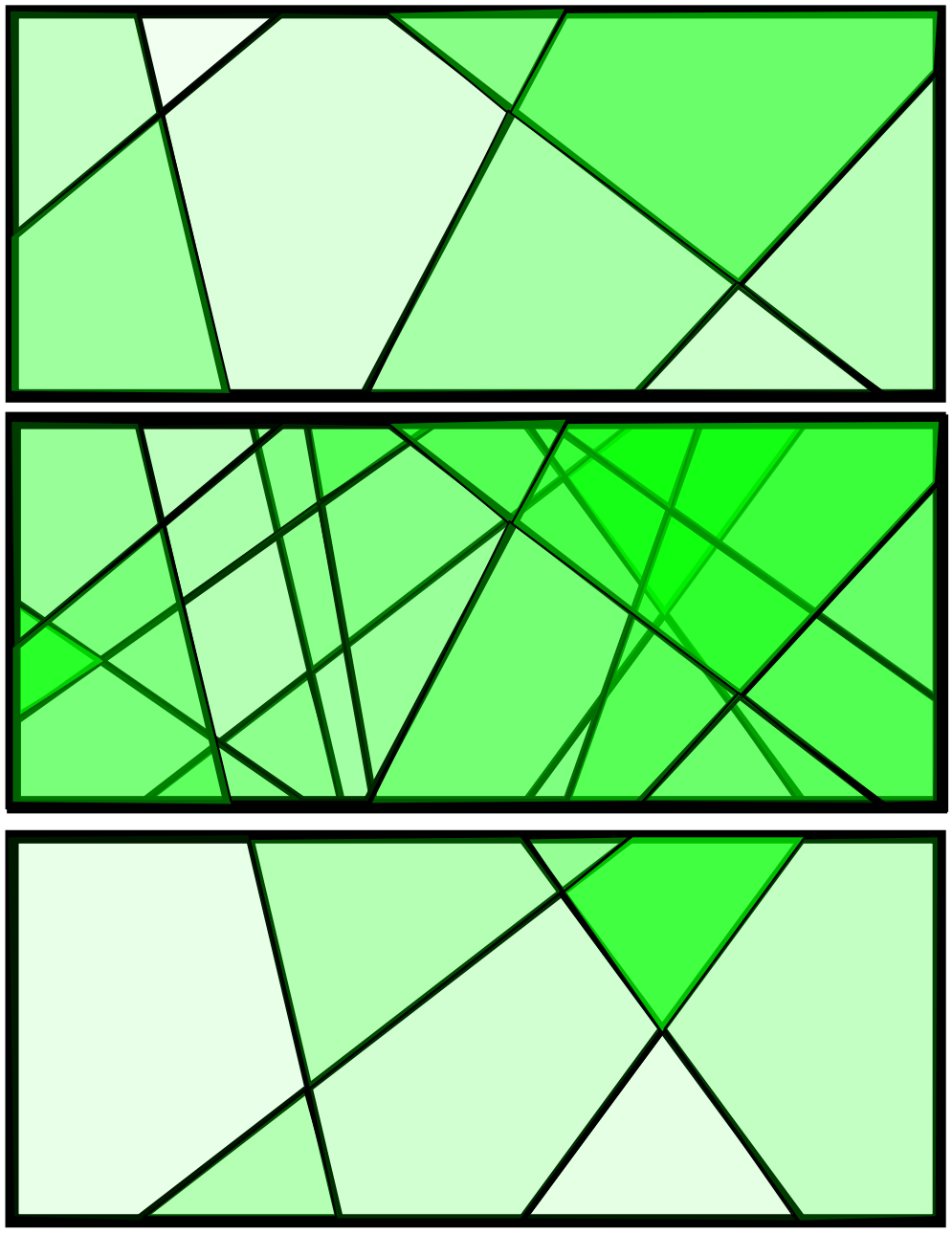
Tree 2

Tree 3



One class probability in each leaf. Darker green means higher probability.

Combining tree probability maps

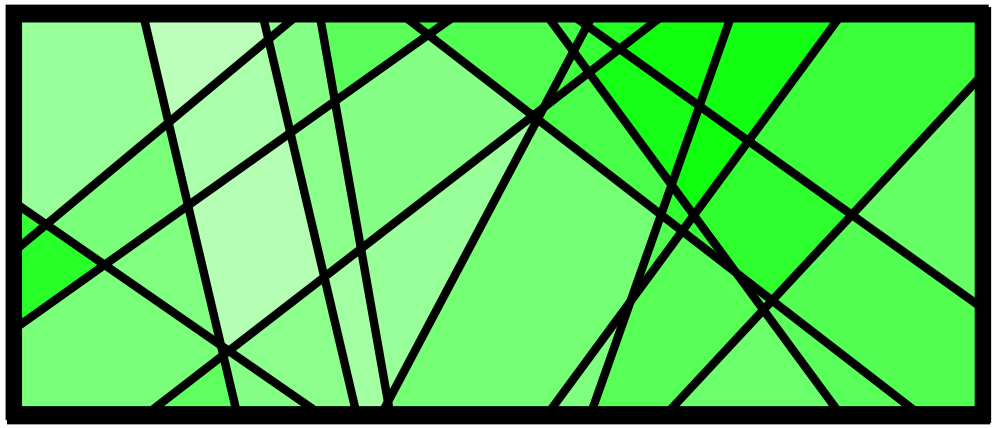


Combined

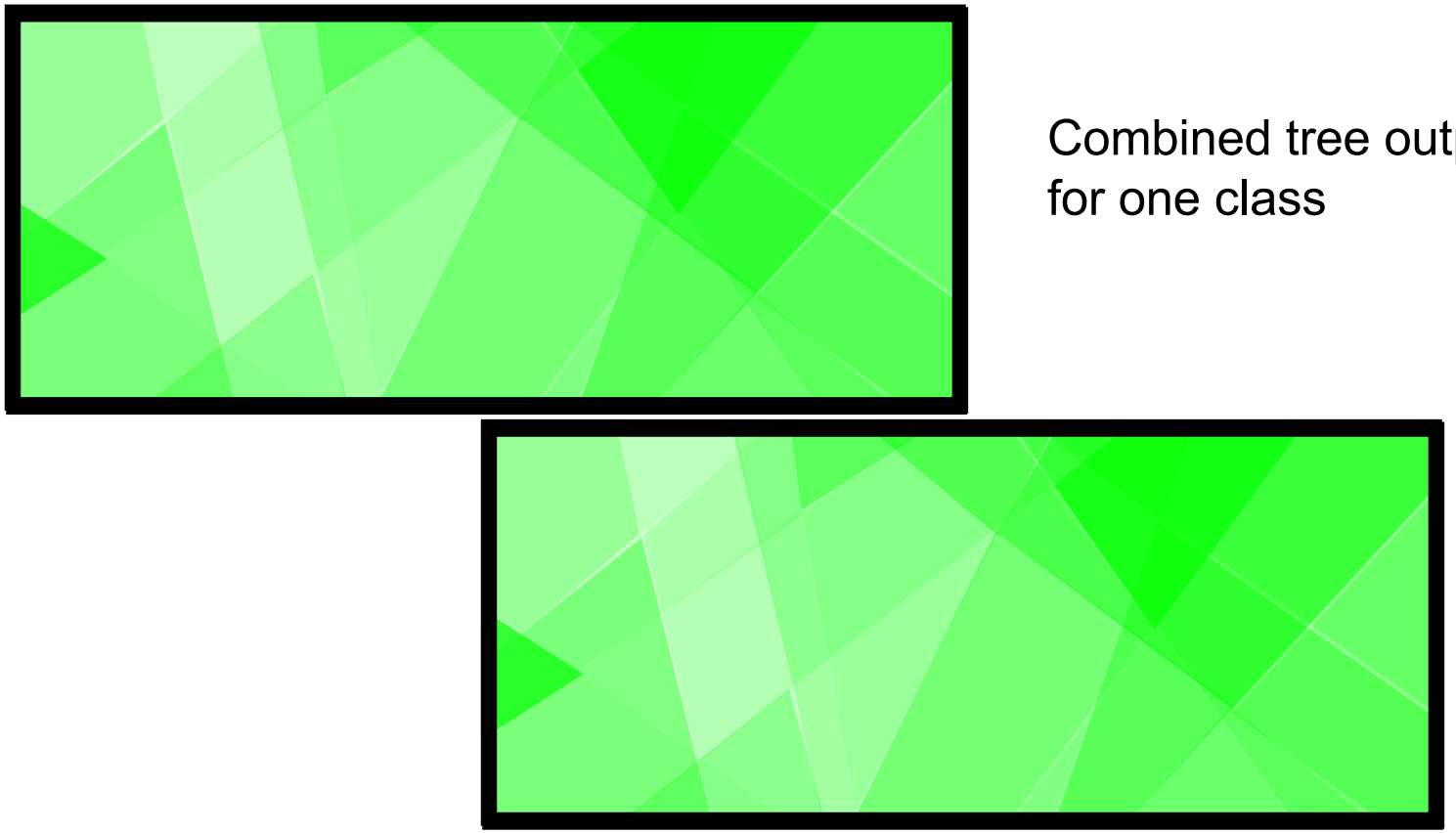
Combining
tree outputs.

A combined class probability map

Combined tree outputs
for one class

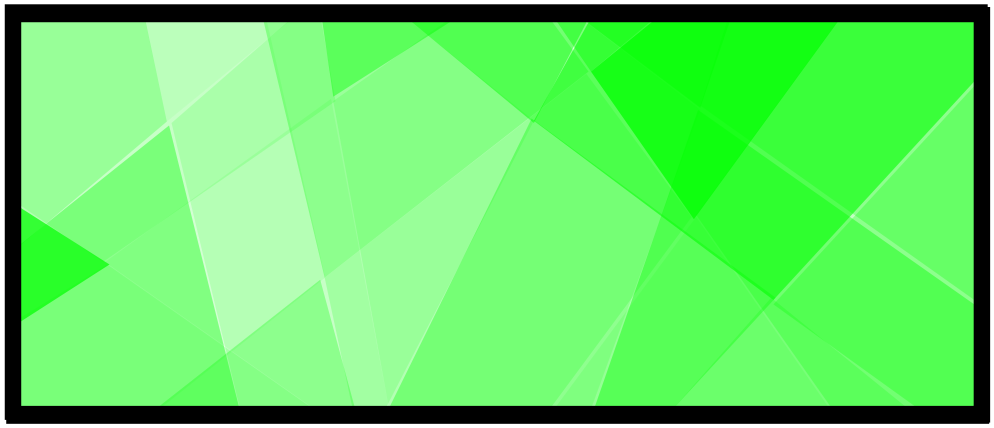


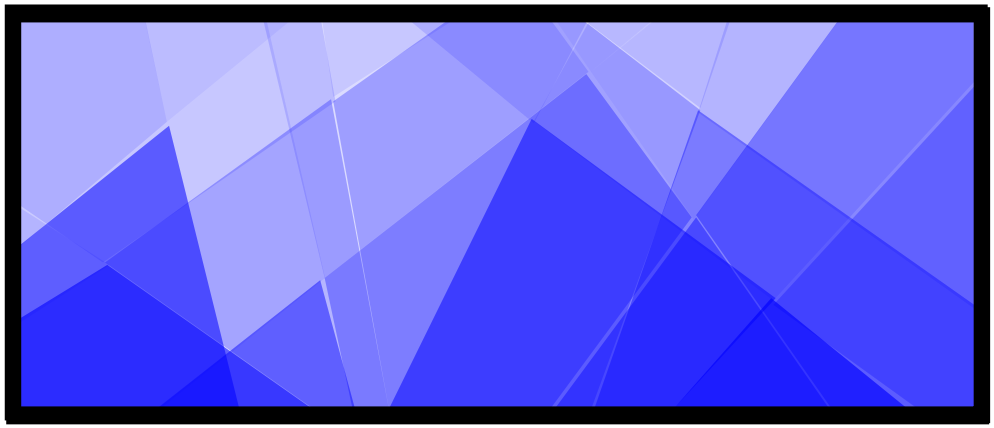
The combined map without split lines

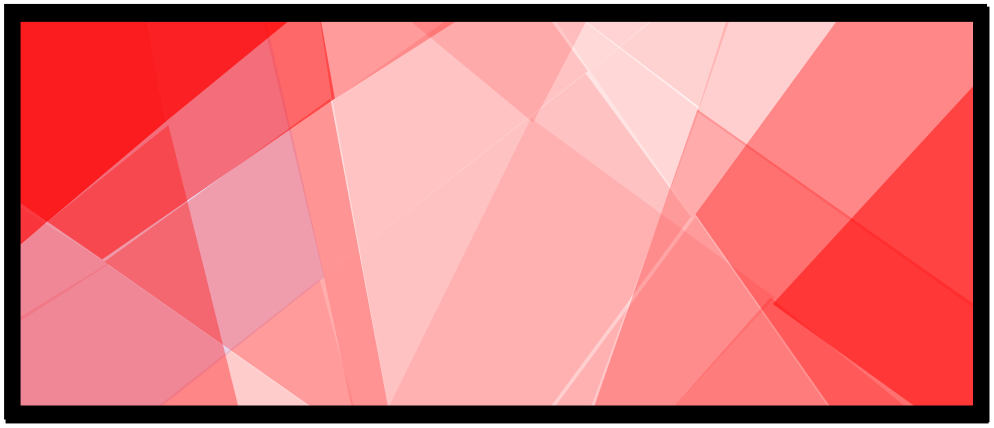


Combined tree outputs
for one class

A probability map for each class

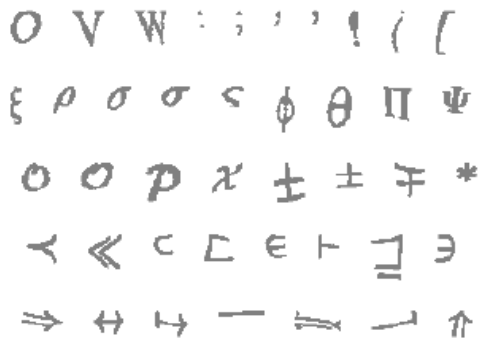






Class probability maps
Green, blue and red
show different classes.

Randomized Forests in Vision



[Amit & Geman, 97]
digit recognition



[Lepetit *et al.*, 06]
keypoint recognition



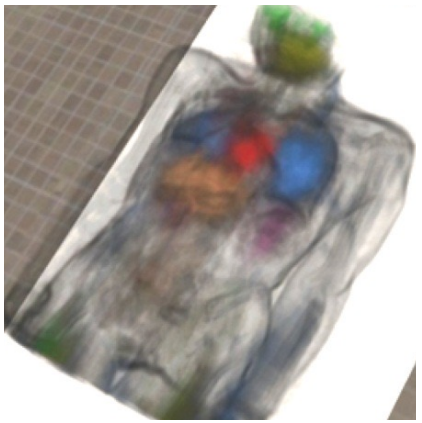
[Moosmann *et al.*, 06]
visual word clustering



[Shotton *et al.*, 08]
object segmentation



[Rogez *et al.*, 08]
pose estimation



[Criminisi *et al.*, 09]
organ detection

Selected applications

Kinect's pipeline







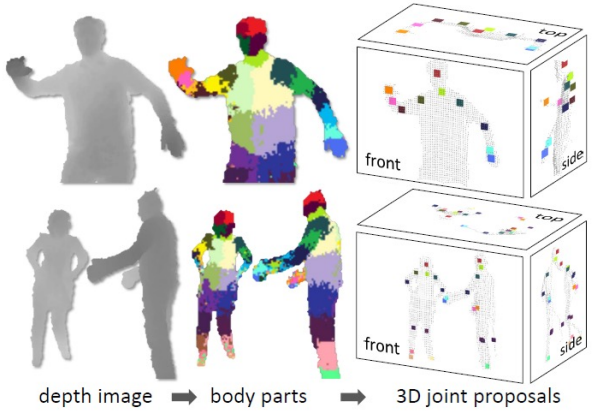


Figure 1. **Overview.** From an single input depth image, a per-pixel body part distribution is inferred. (Colors indicate the most likely part labels at each pixel, and correspond in the joint proposals). Local modes of this signal are estimated to give high-quality proposals for the 3D locations of body joints, even for multiple users.

A single depth image supplies a classification problem at each pixel.

A random forest predicts body-part labels.

The body-part distributions support 3D joint estimates.

Shotton et al., CVPR 2011: Real-Time Human Pose Recognition in Parts from a Single Depth Image

Lecture 2: Classification & optimisation

Classification

Optimisation

Trees and ensembles

Decisions and evaluation

- **Decision costs**
- Evaluation metrics
- Thresholds and reliability

Truth / decision	Class 0	Class 1
Class 0	Λ_{00}	Λ_{01}
Class 1	Λ_{10}	Λ_{11}

Murphy, Probabilistic Machine Learning: An Introduction
§§5.1.1–5.1.2: decision theory and classification

Fitting a classifier and evaluating it

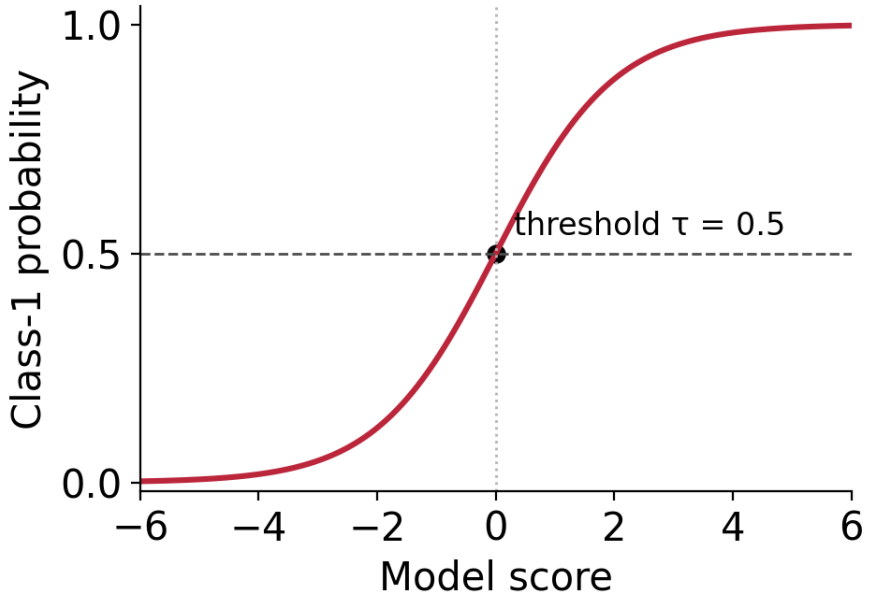
- Fit the classifier on training data. Choose settings and thresholds using validation.
- Use validation data to choose settings and a decision threshold.

- Report held-out classification performance after fitting.
- A low training loss is not evidence of reliable unseen predictions.

Classification thresholds

After fitting probabilities, choose how they become decisions.

$$\widehat{y} = \begin{cases} 1 & p_{\mathbf{w}}(\mathbf{x}) \geq \tau \\ 0 & p_{\mathbf{w}}(\mathbf{x}) < \tau \end{cases}$$



τ is the probability threshold. Equal error costs suggest τ = 1/2.
Choose other thresholds using validation data and the cost of mistakes.

Classes and decisions

The true class describes the input. A decision specifies an action.

Medical example	Possible values
True class	Sick or healthy
Decision	Treat, discharge, or request another examination

An extra examination is an action even though it is not a third true class.
Use a loss function to express the consequence of each action.

Asymmetric decision costs

A false negative and a false positive can have very different consequences.

True condition	Decision	Consequence
Sick	Healthy	Missed illness
Healthy	Sick	Unnecessary treatment

In this example, missing an illness carries a larger loss.
The preferred decision depends on both probabilities and error costs.

A loss matrix for decisions

$$\Lambda_{kj} = \text{loss for decision } j \text{ when the true class is } k$$

Truth / decision	Class 0	Class 1
Class 0	Λ_{00}	Λ_{01}
Class 1	Λ_{10}	Λ_{11}

k indexes the true class and j the decision. Rows describe truth. Columns describe actions. R is the expected loss of an action.

$$R(j \mid \mathbf{x}) = \sum_{k=0}^{1} \Lambda_{kj}\, p_k(\mathbf{x}) \qquad \widehat{j} = \underset{j}{\operatorname{argmin}}\ R(j \mid \mathbf{x})$$

Decision costs: the diagnosis example

An illustrative loss matrix from the course example

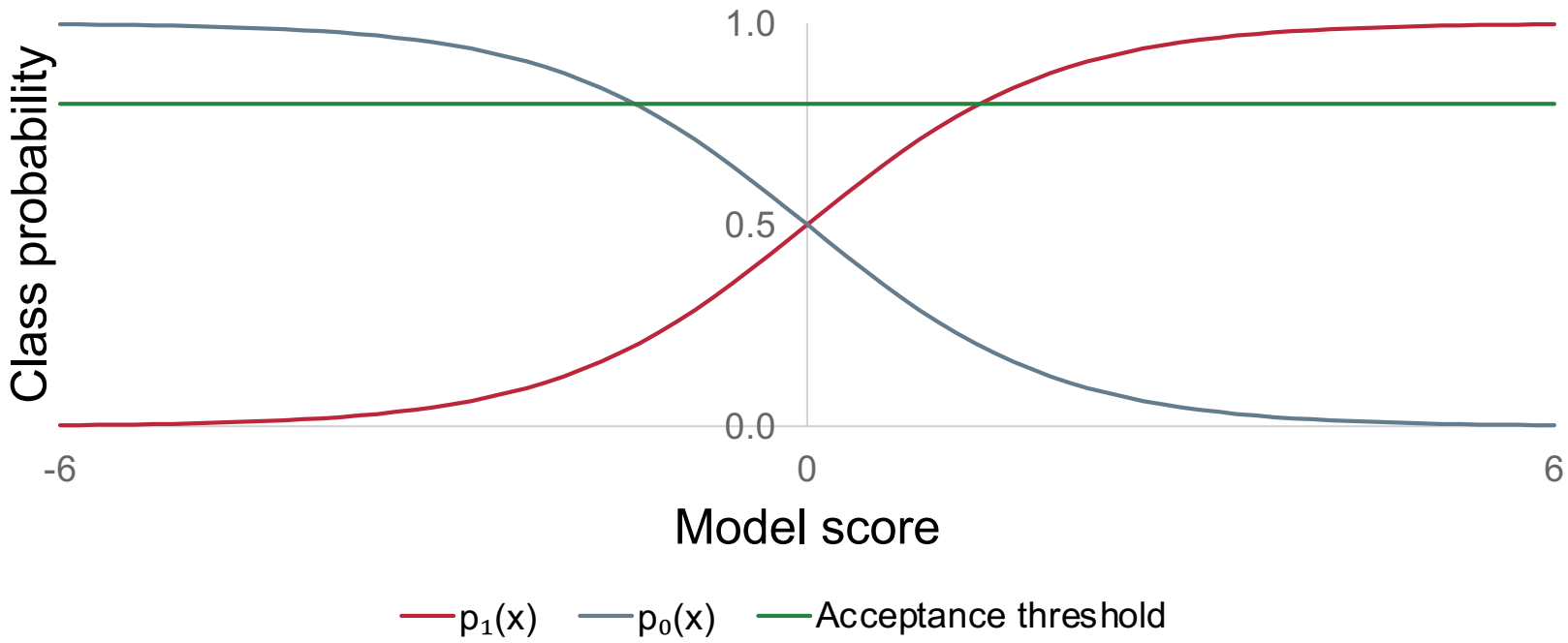
Truth / decision	Cancer (1)	Normal (0)
Cancer (1)	0	1000
Normal (0)	1	0

$$p = p_1(\mathbf{x}) = P(Y = 1 \mid \mathbf{x})$$

$$R(1 \mid \mathbf{x}) = 1 - p, \quad R(0 \mid \mathbf{x}) = 1000p$$

$$\text{Choose class 1 when } p \geq \frac{1}{1001} \approx 0.001$$

A reject option



$$\text{Accept a class if } \max\{p_0(\mathbf{x}), p_1(\mathbf{x})\} \geq \kappa$$

Two classes, three actions: class 0, class 1, or reject.
Choose the most probable class if its probability is at least κ, with $1/2 < \kappa \leq 1$.
Otherwise reject, for further testing or a human decision.

Lecture 2: Classification & optimisation

Classification

Optimisation

Trees and ensembles

Decisions and evaluation

Decision costs

Evaluation metrics

Thresholds and reliability

Held-out counts	Predicted 1	Predicted 0
Actual 1	TP = 3	FN = 1
Actual 0	FP = 2	TN = 2

Murphy, Probabilistic Machine Learning: An Introduction
§§5.1.2, 5.1.4: classification and precision–recall

Confusion matrix

Held-out counts	Predicted 1	Predicted 0
Actual 1	TP = 3	FN = 1
Actual 0	FP = 2	TN = 2

Class 1 is positive. Rows are actual classes and columns are predictions.
TP: true positive. FN: false negative. FP: false positive. TN: true negative.

Total: 8 examples. The matrix shows which kinds of errors occur.

Accuracy

Held-out counts	Predicted 1	Predicted 0
Actual 1	TP = 3	FN = 1
Actual 0	FP = 2	TN = 2

Accuracy counts all correct predictions.
Here, 5 of the 8 held-out examples have the correct label.

$$\text{Accuracy} = \frac{TP + TN}{TP + FN + FP + TN} = \frac{5}{8} = 62.5\%$$

Precision

Held-out counts	Predicted 1	Predicted 0
Actual 1	TP = 3	FN = 1
Actual 0	FP = 2	TN = 2

Among predicted positives, what fraction are truly positive?
False positives reduce precision.

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{3}{5} = 60\%$$

Recall

Held-out counts	Predicted 1	Predicted 0
Actual 1	TP = 3	FN = 1
Actual 0	FP = 2	TN = 2

Among actual positives, what fraction did we find?
False negatives reduce recall.

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{3}{4} = 75\%$$

Precision and recall

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{3}{5} = 60\%$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{3}{4} = 75\%$$

Pattern	Interpretation
High recall, low precision	Finds most positives, with many false alarms
High precision, low recall	Few false alarms, but misses many positives

F-score

Prec is precision and Rec is recall. F_1 is their harmonic mean.

$$F_1 = \frac{2\,\mathrm{Prec}\,\mathrm{Rec}}{\mathrm{Prec} + \mathrm{Rec}} = \frac{2TP}{2TP + FP + FN}$$

$$F_b = (1 + b^2)\frac{\mathrm{Prec}\,\mathrm{Rec}}{b^2\mathrm{Prec} + \mathrm{Rec}}, \qquad b > 0$$

b controls the relative importance of recall and precision.
b > 1 gives recall more weight. b < 1 gives precision more weight.
This b is separate from the momentum coefficient β.

Class imbalance

Inspect class counts and per-class recall alongside aggregate accuracy.

Actual / predicted	Class 1	Class 0
Class 1	70	30
Class 0	10	90

Left: accuracy 80%.
Recall: 70% for class 1, 90% for class 0.

Actual / predicted	Class 1	Class 0
Class 1	700	300
Class 0	100	0

Right: accuracy ≈ 63.6%.
The minority class has zero recall.

A useful-looking accuracy can coexist with failure to detect a class.

Micro and macro averages

ID	Label	Prediction
1	+	+
2	+	×
3	○	○
4	○	○
5	–	–
6	–	+
7	×	×
8	×	+

Macro: equal weight for each class.
Micro: pool counts across classes.

$$P_{\text{macro}} = \frac{1}{C}\sum_{c=1}^{C} P_c$$

$$R_{\text{macro}} = \frac{1}{C}\sum_{c=1}^{C} R_c$$

$$P_{\text{micro}} = \frac{\sum_c TP_c}{\sum_c (TP_c + FP_c)}$$

$$R_{\text{micro}} = \frac{\sum_c TP_c}{\sum_c (TP_c + FN_c)}$$

C is the class count. P_c and R_c are per-class precision and recall.
For single-label predictions over all classes, micro P, R and F_1 equal accuracy.

Lecture 2: Classification & optimisation

Classification

Optimisation

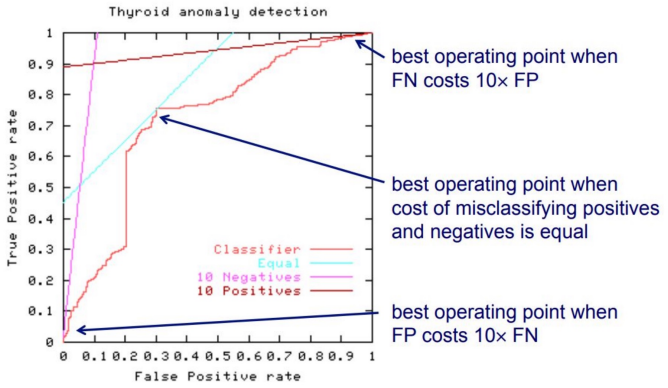
Trees and ensembles

Decisions and evaluation

Decision costs

Evaluation metrics

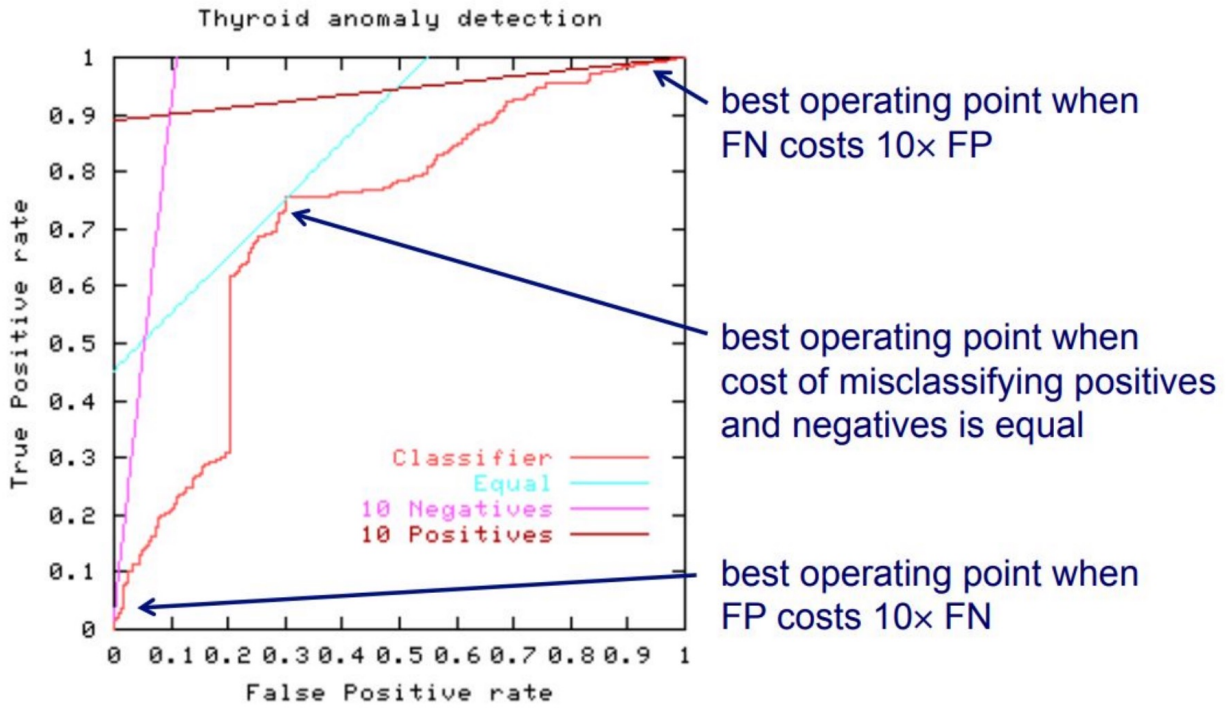
Thresholds and reliability



47

Murphy, Probabilistic Machine Learning: An Introduction
§§5.1.3–5.1.4: ROC and precision–recall curves

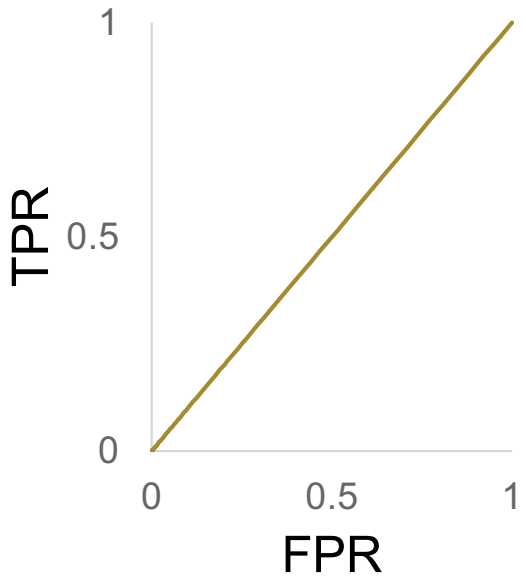
ROC curves and operating points



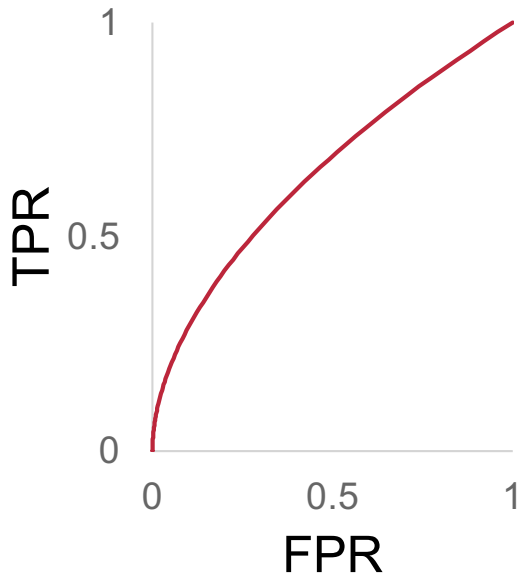
$$TPR = \frac{TP}{TP + FN}, \qquad FPR = \frac{FP}{FP + TN}$$

Sweep the decision threshold. Plot true-positive rate against false-positive rate.
The preferred operating point depends on error costs and class prevalence.

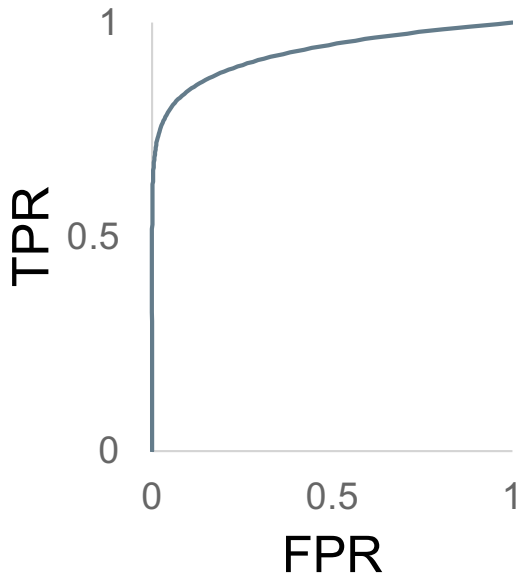
Area under the ROC curve



AUC = 0.50



AUC = 0.65

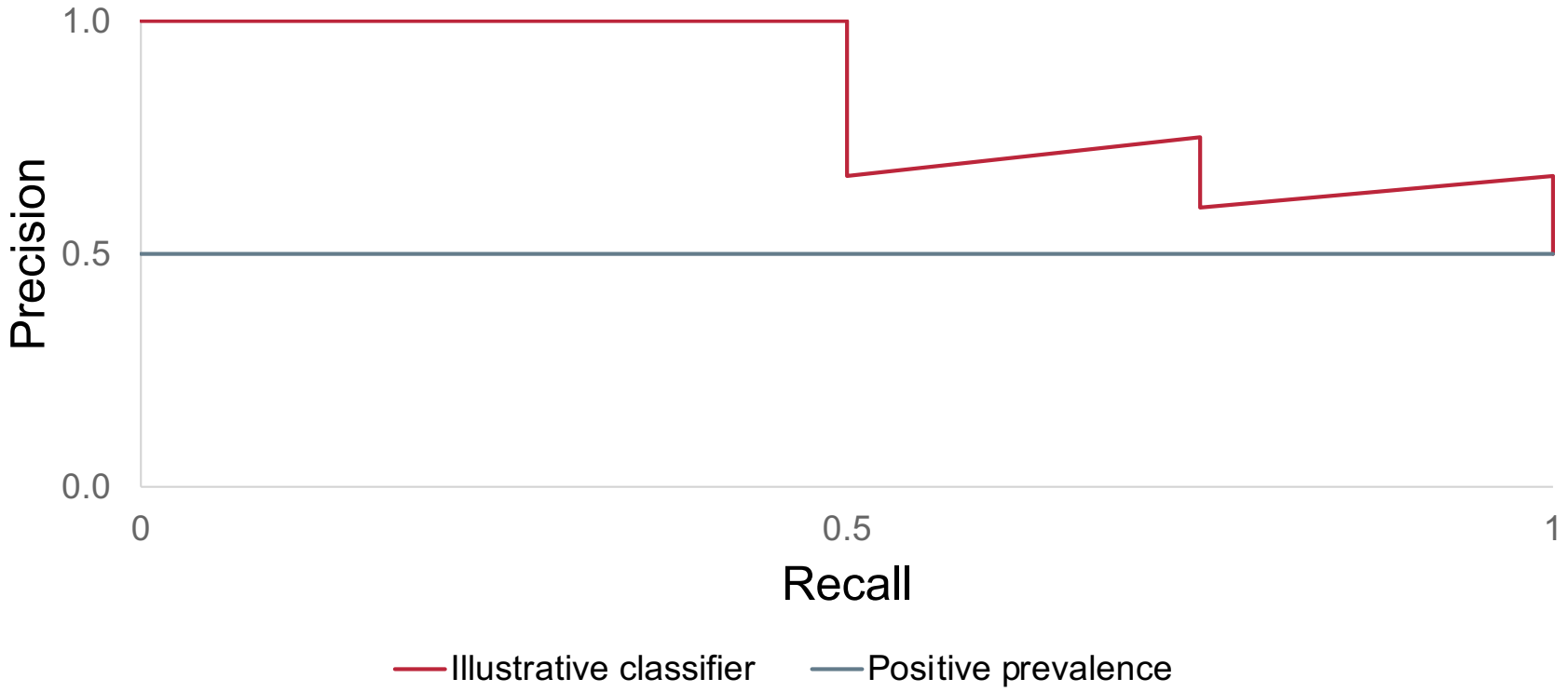


AUC = 0.93

$$\mathrm{AUC} = P(s_+ > s_-) + \tfrac{1}{2}P(s_+ = s_-)$$

s_+ and s_- are scores for a positive and a negative input.
AUC measures ranking across thresholds, with half credit for tied scores.

Precision-recall curves



Plot precision against recall as the threshold changes.
For rare positives, compare against their prevalence in the evaluation set.
This illustration has 4 positive examples among 8 inputs, so prevalence is 0.5.

Confidence intervals

A 95% confidence-interval procedure covers the fixed population quantity in 95% of repeated samples, under its sampling assumptions.

$$\widehat{e} \pm 1.96\sqrt{\frac{\widehat{e}(1-\widehat{e})}{N_{\text{test}}}}$$

ê is the observed error fraction. N_{test} is the number of test examples.
The factor 1.96 is the normal quantile for this approximate 95% interval.

Width depends on sample size and assumptions about independent observations.
Small samples or rates near 0 or 1 may need a better interval method.

Confidence-interval exercise

Original emotion-recognition example: 156 training samples, 50 test samples from three subjects. The rows use different representations and classifiers.

Representation	Attributes	Classes	Classifier	Correct
Face	67	8	C4.5	78%
Body	140	6	BayesNet	90%

For the face result, the observed error fraction is 1 − 0.78 = 0.22.
Treating the 50 observations as independent gives this approximate interval:

$$0.22 \pm 1.96\sqrt{\frac{0.22 \times 0.78}{50}} \approx 0.22 \pm 0.115$$

About [0.105, 0.335]. Dependence between observations changes this calculation.

Comparing classifiers

Evaluate both classifiers on the same held-out examples.

Paired outcome	What it tells us
Both correct or both wrong	No difference on this example
Only classifier A correct	Evidence favouring A
Only classifier B correct	Evidence favouring B

Report the size of the performance difference.
Account for paired predictions, dependence and repeated model selection when assessing statistical evidence.

Probability calibration

Prediction	A calibrated interpretation
Class-1 probability near 0.8	About 80% of these inputs have label 1
Predicted-class confidence near 0.8	About 80% of these predictions are correct

The second statement also applies to multiclass predictions.
High softmax confidence alone does not establish reliability under shift.

Classification: fit, decide, evaluate

Fit a score or class distribution

Choose a loss that suits classification. Optimisation or tree splitting fits the model to the training data.

Choose an action

Class probabilities and error costs determine the decision. A threshold of 1/2 assumes equal costs in the binary case.

Assess unseen performance

Use validation for choices, then independent test data for assessment. Match the metric to the task and check probability calibration.

Lecture 2: Classification & optimisation

Classification

Optimisation

Trees and ensembles

Decisions and evaluation

Appendix

Further derivations and Lecture 1 reference material

Signed-label probabilities

Newton–Raphson for logistic regression

Summation and matrix expressions

Iteratively reweighted least squares (IRLS)

Multiclass objective

Murphy, Probabilistic Machine Learning: An Introduction
§10.2.6: iteratively reweighted least squares

Signed-label probabilities

Recall: $f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ $\quad y_{\pm} = 2y - 1$ $\quad y \in \{0, 1\}$

$y_{\pm} \in \{-1, +1\}$

$$P(Y_{\pm} = +1 \mid \mathbf{x}; \mathbf{w}) = \frac{1}{1 + \exp[-f_{\mathbf{w}}(\mathbf{x})]}$$

$$P(Y_{\pm} = -1 \mid \mathbf{x}; \mathbf{w}) = 1 - P(Y_{\pm} = +1 \mid \mathbf{x}; \mathbf{w})$$

$$= 1 - \frac{1}{1 + \exp[-f_{\mathbf{w}}(\mathbf{x})]}$$

$$= \frac{\exp[-f_{\mathbf{w}}(\mathbf{x})]}{1 + \exp[-f_{\mathbf{w}}(\mathbf{x})]}$$

$$= \frac{1}{1 + \exp[f_{\mathbf{w}}(\mathbf{x})]}$$

Newton–Raphson for logistic regression

Gradient: $$\frac{\partial L}{\partial w_k} = \sum_{i=1}^{N} (p_i - y_i) x_{ik}$$

Hessian:

$$\frac{\partial^2 L}{\partial w_k \partial w_j} = \frac{\partial}{\partial w_j} \left[\sum_{i=1}^{N} (p_i - y_i) x_{ik} \right]$$

$$= \sum_{i=1}^{N} x_{ik} \frac{\partial p_i}{\partial w_j} \qquad = \sum_{i=1}^{N} x_{ik}\, p_i (1 - p_i) x_{ij}$$

$p_i = g(w^T x_i)$. i indexes examples; j and k index coefficients.

Summation- and matrix-based expressions

$$H_{kj} = \frac{\partial^2 L}{\partial w_k \partial w_j} = \sum_{i=1}^{N} x_{ik} p_i (1 - p_i) x_{ij}$$

Matrix version of same result:

$$\mathbf{H}(\mathbf{w}) = \mathbf{X}^T \mathbf{R} \mathbf{X}, \qquad R_{ii} = p_i (1 - p_i)$$

R is an N × N diagonal matrix, with $R_{ii} = p_i(1-p_i)$.
X is N × (D+1); p_i is the predicted probability of label 1.

Newton for logistic regression gives IRLS

At iteration t, evaluate probabilities and curvature weights:

$$\mathbf{p} = g(\mathbf{X}\mathbf{w}_t), \qquad \mathbf{R} = \mathrm{diag}[p_i(1-p_i)]$$

Substitute $\nabla L = X^T(p - y)$ and $H = X^TRX$ into the Newton step:

$$(\mathbf{X}^T\mathbf{R}\mathbf{X})\mathbf{w}_{t+1} = \mathbf{X}^T\mathbf{R}\mathbf{X}\mathbf{w}_t + \mathbf{X}^T(\mathbf{y} - \mathbf{p})$$

Define a working response u. The same update solves weighted least squares:

$$\mathbf{u} = \mathbf{X}\mathbf{w}_t + \mathbf{R}^{-1}(\mathbf{y} - \mathbf{p})$$

$$\mathbf{w}_{t+1} = \arg\min_{\mathbf{w}} \frac{1}{2}(\mathbf{u} - \mathbf{X}\mathbf{w})^T\mathbf{R}(\mathbf{u} - \mathbf{X}\mathbf{w})$$

Training objective, multiclass case

One-hot target vector: $\mathbf{t}_i = (0, 0, 1, 0)^T$

Likelihood of training sample: $(\mathbf{x}_i, y_i), \quad y_i = 3$

$$P(Y = y_i \mid \mathbf{x}_i; \mathbf{W}) = \prod_{c=1}^{C} p_c(\mathbf{x}_i)^{t_{ic}}$$

Optimization criterion:

$$L(\mathbf{W}) = -\sum_{i=1}^{N} \sum_{c=1}^{C} t_{ic} \log p_c(\mathbf{x}_i)$$

Fit W by minimising this summed cross-entropy objective.

Expectation and a sample average

Z has density p(z). ψ(z) is a quantity computed from a value z.

Distribution: average over possible values

$$\mathbb{E}[\psi(Z)] = \int \psi(z)\, p(z)\, dz$$

Sample: estimate from N independent observations

$$\widehat{\mathbb{E}}[\psi(Z)] = \frac{1}{N} \sum_{i=1}^{N} \psi(z_i)$$

zi: observations of Z. With ψ(z) = z, this is the sample mean.

Recall: empirical risk in regression

Training set D: N pairs with input x_i and target y_i, indexed by i.
f predicts a target; $\hat{y}$ is a prediction. ℓ is the squared loss.

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N} \qquad \ell(y, \hat{y}) = (y - \hat{y})^2$$

Training risk: mean squared error on the training set

$$\hat{R}_{\text{train}}(f) = \frac{1}{N} \sum_{i=1}^{N} \ell(y_i, f(\mathbf{x}_i))$$

ERM: choose a function with minimum training risk

$$\hat{f}_{\mathcal{D}} \in \arg\min_{f \in \mathcal{F}} \hat{R}_{\text{train}}(f)$$

F: candidate function class. arg min selects a minimiser.
Subscript D: the training set used for this fit.

Population risk: the quantity we care about

P: real-world distribution. (X, Y): a fresh pair, independent of training.
Hold the fitted function fixed. R(f) is its population risk.

$$R(f) = \mathbb{E}_{(\mathbf{X},Y)\sim P}\big[\ell(Y, f(\mathbf{X}))\big]$$

$$= \int\!\!\int \ell(y, f(\mathbf{x}))\, p(\mathbf{x}, y)\, \mathrm{d}y\, \mathrm{d}\mathbf{x}$$

p(x, y): joint density for continuous inputs and targets.
The integral averages loss over real-world cases.

The population distribution is unknown.
Validation data will estimate unseen performance.

Validation estimates unseen performance

Reserve fresh validation pairs from the same population.
V: validation sample. N_{val}: its size. Superscript val marks its values.

$$\mathcal{V} = \{(\mathbf{x}_i^{\text{val}}, y_i^{\text{val}})\}_{i=1}^{N_{\text{val}}}$$

Evaluate the trained predictor without refitting it

$$\hat{R}_{\text{val}}(\hat{f}_{\mathcal{D}}) = \frac{1}{N_{\text{val}}} \sum_{i=1}^{N_{\text{val}}} \ell(y_i^{\text{val}}, \hat{f}_{\mathcal{D}}(\mathbf{x}_i^{\text{val}}))$$

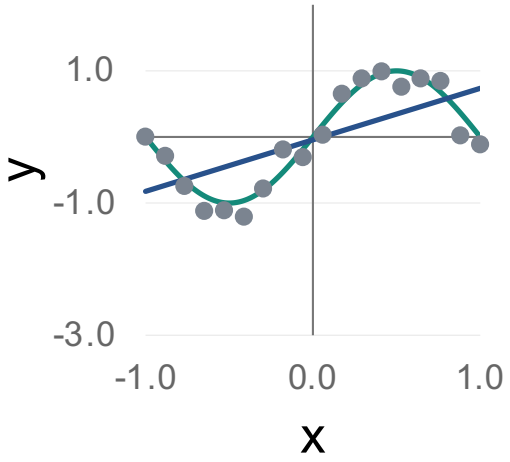
Hold training set D and the fitted model fixed:

$$\mathbb{E}_{\mathcal{V}}[\hat{R}_{\text{val}}(\hat{f}_{\mathcal{D}}) \mid \mathcal{D}] = R(\hat{f}_{\mathcal{D}})$$

E with subscript V averages validation samples, before model selection.
Use validation to choose complexity. Keep test data for final assessment.

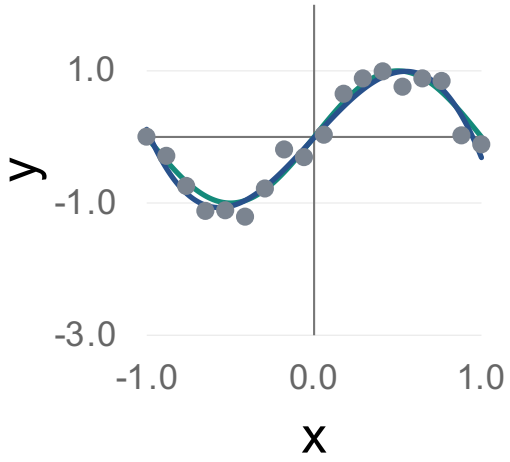
Training fit and unseen predictions

Degree 1



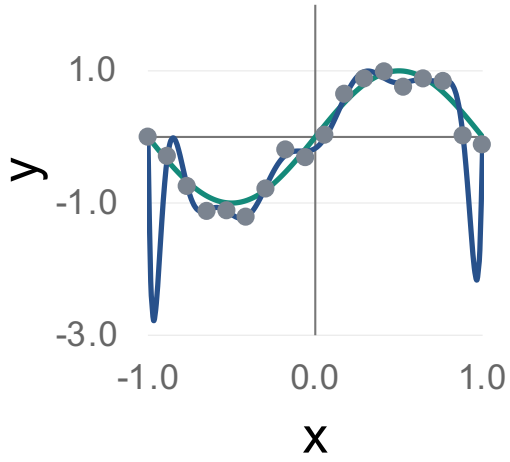
Train MSE: 0.305
Validation MSE: 0.245

Degree 3



Train MSE: 0.029
Validation MSE: 0.057

Degree 14



Train MSE: 0.002
Validation MSE: 0.485

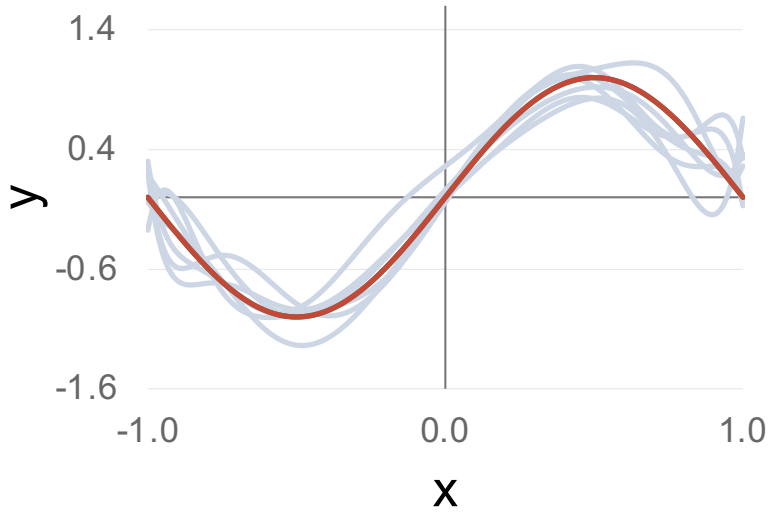
Same training set and 200 independent validation pairs for every model.
Grey points: training data. Blue: fitted predictor. Green: true mean.

The most flexible model wins on training error, but loses on validation.

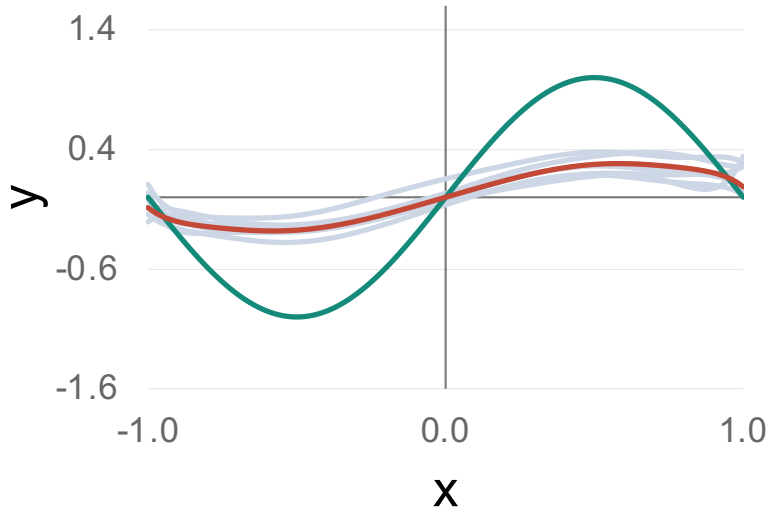
What does “averaging over datasets” mean?

Repeat collecting N examples from the same source and refitting.
Keep the function class and ridge strength λ fixed within each plot.

Weak ridge: λ = 0.0001



Strong ridge: λ = 10



At scalar input x: grey fits, red mean fit m(x), green true mean f*(x).

$$m(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)], \qquad f^*(x) = \mathbb{E}[Y \mid X = x]$$

E with subscript D averages training sets.
Here: fixed inputs, fresh noisy targets. With random inputs, resample those too.

Bias–variance decomposition at one input

Fix x. Fresh target noise ε is independent of the training data.
σ²(x) is the noise variance at x. Keep the training method fixed.

$$Y = f^*(x) + \epsilon, \quad \mathbb{E}[\epsilon \mid x] = 0, \quad \mathbb{E}[\epsilon^2 \mid x] = \sigma^2(x)$$

Add and subtract the true mean and the mean fit

$$Y - \hat{f}_{\mathcal{D}}(x) = \underbrace{f^*(x) - m(x)}_{\text{mean error}} + \underbrace{m(x) - \hat{f}_{\mathcal{D}}(x)}_{\text{fit fluctuation}} + \epsilon$$

Square and average. Cross-terms vanish: the fluctuations have mean zero, and fresh noise is independent of the fitted predictor.

$$\mathbb{E}_{\mathcal{D}}[m(x) - \hat{f}_{\mathcal{D}}(x)] = 0, \qquad \mathbb{E}[\epsilon \mid x] = 0$$

$$\mathbb{E}_{\mathcal{D},Y}[(Y - \hat{f}_{\mathcal{D}}(x))^2 \mid X = x]$$

$$= \underbrace{(m(x) - f^*(x))^2}_{\text{bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - m(x))^2]}_{\text{variance}} + \underbrace{\sigma^2(x)}_{\text{noise}}$$

Bias–variance explains population risk

Population risk averages fresh cases for one trained predictor.
Now also average over possible training sets D.

$$\mathbb{E}_{\mathcal{D}}[R(\hat{f}_{\mathcal{D}})] = \mathbb{E}_{\mathbf{X}}[\text{bias}^2(\mathbf{X}) + \text{variance}(\mathbf{X}) + \text{noise}(\mathbf{X})]$$

Average these terms over real-world inputs X.
This connects the fixed-input decomposition to population risk.

Validation estimates the risk of a particular fitted model

Bias–variance describes average risk as the training sample changes.
Validation estimates risk for the fitted model obtained from one sample.

A controlled ridge experiment

$$y_i = \sin(\pi x_i) + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

18 fixed inputs across [−1, 1]. True mean: sin(πx).
Redraw independent Gaussian noise ε_i and refit on each new dataset.
σ is the noise standard deviation. Compare σ = 0.25 with σ = 1.

Degree-9 polynomial model, same fitting objective

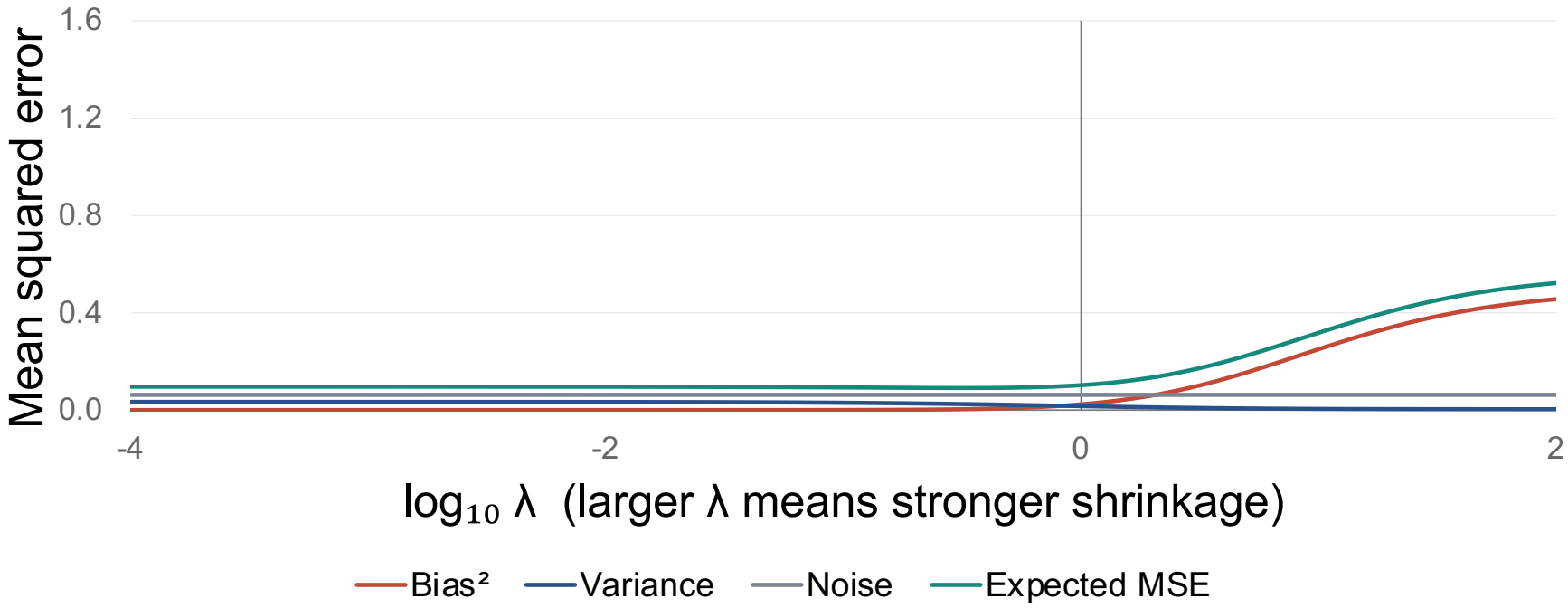
$$J(\mathbf{w}) = \|\mathbf{y} - \mathbf{\Phi w}\|_2^2 + \lambda \sum_{j=1}^{9} w_j^2$$

J: fitting cost. Φ: polynomial design matrix. y: target vector.
w: coefficients. λ: shrinkage strength. Intercept w_0 is unpenalised.

Curves average repeated datasets and fresh noise at 121 fixed inputs.

Ridge strength and expected error

Noise standard deviation σ = 0.25. All other conditions stay fixed.

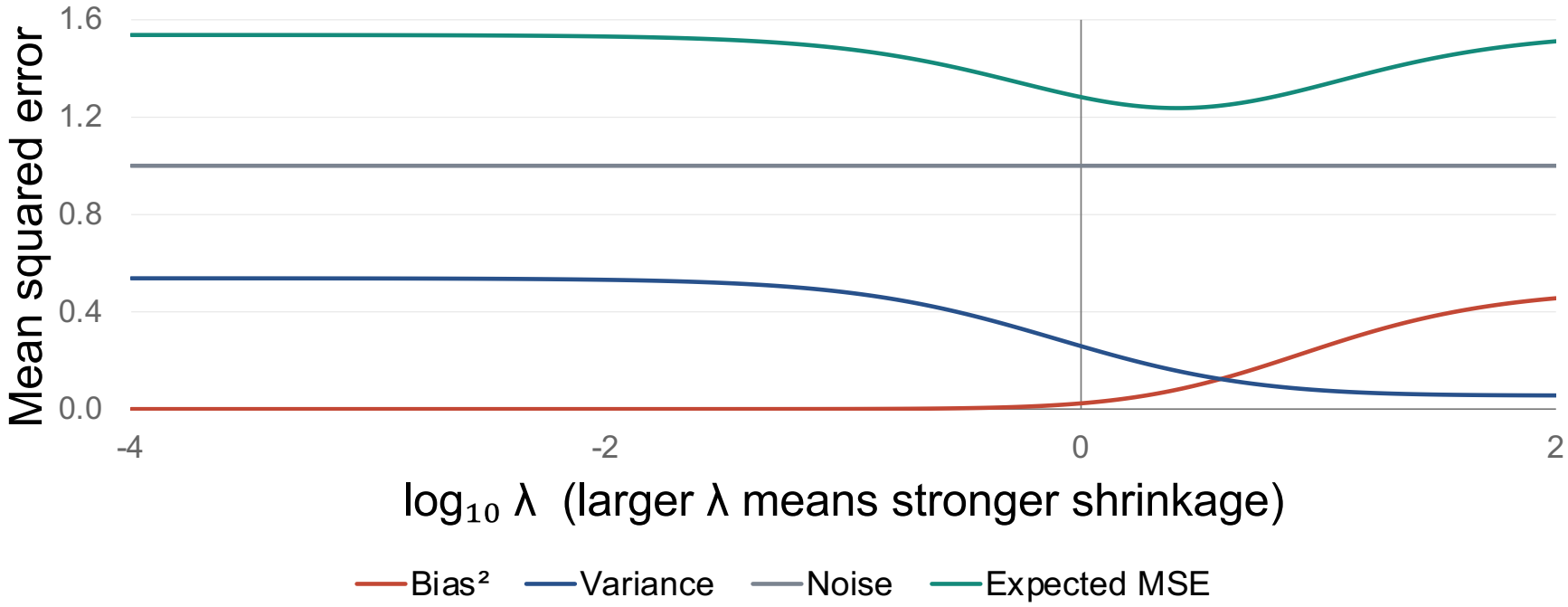


Minimum on this grid: λ ≈ 0.32, expected MSE ≈ 0.091.

Stronger shrinkage reduces variance but eventually adds too much bias.
An optimum need not lie in the interior for every problem.

More noise changes the ridge trade-off

Noise standard deviation σ = 1.00. All other conditions stay fixed.



Minimum on this grid: λ ≈ 2.51, expected MSE ≈ 1.237.

More noise raises the variance and noise floor. In practice, choose λ using validation. These curves use known simulated truth.

Binary objective recap

Training data: $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ Fit: $\mathbf{w}$

Loss measures fit as a function of the parameters.

$$L(\mathbf{w}) = -\log p(\mathbf{y} \mid \mathbf{X}; \mathbf{w})$$

$$= -\sum_{i=1}^{N} \{y_i \log p_i + (1 - y_i) \log(1 - p_i)\}$$

Bernoulli model for posterior distribution:

$$P(Y = y \mid \mathbf{x}; \mathbf{w}) = p_{\mathbf{w}}(\mathbf{x})^y [1 - p_{\mathbf{w}}(\mathbf{x})]^{1-y}$$

A mean objective for stochastic updates

$$J(\mathbf{w}) = \frac{1}{N} L(\mathbf{w}) + \frac{\lambda}{2} \sum_{j=1}^{D} w_j^2$$

L is the summed training loss used earlier. J is its mean plus a coefficient penalty; $\lambda \geq 0$ sets the penalty strength.

The intercept w_0 is not penalised.
The factor 1/2 makes the penalty derivative λw_j.

A full gradient uses all N examples before each update.
A mini-batch estimates it using only B examples.

Splitting the examples at a node

At node n, I_n indexes labelled training examples (x_i, y_i).
$P_n(c)$ is the histogram of their labels: the fraction belonging to class c.

$$P_n(c) = \frac{1}{|I_n|} \sum_{i \in I_n} \mathbb{I}[y_i = c]$$

$$I_l = \{i \in I_n : h(\mathbf{x}_i) < \theta\}$$

$$I_r = I_n \setminus I_l$$

$h(x_i)$ tests a feature of example i. θ is its threshold.
Each example follows one branch. Recursively split the child sets.

Learning a Forest

Draw a bootstrap sample for each tree

Sample training examples with replacement.
Train one decision tree on each sample.

Use random feature subsets when choosing splits.
Combine the trees' votes or class probabilities.

Randomisation reduces correlation between trees.

More Randomized Learning

left split $I_{\mathrm{L}} = \{i \in I_n \mid h_n(\mathbf{x}_i) < \theta_n\}$

right split $I_{\mathrm{R}} = I_n \setminus I_{\mathrm{L}}$

- Choose candidate feature tests h_n at random from a feature pool.

- Choose thresholds θ_n within the observed feature-test range.

$$\theta_n \in \left(\min_{i \in I_n} h_n(\mathbf{x}_i), \max_{i \in I_n} h_n(\mathbf{x}_i) \right)$$

- Keep the test and threshold giving the greatest information gain.

Reading alongside Lecture 2

Murphy, Probabilistic Machine Learning: An Introduction (book1.pdf)

Classification

§§10.2.1–10.2.3, 10.3.1–10.3.2

Vision Book: §§9.6, 9.7.3 — classification and cross-entropy

Optimisation

§§8.2.1–8.2.4, 8.3.1, 8.4.1–8.4.3; §§11.4.1–11.4.2 (L1)

Vision Book: §§10.3–10.5, 10.7 — gradient descent, momentum and SGD

Trees and ensembles

§§18.1, 18.3–18.4

Decisions and evaluation

§§5.1.1–5.1.4

Vision Book: Foundations of Computer Vision — Torralba, Isola and Freeman
visionbook.mit.edu

Notation: models and optimisation

Data and model

$N,\ D,\ M$

Examples; raw/mapped size

$\mathbf{x}_i,\ y_i$

Example i: input and observed label

$\boldsymbol{\phi}(\mathbf{x}),\ \mathbf{w}$

Features and coefficients

$z_i,\ g$

Score and logistic sigmoid

$p_i,\ p_c(\mathbf{x})$

Class-1 and class-c probabilities

$y_{\pm},\ m = y_{\pm} z$

Signed label (±1) and margin

Fitting and optimisation

$\ell_i,\ L,\ J$

Example loss; total loss; objective

$\lambda,\ \mathbf{r}(\mathbf{w})$

Penalty strength; penalty gradient

$\nabla_{\mathbf{w}} J,\ \mathbf{H}$

Gradient and Hessian

$t,\ \eta_t,\ \alpha_t$

Iteration; learning rate; step scale

$\mathcal{B}_t,\ B$

Mini-batch indices and size

$\mathbf{g}_t,\ \mathbf{v}_t,\ \beta$

Gradient estimate; momentum; decay

Notation: trees and decisions

Trees and ensembles

$C,\ K$

Classes; learners

$f_k,\ \omega_k$

Learner and its ensemble weight

$\mathcal{R},\ p_c(\mathbf{x})$

Leaf region and class probability

$\theta_n,\ H,\ \mathrm{IG}$

Split threshold; entropy; split gain

$f^*,\ \epsilon_k$

Target function; learner error

$R(f),\ \widehat{R}_{\mathrm{train}}$

Population risk; training risk

Decisions and evaluation

$\tau,\ \kappa$

Decision and acceptance thresholds

$\Lambda_{kj},\ R(j \mid \mathbf{x})$

Cost (truth k, action j); risk

$\mathrm{TP},\ \mathrm{FP},\ \mathrm{TN},\ \mathrm{FN}$

True/false positives and negatives

$\mathrm{Prec},\ \mathrm{Rec},\ F_b$

Precision, recall, weighted F-score

$\mathrm{TPR},\ \mathrm{FPR},\ \mathrm{AUC}$

Positive rates; area under ROC

$e,\ N_{\mathrm{test}}$

Test error fraction; test-set size